Table Of Contents

1. ***Title of the Project***: - **INCOME TAX EVALUATION & MAINTENANCE SYSTEM**

2. ***Introduction & Objectives of the project: -***

   a. **Introduction: -** ITEMS i.e. **INCOME TAX EVALUATION & MAINTENANCE SYSTEM**, is application software, which implements the basics of Income Tax department.

   b. **Objectives:** - The aim and objectives of an important project such as this are multidimensional as it involves the theoretical and practical knowledge, believes, biases and most of all, the attitude earned during the course, and this project is no exception.

      The most important objectives of this project are as follows: -

      i. Maintain the records of each client.

      ii. Based upon the category of each client, provides the forms to file their original return for specific fiscal.

      iii. Provides the facility to file the revised return in the case of any mistake in original return.

      iv. For firms, it provides the facility to maintain their records regarding Trading A/c, P/L A/c & Balance Sheet.

      v. This project will generate reports for: -

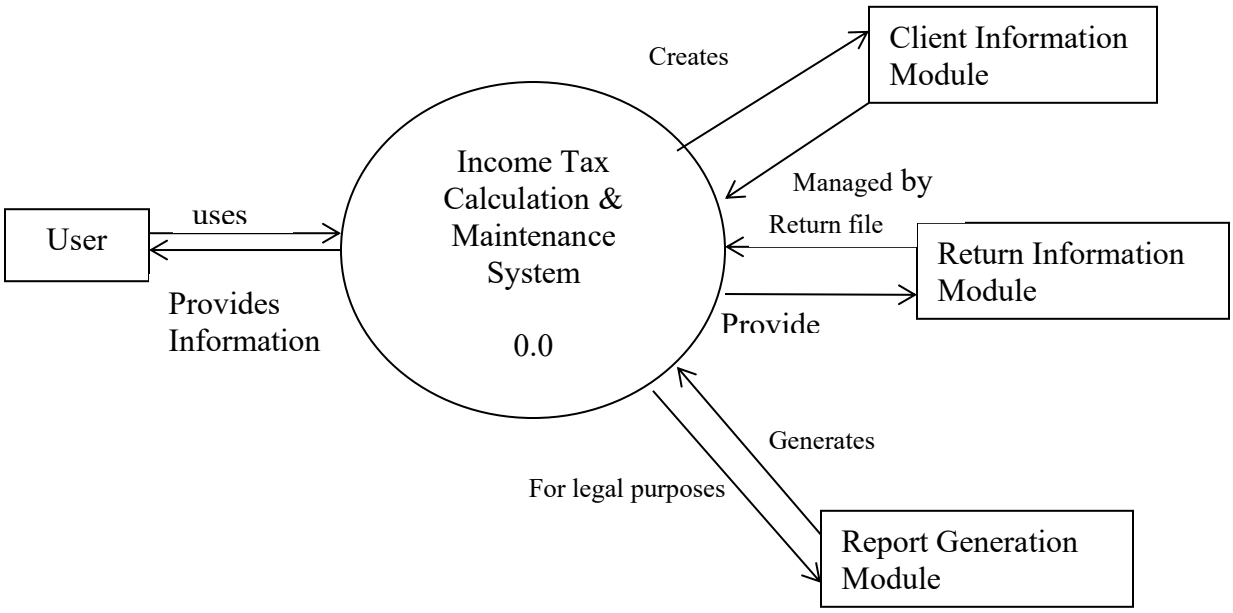
         ✓ Return history of particular client.

- ✓ Return filed in a particular fiscal
- ✓ Total revised return filed by a particular client

3. **Project Category**: - OOPS

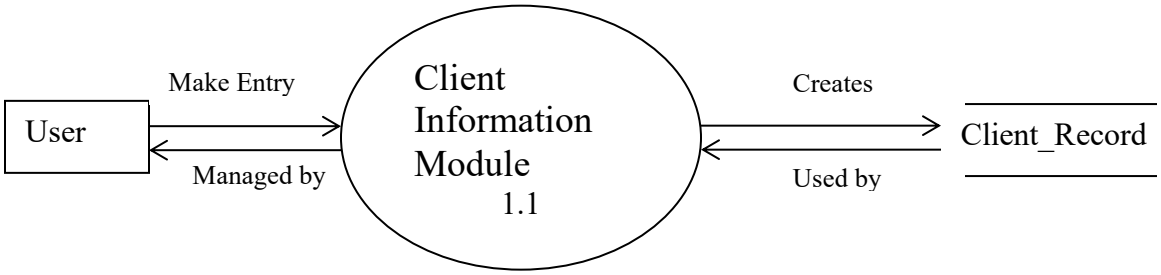
4. a. DFD: -

A. Context Level DFD: -

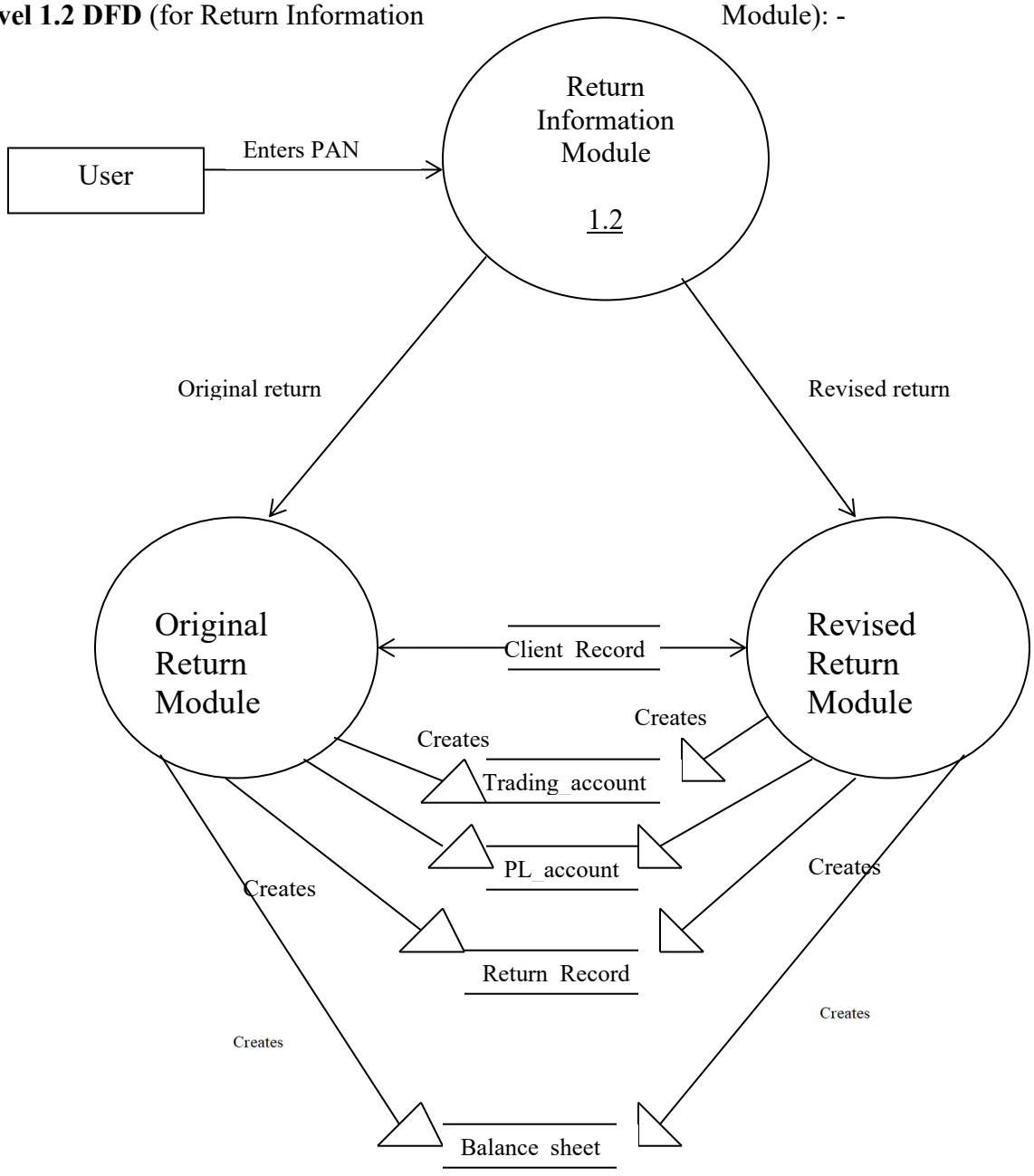


1st Level DFD: -

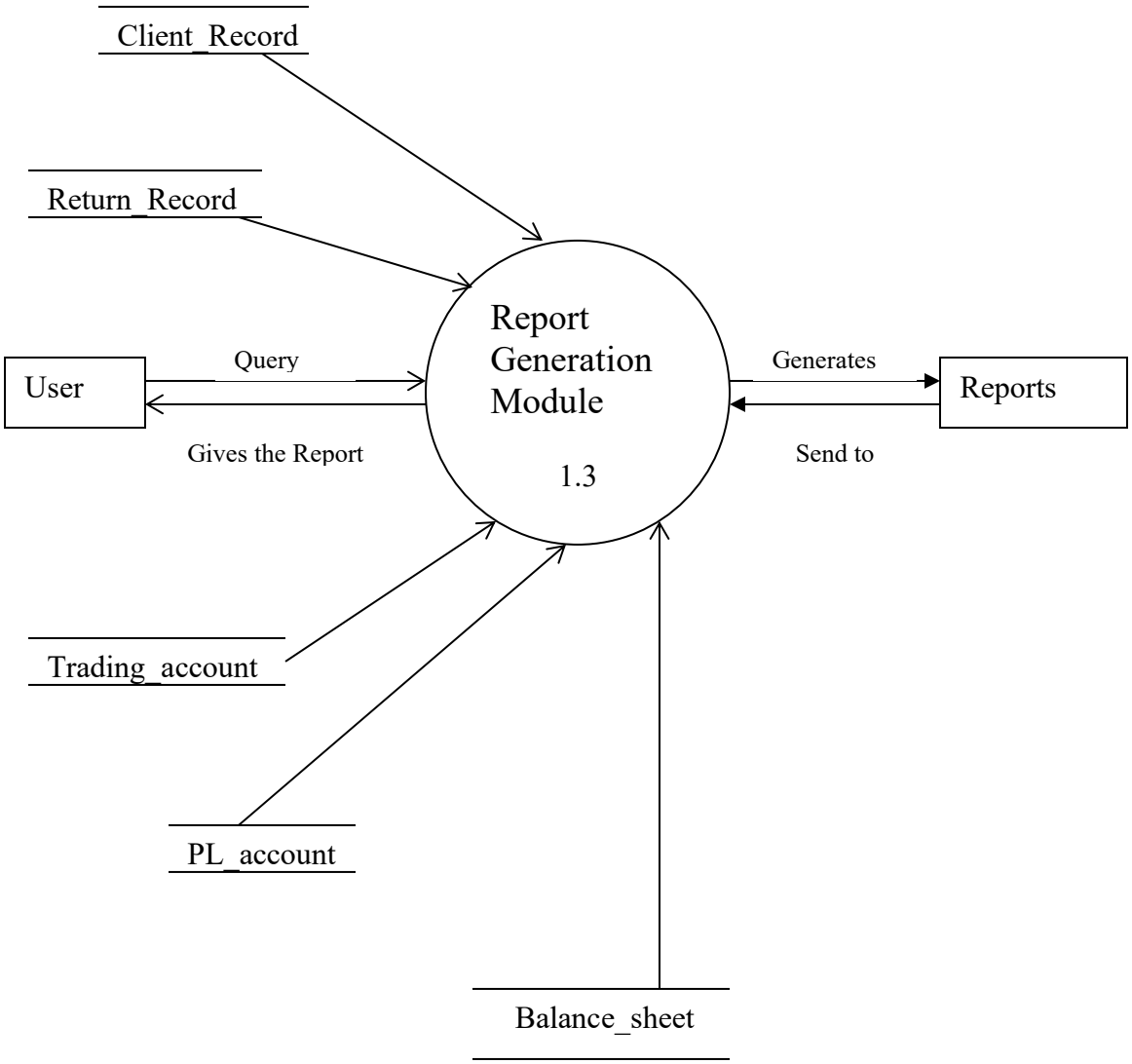
Level 1.1 DFD (for Client Information Module): -+



Level 1.2 DFD (for Return Information Module): -



Level 1.3 DFD (for Report Generation Module): -



5. **a. <u>Modulus and its description</u>**: -

My project has seven modules. Description of each module is as follows: -

Client Information Module: - This module will maintain the basic information of each client such as PAN, NAME FATHER'S NAME, and DOB etc. This module will be responsible for addition, deletion, and update ion of each client's record.

Original Return Module: - This module will ensure the filing of original return of particular client for particular assessment year. This module will consult the client information module for verification and fetching the records of particular employee.

Revised Return Module: - This module will handle the filling of revised return for a particular assessment year for a particular employee.

Trading Account module: - If the client is a firm, then it will be essential to keep track of their Trading account. And in this proposed software, this very purpose will served by trading account module.

Profit and Loss Account Module: - If the client is firm, then the record of their profit & loss account will be essential for future use and this function is served by this very account.

Balance Sheet Module: - If the client is firm, then together with Trading A/c & Profit & Loss A/c, record regarding their Balance Sheet will be needed as well and this module will serve this purpose.

Report Generation Module: - Report generation is one of the most obvious purposes of any software and this software is not an exception. This software will generate 4 types of reports, as follows: -

- ✓ Return history of particular client.
- ✓ Return filed by a particular client in a particular fiscal.

- ✓ Total return filed in a particular fiscal.
- ✓ Total revised return filed by a particular client.

b. **Data Structure**: -

CLIENT_RECORD

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Primary key
NAME	Varchar2	30	Name of client
FATHERS_NAME	Varchar2	30	Father's name
DOB	Date		Date of birth
ADDRESS	Varchar2	40	Address of client
TELEPHONE	Varchar2	12	Contact number
SEX	Boolean		Male or female
INDV_HUF_FIRM_AOP_LA	Number	1	Category of client
RESIDENT_NR_NOR	Number	1	About residence
WARD_CIRCLE_SPECIAL_RANGE	Varchar2	20	Client's ward

INCOME_TAX_RECORD

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
INCOME_FOR_PREV_YEAR	Number	(15,2)	Previous income
RETURN_ORIGINAL_REVISED	Boolean		Original or revised
INCOME_FROM_SALARY	Number	(15,2)	Salary from income
INCOME_FROM_HOUSE_PROPERTY	Number	(15,2)	Income from house
INCOME_FROM_BUSINESS_OR_PROFESSION	Number	(15,2)	Income from business
TOTAL_SHORT_TERM_GAIN	Number	(15,2)	Short term gain
TOTAL_LONG_TERM_GAIN	Number	(15,2)	Long term gain
TOTAL_CAPITAL_GAINS	Number	(15,2)	Capital gain
INCOME_FROM_OTHER_SOURCES	Number	(15,2)	From other sources
INCOME_OF_OTHER_PERSON_TO_BE_ADDED	Number	(15,2)	Income of other person to be added
GROSS_TOTAL_INCOME	Number	(15,2)	Gross total income
LESS_DEDUCTION_UNDER_VIA	Number	(15,2)	Deduction under Via

TOTAL_INCOME	Number	(15,2)	Total income
AGRICULTURAL_INCOME	Number	(15,2)	Agricultural income
INCOME_TO_BE_EXEMPTED	Number	(15,2)	Income to be exempted
INCOME_AT_NORMAL_RATES	Number	(15,2)	Income at normal rates
INCOME_TAX_AT_NORMAL_RATES	Number	(15,2)	Income at normal rate
INCOME_AT_SPECIAL_RATES	Number	(15,2)	At special rate
INCOME_TAX_AT_SPECIAL_RATES	Number	(15,2)	Tax at special rate
TAX_ON_TOTAL_INCOME	Number	(15,2)	Tax on total income
LESS_REBATE	Number	(15,2)	Rebate
TAX_PAYABLE	Number	(15,2)	Tax payable
ADDITIONAL_SURCHARGE	Number	(15,2)	Additional surcharge
TOTAL_TAX_PAYABLE	Number	(15,2)	Total payable tax
LESS_RELIEF	Number	(15,2)	Relief
NET_TAX_PAYABLE	Number	(15,2)	Net tax payable
TAX_DEDUCTED_AT_SOURCES	Number	(15,2)	Deduction at sources
ADVANCE_TAX_PAID	Number	(15,2)	Tax paid in advance
INTEREST_PAYABLE	Number	(15,2)	Interest payable
SELF_ASSESSMENT_PAID_UPTO_31_MAY	Number	(15,2)	Self assessment paid up to 31st May
SELF_ASSESSMENT_PAID_AFTER_31_MAY	Number	(15,2)	Self assessment paid after 31st May
TOTAL_SELF_ASSESSMENT_TAX_PAID	Number	(15,2)	Total self assessment tax paid
BSR_CODE_OF_BANK_BRANCH	Varchar2	15	BSR code of bank
DATE_OF_DEPOSIT	Date		Date of deposit
SERIAL_NO_OF_CHALLAN	Varchar2	10	Serial no of Chelan
AMOUNT	Number	(15,2)	Amount
NAME_OF_BANK_FOR_ATTACHED_CHALLAN	Varchar2	20	Name of bank for attached Chelan
BALANCE_TAX_PAYABLE_REFUNDABLE	Number	(15,2)	Balance tax payable refundable
BALANCE_TAX	Number	(15,2)	Balance tax
ENCLOSURE1	Varchar2	20	1st enclosure
ENCLOSURE2	Varchar2	20	2nd enclosure
ENCLOSURE3	Varchar2	20	3rd enclosure
ENCLOSURE4	Varchar2	20	4th enclosure
ENCLOSURE5	Varchar2	20	5th enclosure
ENCLOSURE6	Varchar2	20	6th enclosure

TRADING_ACCOUNT

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
TO_OPENING_STOCK	Number	(15,2)	Opening stock
TO_STOCK	Number	(15,2)	Stock
TO_PURCHASES	Number	(15,2)	Purchases
TO_CARRIAGE	Number	(15,2)	Carriage
TO_OCTROI	Number	(15,2)	Octroi
TO_IMPORT_DUTY_CUSTOMS	Number	(15,2)	Import duty & custom
TO_WAGES	Number	(15,2)	Wages
TO_COAL_WATER_GAS	Number	(15,2)	Coal, water & gas
TO_HEATING_LIGHTING_POWER	Number	(15,2)	Heating, light & power
TO_MANUFACTURING_ASSEMBLING_EXPENSES	Number	(15,2)	Manufacturing & assembling expenses
TO_CONSUMABLE_STORES	Number	(15,2)	Consumable stores
TO_DIRECT_FACTORY_PRODUCTIVE_EXPENSES	Number	(15,2)	Direct factory product expenses
TO_ROYALTY	Number	(15,2)	Royalty
TO_GROSS_PROFIT	Number	(15,2)	Gross profit
BY_SALES	Number	(15,2)	Sales
BY_CLOSING_STOCK	Number	(15,2)	Closing stock
BY_STOCK	Number	(15,2)	Stock
BY_GROSS_LOSS	Number	(15,2)	Gross loss

PL_ACCOUNT

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
TO_GROSS_LOSS	Number	(15,2)	Gross loss
TO_SALARIES_WAGES	Number	(15,2)	Salaries & wages
TO_OFFICE_GODOWN_RENT	Number	(15,2)	Office & go down rent
TO_OFFICE_EXPENSES	Number	(15,2)	Office expenses
TO_MISCELLANEOUS_SUNDRY_EXPENSES	Number	(15,2)	Miscellaneous & sundry expenses
TO_INSURANCE	Number	(15,2)	Insurance
TO_STATIONERY_PRINTING	Number	(15,2)	Stationery expenses
TO_STAFF_WELFARE_EXPENSES	Number	(15,2)	Staff welfare expenses

TO_LIGHTING_WATER_ELECTRICITY	Number	(15,2)	Lighting, water & electricity
TO_ESTABLISHMENT_EXPENSES	Number	(15,2)	Establishment expenses
TO_POSTAGE_TELEGRAM_FAX_COURIER_TELEPHONE	Number	(15,2)	Postage, telegram, fax, courier & telephone
TO_LAW_CHARGES	Number	(15,2)	Law charges
TO_REPAIRS	Number	(15,2)	Repairs
TO_DISTRIBUTION_EXPENSES	Number	(15,2)	Distribution expenses
TO_TRAVELLING_EXPENSES	Number	(15,2)	Traveling expenses
TO_GENERAL_EXPENSES	Number	(15,2)	General expenses
TO_STABLE_EXPENSES	Number	(15,2)	Stable expenses
TO_SELLING_EXPENSES	Number	(15,2)	Selling expenses
TO_CARRIAGE_OUTWARD	Number	(15,2)	Carriage outward
TO_CARRIAGE_ON_SALES	Number	(15,2)	Carriage on sales
TO_INDIRECT_WAGES	Number	(15,2)	Indirect wages
TO_AUDIT_FEES	Number	(15,2)	Audit fees
TO_ENTERTAINMENT_EXPENSES	Number	(15,2)	Entertainment expenses
TO_INTEREST_PAID	Number	(15,2)	Interest paid
TO_DISCOUNT_ALLOWED	Number	(15,2)	Discount allowed
TO_BAD_DEBTS	Number	(15,2)	Bad debts
TO_RESERVE_FOR_BAD_DEBTS	Number	(15,2)	Reserve for bad debts
TO_DEPRECIATION	Number	(15,2)	Depreciation
TO_INTEREST_ON_CAPITAL	Number	(15,2)	Interest on capital
TO_DISCOUNTING_CHARGES	Number	(15,2)	Discounting charges
TO_BANK_CHARGES	Number	(15,2)	Bank charges
TO_EXPORT_CHARGES	Number	(15,2)	Export charges
TO_TRADE_EXPENSES	Number	(15,2)	Trade expenses
TO_ADMINISRTATIVE_EXPENSES	Number	(15,2)	Administrative expense
TO_FINANCIAL_EXPENSES	Number	(15,2)	Financial expenses
TO_COMMISSION_PAID	Number	(15,2)	Commission paid
TO_ADVERTISEMENT	Number	(15,2)	Advertisement
TO_CHARITY	Number	(15,2)	Charity
TO_SAMPLE_EXPENSES	Number	(15,2)	Sample expenses
TO_LICENCE_FEE	Number	(15,2)	License fee
TO_DELIVERY_CHARGES	Number	(15,2)	Delivery charges
TO_BROKERAGE	Number	(15,2)	Brokerage
TO_SALES_TAX	Number	(15,2)	Sales tax
TO_LOSS_ON_SALE_OF_ASSETS	Number	(15,2)	Loss on sale of assets
TO_LOSS_BY_FIRE_THEFT_ACCIDENT	Number	(15,2)	Loss by fire, theft or accident
TO_NET_PROFIT	Number	(15,2)	Net profit
BY_GROSS_PROFIT	Number	(15,2)	Gross profit
BY_INTEREST_RECEIVED	Number	(15,2)	Interest received
BY_RENT_RECEIVED	Number	(15,2)	Rent received

BY_DISCOUNT_RECEIVED	Number	(15,2)	Discount received
BY_DIVIDENDS_RECEIVED	Number	(15,2)	Dividend received
BY_PROFIT_FROM_SALES_OF_ASSETS	Number	(15,2)	Profit from sales of assets
BY_REFUND_OF_TAX	Number	(15,2)	Refund of tax
BY_COMPENSATION_RECEIVED	Number	(15,2)	Compensation received
BY_APPRENTICESHIP_PREMIUM	Number	(15,2)	Apprenticeship premium
BY_DIFFERENCE_IN_EXCHANGE	Number	(15,2)	Difference in exchange
BY_INTEREST_ON_DRAWINGS	Number	(15,2)	Interest on drawings
BY_DISCOUNT_ON_CREDITORS	Number	(15,2)	Discount on creditors
BY_BAD_DEBTS_RECOVERED	Number	(15,2)	Bad debts recovered
BY_MISCELLANEOUS_RECEIPTS	Number	(15,2)	Miscellaneous receipt
BY_INCREASE_IN_VALUE_OF_ASSETS	Number	(15,2)	Increase in value of assets
BY_INCOME_FROM_INVESTMENT	Number	(15,2)	Income from investment
BY_RESERVE_FOR_BAD_DOUBTS	Number	(15,2)	Reserve for bad doubts
BY_NET_LOSS	Number	(15,2)	Net loss

BALANCE_SHEET

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
BILLS_PAYABLE	Number	(15,2)	Bills payable
SUNDRY_CREDITORS	Number	(15,2)	Sundry creditors
LOANS	Number	(15,2)	Loans
OUTSTANDING_EXPENSES	Number	(15,2)	Outstanding expenses
CAPITAL	Number	(15,2)	Capital
NET_PROFIT	Number	(15,2)	Net profit
INTEREST_ON_CAPITAL	Number	(15,2)	Interest on capital
DRAWINGS	Number	(15,2)	Drawings
NET_LOSS	Number	(15,2)	Net loss
INCOME_TAX	Number	(15,2)	Income tax
CASH_IN_HAND	Number	(15,2)	Cash in hand
CASH_AT_BANK	Number	(15,2)	Cash at bank
INVESTMENTS	Number	(15,2)	Investments
BILLS_RECEIVABLE	Number	(15,2)	Bills receivable
SUNDRY_DEBTORS	Number	(15,2)	Sundry debtors
CLOSING_STOCK	Number	(15,2)	Closing stock
STORES	Number	(15,2)	Stores

PLANT_AND_MACHINERY	Number	(15,2)	Plant & machinery
FREEHOLD_PREMISES	Number	(15,2)	Freehold premises
UNEXPIRED_EXPENSES	Number	(15,2)	Unexpired expenses
GOODWILL	Number	(15,2)	Goodwill

c. **Process Logic**: -

This project will follow the following process logic.

Client Information Module: - Here the user will enter the basic information of each client. In this module, consulting the CLIENT_RECORD table will check the validation of PAN for each client. To access this module, user will have to have proper authorization, which is password protected.

Original Return Module: - Here, the user will provide the PAN, which will be verified using CLIENT_RECORD table. If ok, the record of client will be fetched. Verification of Assessment year will be made as well. If alright thin this module will accept the each information and store it in table RETURN_RECORD for future use.

Revised Return Module: - Similar to above module, user will supply the PAN, and after verification, user will provide assessment year, after making sure that Original return has already been filed, it will accept the information regarding the revised return and store it in RETURN_RECORD table.

Trading Account Module: - After filing of Original return or revised return, if the client is firm, the user will prompted with this module to enter the details of Trading A/c. This module will ensure the balance between credit and debit side, by performing proper calculation. Records regarding Trading account will be maintained in TRADING_ACCOUNT table.

Profit & Loss Account Module: - After filling the Trading Account Module, user will again prompted with P/L A/c Module to enter details of Profit & Loss Account. This module will also ensure the balancing of credit and debit side, by performing proper calculation. Records regarding Profit & Loss account will be stored in PL_ACCOUNT table.

Balance Sheet Module: - After completion of P/L A/c, the user will be prompted with Balance Sheet Module, which will follow trading A/c and P/L A/c. Here, balance of assets and liabilities will be maintained. This module will store the record of Balance Sheet in BALANCE_SHEET table.

Report Generation Module: - After getting the query request from user, this module will consult CLIENT_RECORD, RETURN_RECORD, TRADING_ACCOUNT, PL_ACCOUNT & BALANCE_SHEET and this will generate different reports.

6. **Tools/Platform**: -

Operating System: - Windows 98/ Windows 2000ME/ Windows XP

Front end: - VC++

Back end: - Oracle8i

Hardware (Optimal): -

Processor: - PIII

Memory: - 128MB RAM

Secondary memory: - 10GB

7. **Security Mechanism**: - INCOME TAX EVALUATION & MAINTENANCE SYSTEM will have number of security mechanism implemented on different levels. At first the user will have to enter the right user name & associated password in order to

have access to the S/w. To handle the information regarding clients, the user again will have to enter the valid user name & password to access the User Information Module. As the security of return record of every client is very important, so Return Module will also be protected with the user name & password, same thing will be applicable with Report Generation Module. By doing all this it will be ensured to have only appropriate authority to different users. As Oracle is used as backend, so all the security mechanism of Oracle will be incorporated in this project as well.

8. **Future scope**: - As in this project I am going to concentrate mostly on the return record regarding firms along with their all concerned enclosures. So, obviously future scope as well as future enhancement of this project is luminous, in its improved version all other categories of return filer can be included. At the same time, introduction of network in this project will make it universal in the sense to make it available to all the users, residing at distant.

2. Introduction To The System:

ITEMS i.e. **INCOME TAX EVALUATION & MAINTENANCE SYSTEM,** is an application software, which implements the basics of Income Tax department.

1. This software implements all the minute provisions made in income tax.

2. This system is very handy in terms of maintaining the records of income tax already paid, the revised income tax paid.

3. This system keeps track of the original & revised income tax paid by the client.

4. This system gives the proper provision for security as well.

3. Objectives Of The Project:

The aim and objectives of an important project such as this are multidimensional as it involves the theoretical and practical knowledge, believes, biases and most of all, the attitude earned during the course, and this project is no exception.

The most important objectives of this project are as follows: -

1. Maintain the records of each client.

2. Based upon the category of each client, provides the forms to file their original return for specific fiscal.

3. Provides the facility to file the revised return in the case of any mistake in original return.

4. For firms, it provides the facility to maintain their records regarding Trading A/c, P/L A/c & Balance Sheet.

5. This project will generate reports for: -

- ✓ Return history of particular client.
- ✓ Return filed in a particular fiscal
- ✓ Total revised return filed by a particular client

4. System Analysis:

According to this project (***Income Tax Evaluation & Maintenance System***), system analysis refers to the process of examining a business situation with the intent of improving it through better procedures and methods. System analysis is the process of gathering and interpreting facts, diagnosing problems and using the information to recommend improvement to the system. Actually analysis refers that, what the System should do.

A. Identification of the Need:

Present System: Before this computerization, the lawyers manually maintained the whole process applying their mind and capability, which very tedious & on number of times turns into very messy. Some of the features of the existing systems are as follows:

- The clients came to lawyer for enquiry, as well to take advise for filling their return.
- Lawyers attend the clients, if convinced with the lawyer, client decide whether they have the file their return with this lawyer or not.
- If client convinced enough, they decide to be the client of this very lawyer. They provide their basic detail to the lawyer. The basic detail include the name, father's name, date of birth, address, PAN number etc.
- The details of client is maintained by the lawyer very carefully, as it is very confidential and could very easily lead to mishandling if leaked.
- Clients also provide the detail of their income, from various sources, such as from salary, from house property, from agricultural, from various investments etc.
- Clients also provide the details of their expenditure.

- Based upon the details provided by the client, lawyer take decision how much money will be taken into account as income, how much income tax his client have to pay. How much money will be deducted from income tax as a result of number of investments etc.
- In case if client is a firm, lawyer also maintain the details of Trading Account, Profit and Loss Account & Balance Sheet.
- Lawyers also maintain the previous income tax record of their client, as it will help them in case of revised return, as well as for future prospects of their client.

Drawbacks of the existing system:

As the existing system is more human intensive there came the need for the system in which the involvement of human factor is to the minimum. In the present system all the process is carried by the human and then the reports are send to the income tax department for future consideration.

- The lawyer & his colleague carry out all the work manually.
- The chances of human error are more.
- The redundancy of work is there.
- Lots of paperwork and space is needed to keep backup of information.
- Handling and maintenance of income tax for number of years has become cumbersome.
- The biggest problem of existing system is that all the operations are time consuming.

- And finally there is an utmost need for computerization for any institute, office, hotel, banks etc. In today's world, so as to facilitate easy job handling and maintenance.
- A proper computerized system also increases standard and portrays a good image in the society.

B. Preliminary Investigation:

The First Step in the system development life cycle is the preliminary investigation to determine the feasibility of the system .the purpose of the preliminary investigation to evaluate project requests .To have an idea about the working of the existing system, information is gathered using the following information gathering techniques:

1. **Review Organisation Documents**
2. **On Site Observation**
3. **Conducting Interviews**

(1) Review Organisation Documents: Under this we investigate the registers used to maintain the personal information of clients, the income tax records maintained for different clients, in case of firms trading account, profit and loss account & balance sheet maintained, registers maintained for having keep track of original as well as revised returns filed by the clients, apart these details records for income from various sources, expenditure in different forms, details of different investments.

(2) On Site Observation: Through this method we observe the activities of the number of income tax specialist lawyers. The purpose of On Site Observation is to get as soon as possible to the real system being studied. During On Site Observation we can see the office environment, workload of the system and the users, methods of work and the facility provided by the users.

(3) Conducting Interviews: For developing Income Tax Evaluation & Maintenance System, interviews proved to be the best source of qualitative information in the system. Reason being interview allows discovering the areas of misunderstanding, unrealistic expectations and resistance to the proposed system. Interview helped to understand the functioning of the income tax specialized lawyers and how actually the works is carried out. Interview with the number of income tax specialist lawyers clear what are the problems they have to face in the manual system and what are their requirements.

C. Feasibility study:

TECHNICAL FEASIBILITY: Technical feasibility means the hardware and software support needed for the system. The technical needs of the system may vary considerably, but might include, the facility to produce outputs in a given time, ability to process a certain volume of transaction at a particular speed, facility to communicate data to distant location.

In examining technical feasibility, configuration of the system is given were importance than the actual make of hardware. The configuration should give the complete picture about the system's requirement.

It is tested on corporate LAN with ETHERNET Card, the configuration of this system is appropriate for the project therefore it provide smooth communication between client and server.

Therefore this project is technical feasible.

OPERATIONAL FEASIBILITY: In the operational feasibility we consider the following points:

What changes will be brought with the system?

What organizational structures are distributed?

Will new skill be required? Do existing staff members have these skills?

If not, can they be trained in due course of time?

In this project no changes will be brought with the system and organizational structures are not distributed and no skill will be required. Because this project is made for a computer institute the all staffs are well versed with computer.

Therefore this project is operational feasible.

ECONOMIC FEASIBILITY: In economic feasibility we consider the most frequently used techniques for evaluating the effectiveness of a proposed system. More commonly known as cost / benefit analysis; the procedure is to determine the benefits and savings that are expected proposed system and compose them with costs, If benefits out weigh costs. A decision is taken to design and implement the system. Otherwise, further justification or alternative in the proposed system will have to be made if it is to have a chance being approved.

The cost incurred is nothing but hardware and software installation, if there are no computers. Now a day's price of hardware is declining and the facilities offered against that are increasing. The minimum requirements of this system are an Intel Pentium Processor based computer with minimum 128 MB RAM, Ethernet Card, an 80-column dot matrix printer for printing receipts and reports. So regarding financial

consideration even a small-scale unit can afford this. According to above conditions in this project cost / benefit analysis and therefore economic feasibility will be applicable.

SOCIAL FEASIBILITY: In the social feasibility we consider, whether a proposed project will be acceptable to the people or not and also examines the probability of the project being accepted by the group directly affected by the proposed system change.

Because this project is client server model based which is widely used by the general people in various tasks. Every network depends on client server model.

Therefore this project is social feasible.

MANAGEMENT FEASIBILITY: Management feasibility is a determination of whether a proposal project will be acceptable to management if management does not accept a project or gives a negligible support to it, the analyst will tend to view the project as a new feasible one.

According to above criteria this project is management feasible.

LEGAL FEASIBILITY: The legal feasibility is a determination whether proposed project infringes on known acts, statues, as well as, any pending legislation. Although in some instances the project might appear sand, on closer investigation it may be found to infringe on several legal areas.

This project is according to the rules and regulations it follows all the legal acts status. Therefore this project is legal feasible.

TIME FEASIBILITY: In the time feasibility, the time feasibility is a determination of whether a proposal project can be implemented fully within a stipulated time frame. If a project takes too much time it is likely to rejected.

This project is time feasible because this project can be implemented fully within a stipulated time frame.

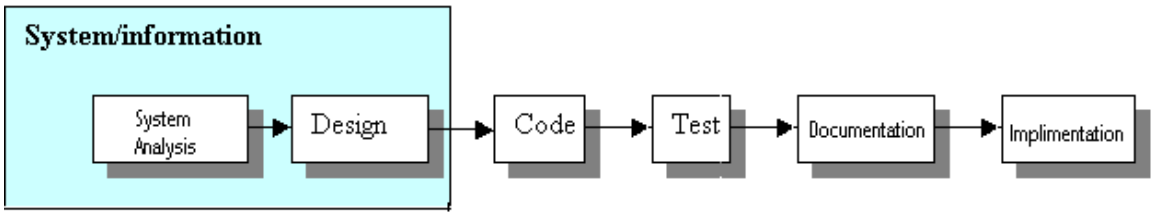
5. Software Engineering Paradigm Applied:

THE LINEAR SEQUENTIAL MODEL:

While making this project we have used The Linear Sequential Model sometimes called as The Classic Life Cycle or The Waterfall Model. It is the oldest and the most widely used paradigm for software engineering. This model suggests a systematic, sequential approach to software development that begins at the system level and progresses through analysis, design, coding, testing, and support. The Linear Sequential Model encompasses the following activities:

1) System/information engineering and modeling
2) Software requirements analysis
3) Design
4) Code generation
5) Testing

6) Support



System/information engineering and modeling:

System engineering and analysis encompass requirements gathering at the system level with a small amount of top-level design and analysis. Information engineering encompasses requirements gathering at the strategic business level and at the business area level.

Software requirements analysis:

The requirements gathering process is intensified and focused specifically on software. To understand the nature of the program to be built, the software engineer must understand the information domain for the software, as well as required function, behavior, performance, and interface. Requirements for both the system and the software are documented and reviewed with the customer.

Design:

Software design is actually a multi step process that focuses on four distinct attributes of a program: data structure, software architecture, interface representations, and procedural (algorithmic) detail. Like requirements, the design is documented and becomes part of the software configuration.

Code generation:

The design must be translated into a machine-readable form. The code generation step performs this task. If design is performed in a detailed manner, code generation can be accomplished mechanistically.

Testing:

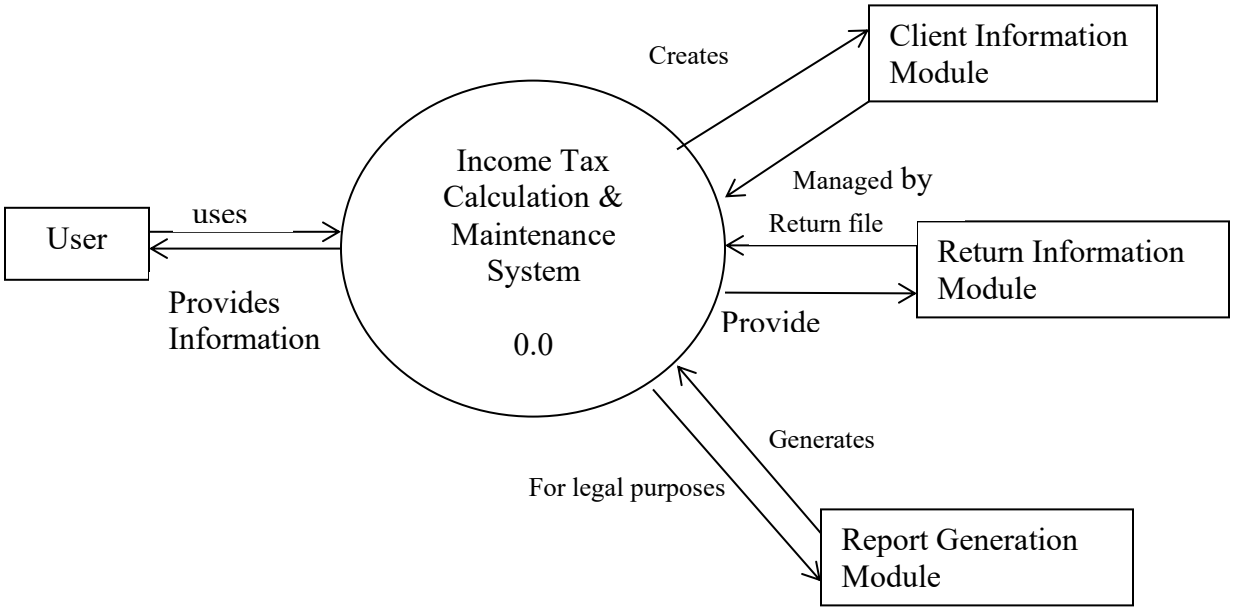
Once code has been generated, program testing begins. The testing process focuses on logical internals of the software, ensuring that all statements have been tested, and on the functional externals i.e. conducting tests to uncover errors and ensure that defined input will produce actual results that agree the required results.

Support:

Software will undoubtedly undergo change after it is delivered to the customer. Changes will occur because errors have been encountered, because the software must be adapted to accommodate changes in its external environment. Software support/maintenance reapplies each of the preceding phases to an existing program rather than a new one.

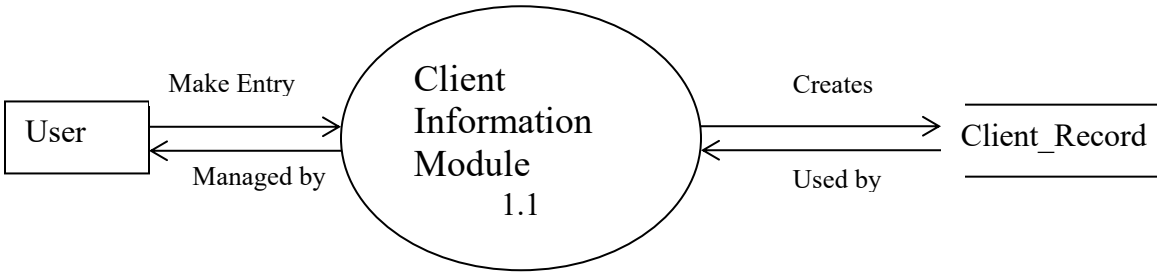
6. (A)Software Requirement Specifications:

(i) ***First Level Dfd***:

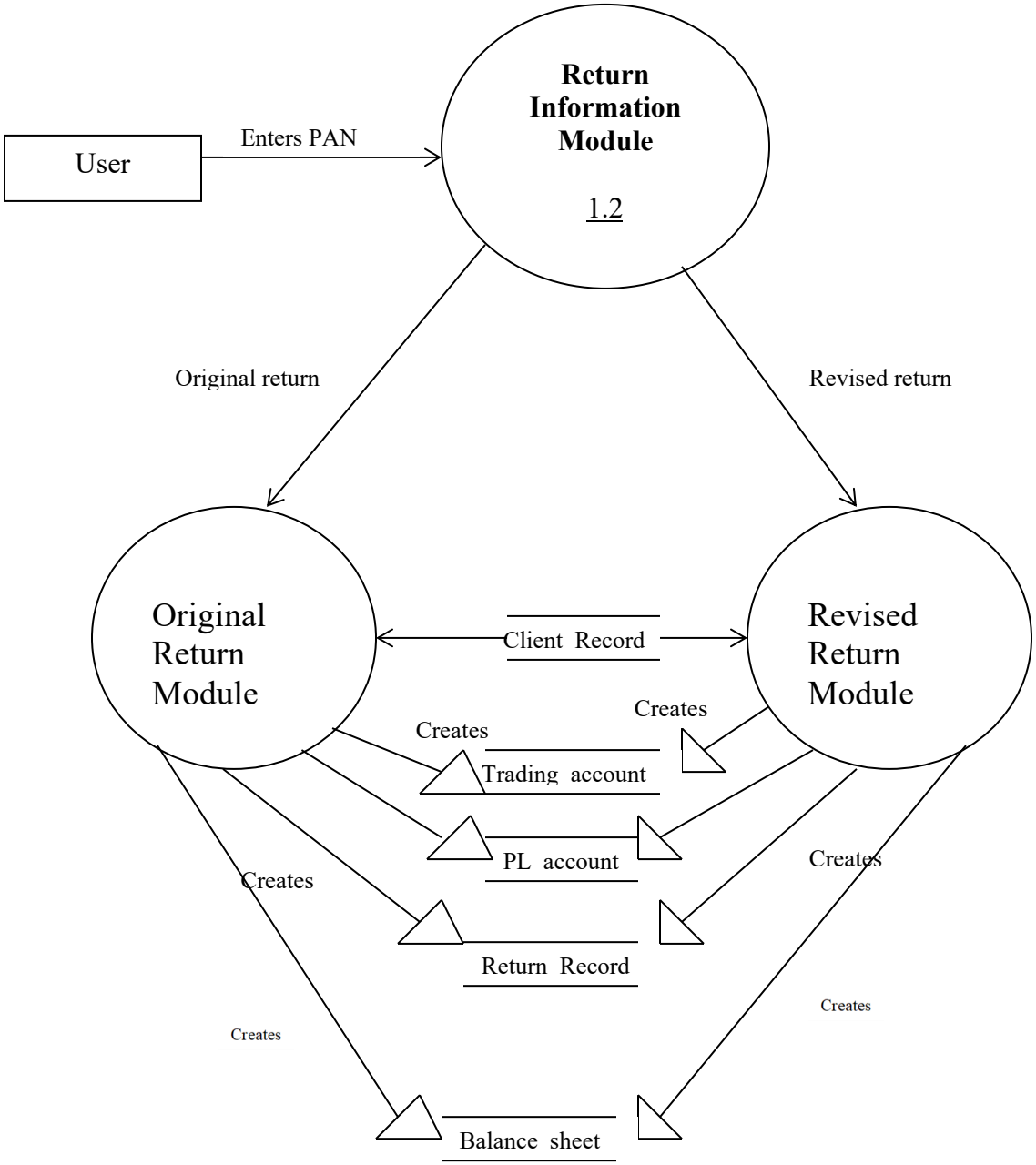


(ii) First Level Dfd:

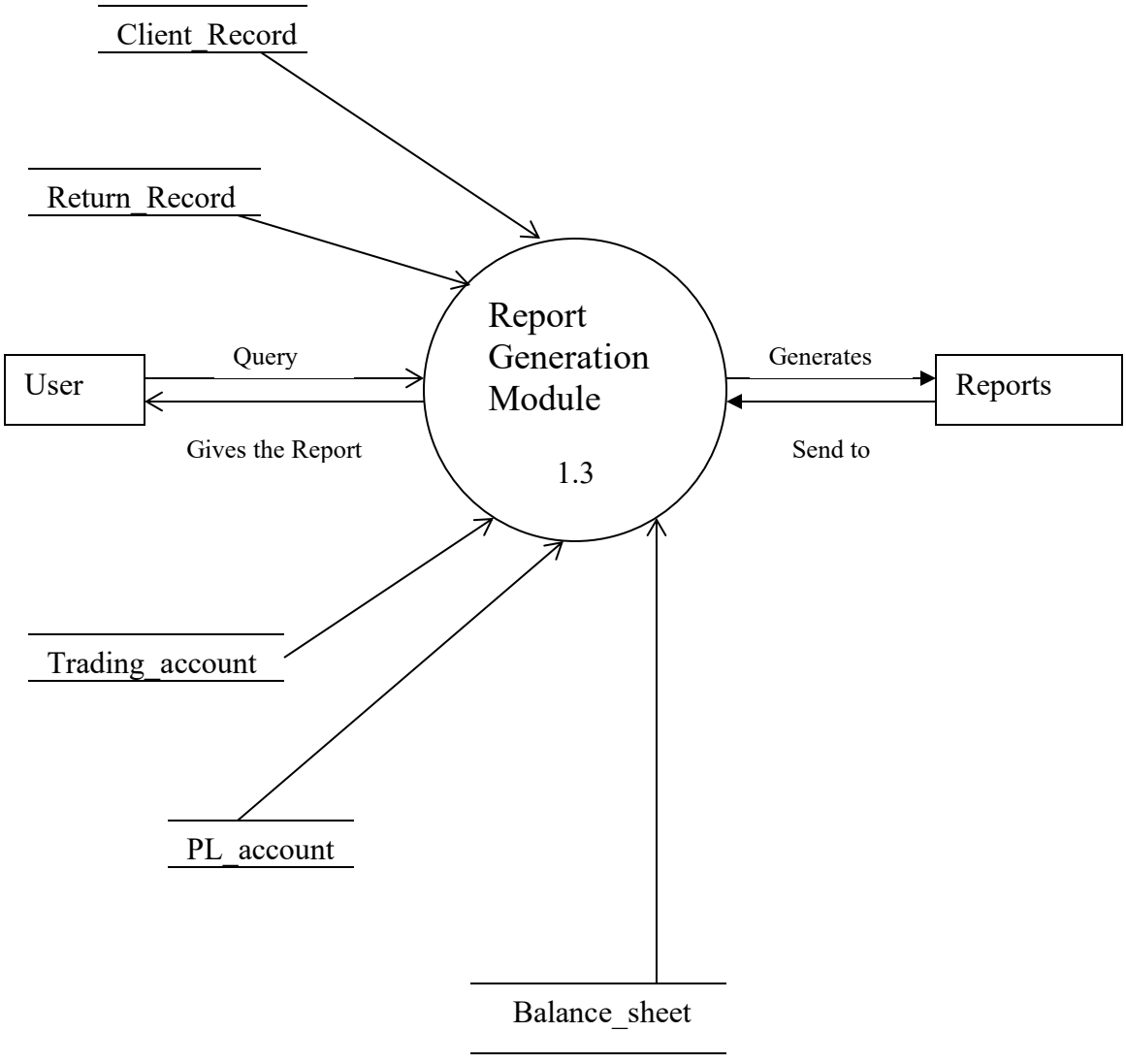
Level 1.1 DFD (for Client Information Module): -



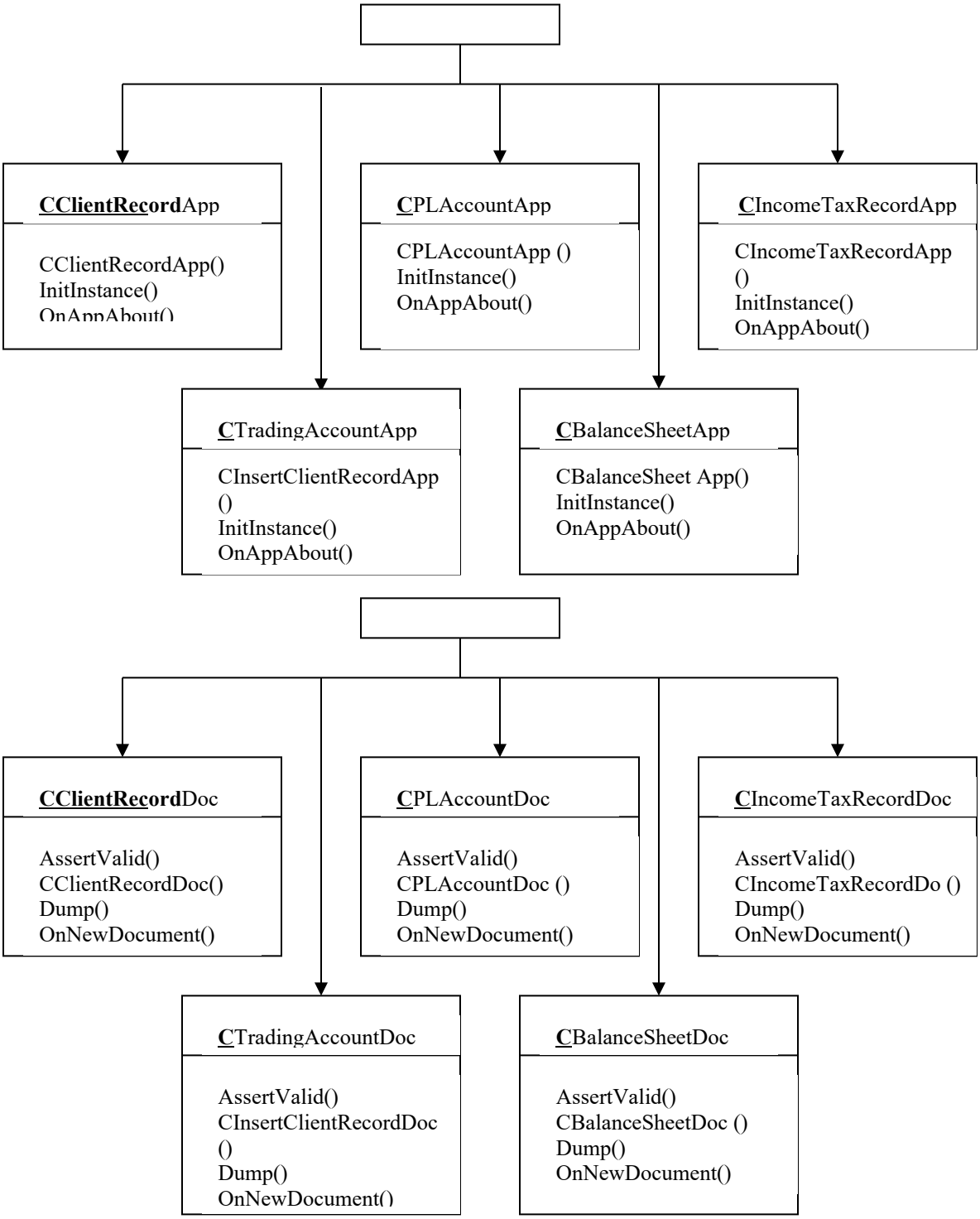
Level 1.2 DFD (for Return Information Module): -

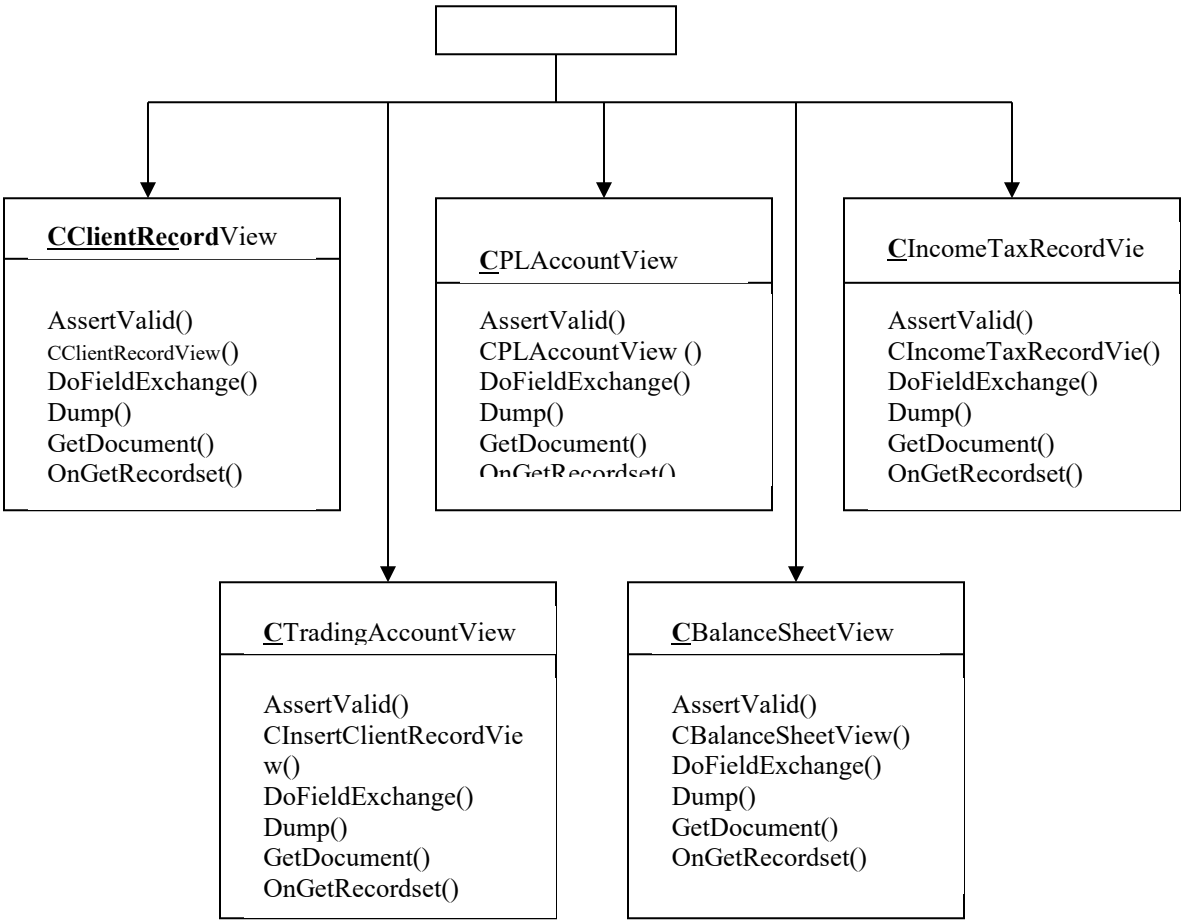


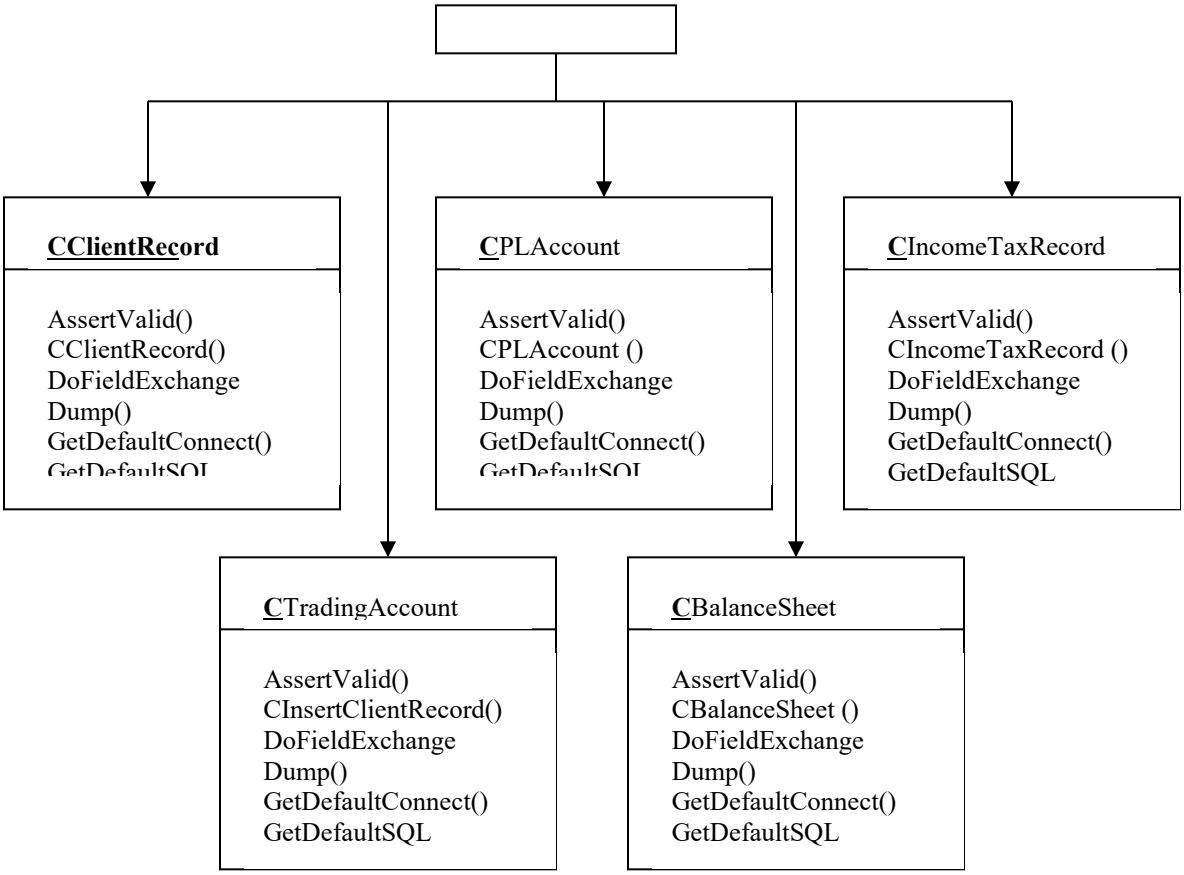
Level 1.3 DFD (for Report Generation Module): -

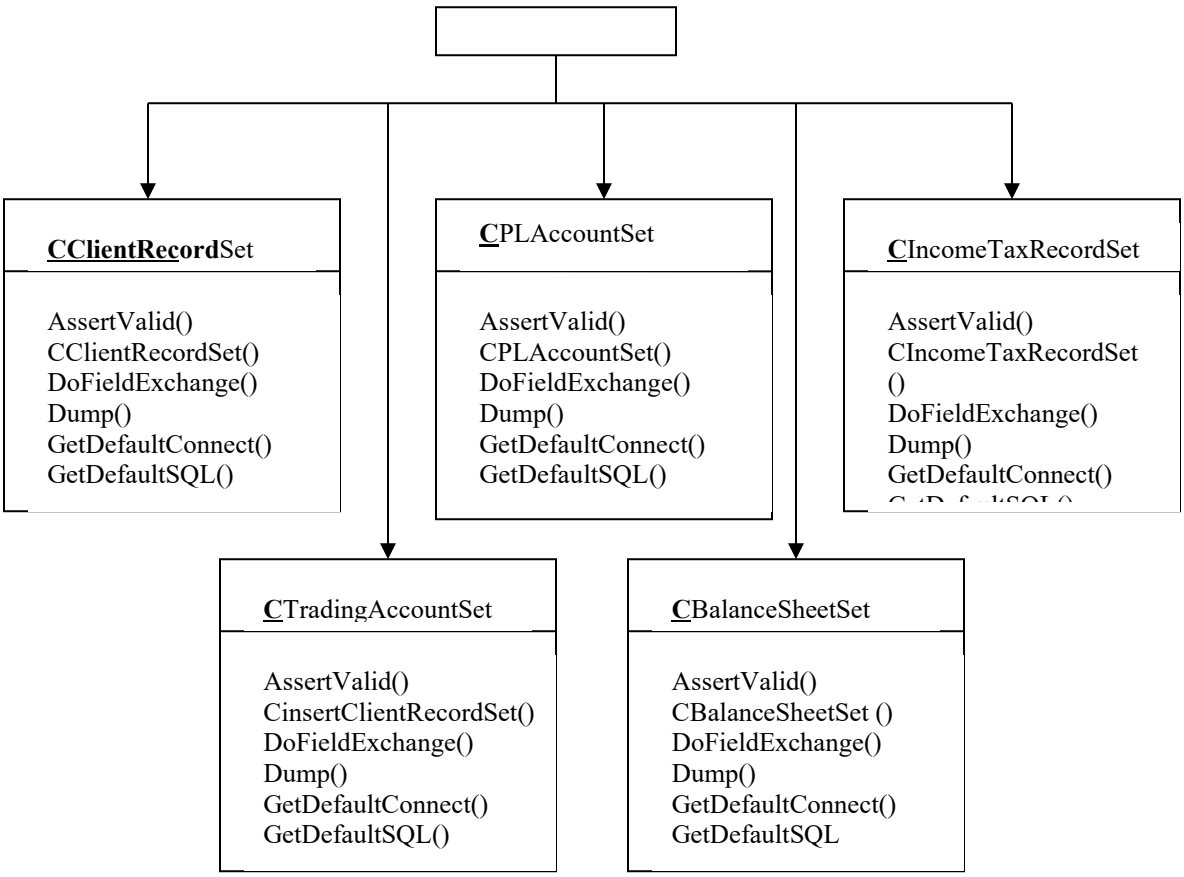


(5)(B)CLASS DIAGRAMS: My project following class diagrams: -




CClientRecordView
AssertValid()
CClientRecordView()
DoFieldExchange()
Dump()
GetDocument()
OnGetRecordset()
CPLAccountView
AssertValid()
CPLAccountView ()
DoFieldExchange()
Dump()
GetDocument()
CIncomeTaxRecordVie
AssertValid()
CIncomeTaxRecordVie()
DoFieldExchange()
Dump()
GetDocument()
OnGetRecordset()
CTradingAccountView
AssertValid()
CInsertClientRecordVie
w()
DoFieldExchange()
Dump()
GetDocument()
OnGetRecordset()
CBalanceSheetView
AssertValid()
CBalanceSheetView()
DoFieldExchange()
Dump()
GetDocument()
OnGetRecordset()


CClientRecord
AssertValid()
CClientRecord()
DoFieldExchange
Dump()
GetDefaultConnect()
GetDefaultSQL
CPLAccount
AssertValid()
CPLAccount ()
DoFieldExchange
Dump()
GetDefaultConnect()
GetDefaultSQL
CIncomeTaxRecord
AssertValid()
CIncomeTaxRecord ()
DoFieldExchange
Dump()
GetDefaultConnect()
GetDefaultSQL
CTradingAccount
AssertValid()
CInsertClientRecord()
DoFieldExchange
Dump()
GetDefaultConnect()
GetDefaultSQL
CBalanceSheet
AssertValid()
CBalanceSheet ()
DoFieldExchange
Dump()
GetDefaultConnect()
GetDefaultSQL



(5)(C) NUMBER Of Module And Its Description: My project has seven modules. Description of each module is as follows: -

Client Information Module: - This module will maintain the basic information of each client such as PAN, NAME FATHER'S NAME, and DOB etc. This module will be responsible for addition, deletion, and update ion of each client's record.

Original Return Module: - This module will ensure the filing of original return of particular client for particular assessment year. This module will consult the client information module for verification and fetching the records of particular employee.

Revised Return Module: - This module will handle the filling of revised return for a particular assessment year for a particular employee.

Trading Account module: - If the client is a firm, then it will be essential to keep track of their Trading account. And in this proposed software, this very purpose will served by trading account module.

Profit and Loss Account Module: - If the client is firm, then the record of their profit & loss account will be essential for future use and this function is served by this very account.

Balance Sheet Module: - If the client is firm, then together with Trading A/c & Profit & Loss A/c, record regarding their Balance Sheet will be needed as well and this module will serve this purpose.

Report Generation Module: - Report generation is one of the most obvious purposes of any software and this software is not an exception. This software will generate 4 types of reports, as follows: -

- ✓ Return history of particular client.
- ✓ Return filed by a particular client in a particular fiscal.
- ✓ Total return filed in a particular fiscal.

Total revised return filed by a particular client.

(5) (D) Minimum Hardware Required:

Pentium processor based computer	
RAM	128 MB
Ethernet card	
Mouse and Keyboard	
Hard disk	10 GB.
Monitor	15’ Monochrome
Dot Matrix Printer	80 column

(5) (E) Software Required:

Operating System	Windows 98 / Windows 200ME / Windows XP
Front End	Visual C++ 6.0
Back End	Oracle 8i
Report	Seagate Crystal Report 7.0

(5)(F) Data Structure (Table Etc) Of All Modules.

CLIENT_RECORD

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Primary key
NAME	Varchar2	30	Name of client
FATHERS_NAME	Varchar2	30	Father's name
DOB	Date		Date of birth
ADDRESS	Varchar2	40	Address of client
TELEPHONE	Varchar2	12	Contact number
SEX	Boolean		Male or female
INDV_HUF_FIRM_AOP_LA	Number	1	Category of client
RESIDENT_NR_NOR	Number	1	About residence
WARD_CIRCLE_SPECIAL_RANGE	Varchar2	20	Client's ward

INCOME_TAX_RECORD

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
INCOME_FOR_PREV_YEAR	Number	(15,2)	Previous income
RETURN_ORIGINAL_REVISED	Boolean		Original or revised
INCOME_FROM_SALARY	Number	(15,2)	Salary from income
INCOME_FROM_HOUSE_PROPERTY	Number	(15,2)	Income from house
INCOME_FROM_BUSINESS_OR_PROFESSION	Number	(15,2)	Income from business
TOTAL_SHORT_TERM_GAIN	Number	(15,2)	Short term gain
TOTAL_LONG_TERM_GAIN	Number	(15,2)	Long term gain
TOTAL_CAPITAL_GAINS	Number	(15,2)	Capital gain
INCOME_FROM_OTHER_SOURCES	Number	(15,2)	From other sources
INCOME_OF_OTHER_PERSON_TO_BE_ADDED	Number	(15,2)	Income of other person to be added
GROSS_TOTAL_INCOME	Number	(15,2)	Gross total income
LESS_DEDUCTION_UNDER_VIA	Number	(15,2)	Deduction under Via
TOTAL_INCOME	Number	(15,2)	Total income
AGRICULTURAL_INCOME	Number	(15,2)	Agricultural income
INCOME_TO_BE_EXEMPTED	Number	(15,2)	Income to be exempted
INCOME_AT_NORMAL_RATES	Number	(15,2)	Income at normal rates
INCOME_TAX_AT_NORMAL_RATES	Number	(15,2)	Income at normal rate
INCOME_AT_SPECIAL_RATES	Number	(15,2)	At special rate

INCOME_TAX_AT_SPECIAL_RATES	Number	(15,2)	Tax at special rate
TAX_ON_TOTAL_INCOME	Number	(15,2)	Tax on total income
LESS_REBATE	Number	(15,2)	Rebate
TAX_PAYABLE	Number	(15,2)	Tax payable
ADDITIONAL_SURCHARGE	Number	(15,2)	Additional surcharge
TOTAL_TAX_PAYABLE	Number	(15,2)	Total payable tax
LESS_RELIEF	Number	(15,2)	Relief
NET_TAX_PAYABLE	Number	(15,2)	Net tax payable
TAX_DEDUCTED_AT_SOURCES	Number	(15,2)	Deduction at sources
ADVANCE_TAX_PAID	Number	(15,2)	Tax paid in advance
INTEREST_PAYABLE	Number	(15,2)	Interest payable
SELF_ASSESSMENT_PAID_UPTO_31_MAY	Number	(15,2)	Self assessment paid up to 31st May
SELF_ASSESSMENT_PAID_AFTER_31_MAY	Number	(15,2)	Self assessment paid after 31st May
TOTAL_SELF_ASSESSMENT_TAX_PAID	Number	(15,2)	Total self assessment tax paid
BSR_CODE_OF_BANK_BRANCH	Varchar2	15	BSR code of bank
DATE_OF_DEPOSIT	Date		Date of deposit
SERIAL_NO_OF_CHALLAN	Varchar2	10	Serial no of Chelan
AMOUNT	Number	(15,2)	Amount
NAME_OF_BANK_FOR_ATTACHED_CHALLAN	Varchar2	20	Name of bank for attached Chelan
BALANCE_TAX_PAYABLE_REFUNDABLE	Number	(15,2)	Balance tax payable refundable
BALANCE_TAX	Number	(15,2)	Balance tax
ENCLOSURE1	Varchar2	20	1st enclosure
ENCLOSURE2	Varchar2	20	2nd enclosure
ENCLOSURE3	Varchar2	20	3rd enclosure
ENCLOSURE4	Varchar2	20	4th enclosure
ENCLOSURE5	Varchar2	20	5th enclosure
ENCLOSURE6	Varchar2	20	6th enclosure

TRADING_ACCOUNT

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
TO_OPENING_STOCK	Number	(15,2)	Opening stock
TO_STOCK	Number	(15,2)	Stock
TO_PURCHASES	Number	(15,2)	Purchases
TO_CARRIAGE	Number	(15,2)	Carriage

TO_OCTROI	Number	(15,2)	Octroi
TO_IMPORT_DUTY_CUSTOMS	Number	(15,2)	Import duty & custom
TO_WAGES	Number	(15,2)	Wages
TO_COAL_WATER_GAS	Number	(15,2)	Coal, water & gas
TO_HEATING_LIGHTING_POWER	Number	(15,2)	Heating, light & power
TO_MANUFACTURING_ASSEMBLING_EXPENSES	Number	(15,2)	Manufacturing & assembling expenses
TO_CONSUMABLE_STORES	Number	(15,2)	Consumable stores
TO_DIRECT_FACTORY_PRODUCTIVE_EXPENSES	Number	(15,2)	Direct factory product expenses
TO_ROYALTY	Number	(15,2)	Royalty
TO_GROSS_PROFIT	Number	(15,2)	Gross profit
BY_SALES	Number	(15,2)	Sales
BY_CLOSING_STOCK	Number	(15,2)	Closing stock
BY_STOCK	Number	(15,2)	Stock
BY_GROSS_LOSS	Number	(15,2)	Gross loss

PL_ACCOUNT

1. ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
TO_GROSS_LOSS	Number	(15,2)	Gross loss
TO_SALARIES_WAGES	Number	(15,2)	Salaries & wages
TO_OFFICE_GODOWN_RENT	Number	(15,2)	Office & go down rent
TO_OFFICE_EXPENSES	Number	(15,2)	Office expenses
TO_MISCELLANEOUS_SUNDRY_EXPENSES	Number	(15,2)	Miscellaneous & sundry expenses
TO_INSURANCE	Number	(15,2)	Insurance
TO_STATIONERY_PRINTING	Number	(15,2)	Stationery expenses
TO_STAFF_WELFARE_EXPENSES	Number	(15,2)	Staff welfare expenses
TO_LIGHTING_WATER_ELECTRICITY	Number	(15,2)	Lighting, water & electricity
TO_ESTABLISHMENT_EXPENSES	Number	(15,2)	Establishment expenses
TO_POSTAGE_TELEGRAM_FAX_COURIER_TELEPHONE	Number	(15,2)	Postage, telegram, fax, courier & telephone
TO_LAW_CHARGES	Number	(15,2)	Law charges
TO_REPAIRS	Number	(15,2)	Repairs
TO_DISTRIBUTION_EXPENSES	Number	(15,2)	Distribution expenses
TO_TRAVELLING_EXPENSES	Number	(15,2)	Traveling expenses
TO_GENERAL_EXPENSES	Number	(15,2)	General expenses
TO_STABLE_EXPENSES	Number	(15,2)	Stable expenses

TO_SELLING_EXPENSES	Number	(15,2)	Selling expenses
TO_CARRIAGE_OUTWARD	Number	(15,2)	Carriage outward
TO_CARRIAGE_ON_SALES	Number	(15,2)	Carriage on sales
TO_INDIRECT_WAGES	Number	(15,2)	Indirect wages
TO_AUDIT_FEES	Number	(15,2)	Audit fees
TO_ENTERTAINMENT_EXPENSES	Number	(15,2)	Entertainment expenses
TO_INTEREST_PAID	Number	(15,2)	Interest paid
TO_DISCOUNT_ALLOWED	Number	(15,2)	Discount allowed
TO_BAD_DEBTS	Number	(15,2)	Bad debts
TO_RESERVE_FOR_BAD_DEBTS	Number	(15,2)	Reserve for bad debts
TO_DEPRECIATION	Number	(15,2)	Depreciation
TO_INTEREST_ON_CAPITAL	Number	(15,2)	Interest on capital
TO_DISCOUNTING_CHARGES	Number	(15,2)	Discounting charges
TO_BANK_CHARGES	Number	(15,2)	Bank charges
TO_EXPORT_CHARGES	Number	(15,2)	Export charges
TO_TRADE_EXPENSES	Number	(15,2)	Trade expenses
TO_ADMINISRTATIVE_EXPENSES	Number	(15,2)	Administrative expense
TO_FINANCIAL_EXPENSES	Number	(15,2)	Financial expenses
TO_COMMISSION_PAID	Number	(15,2)	Commission paid
TO_ADVERTISEMENT	Number	(15,2)	Advertisement
TO_CHARITY	Number	(15,2)	Charity
TO_SAMPLE_EXPENSES	Number	(15,2)	Sample expenses
TO_LICENCE_FEE	Number	(15,2)	License fee
TO_DELIVERY_CHARGES	Number	(15,2)	Delivery charges
TO_BROKERAGE	Number	(15,2)	Brokerage
TO_SALES_TAX	Number	(15,2)	Sales tax
TO_LOSS_ON_SALE_OF_ASSETS	Number	(15,2)	Loss on sale of assets
TO_LOSS_BY_FIRE_THEFT_ACCIDENT	Number	(15,2)	Loss by fire, theft or accident
TO_NET_PROFIT	Number	(15,2)	Net profit
BY_GROSS_PROFIT	Number	(15,2)	Gross profit
BY_INTEREST_RECEIVED	Number	(15,2)	Interest received
BY_RENT_RECEIVED	Number	(15,2)	Rent received
BY_DISCOUNT_RECEIVED	Number	(15,2)	Discount received
BY_DIVIDENDS_RECEIVED	Number	(15,2)	Dividend received
BY_PROFIT_FROM_SALES_OF_ASSETS	Number	(15,2)	Profit from sales of assets
BY_REFUND_OF_TAX	Number	(15,2)	Refund of tax
BY_COMPENSATION_RECEIVED	Number	(15,2)	Compensation received
BY_APPRENTICESHIP_PREMIUM	Number	(15,2)	Apprenticeship premium
BY_DIFFERENCE_IN_EXCHANGE	Number	(15,2)	Difference in exchange
BY_INTEREST_ON_DRAWINGS	Number	(15,2)	Interest on drawings
BY_DISCOUNT_ON_CREDITORS	Number	(15,2)	Discount on creditors

BY_BAD_DEBTS_RECOVERED	Number	(15,2)	Bad debts recovered
BY_MISCELLANEOUS_RECEIPTS	Number	(15,2)	Miscellaneous receipt
BY_INCREASE_IN_VALUE_OF_ASSETS	Number	(15,2)	Increase in value of assets
BY_INCOME_FROM_INVESTMENT	Number	(15,2)	Income from investment
BY_RESERVE_FOR_BAD_DOUBTS	Number	(15,2)	Reserve for bad doubts
BY_NET_LOSS	Number	(15,2)	Net loss

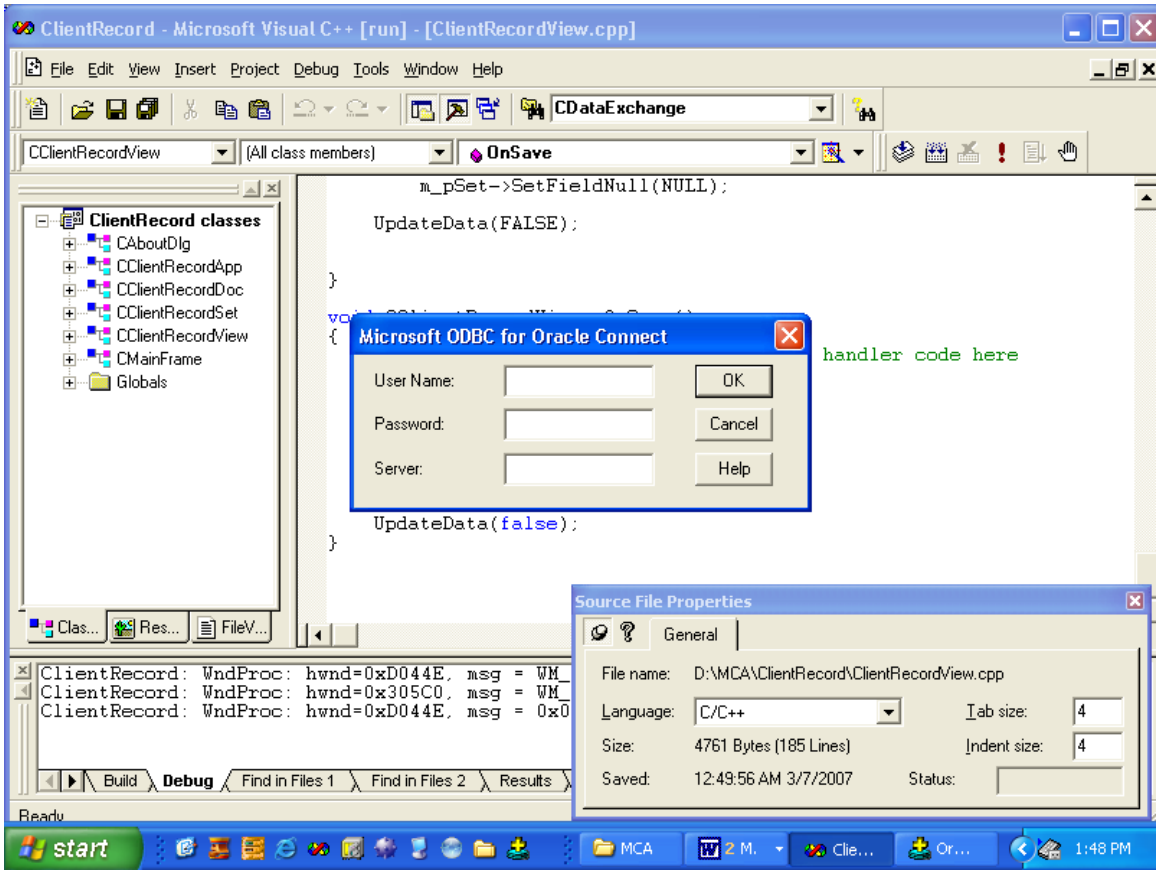
BALANCE_SHEET

ATTRIBUTE	TYPE	WIDTH	DESCRIPTION
PAN	Varchar2	10	Foreign key
ASSES_YEAR_1	Date		Composite key
ASSES_YEAR_2	Date		Composite key
BILLS_PAYABLE	Number	(15,2)	Bills payable
SUNDRY_CREDITORS	Number	(15,2)	Sundry creditors
LOANS	Number	(15,2)	Loans
OUTSTANDING_EXPENSES	Number	(15,2)	Outstanding expenses
CAPITAL	Number	(15,2)	Capital
NET_PROFIT	Number	(15,2)	Net profit
INTEREST_ON_CAPITAL	Number	(15,2)	Interest on capital
DRAWINGS	Number	(15,2)	Drawings
NET_LOSS	Number	(15,2)	Net loss
INCOME_TAX	Number	(15,2)	Income tax
CASH_IN_HAND	Number	(15,2)	Cash in hand
CASH_AT_BANK	Number	(15,2)	Cash at bank
INVESTMENTS	Number	(15,2)	Investments
BILLS_RECEIVABLE	Number	(15,2)	Bills receivable
SUNDRY_DEBTORS	Number	(15,2)	Sundry debtors
CLOSING_STOCK	Number	(15,2)	Closing stock
STORES	Number	(15,2)	Stores
PLANT_AND_MACHINERY	Number	(15,2)	Plant & machinery
FREEHOLD_PREMISES	Number	(15,2)	Freehold premises
UNEXPIRED_EXPENSES	Number	(15,2)	Unexpired expenses
GOODWILL	Number	(15,2)	Goodwill

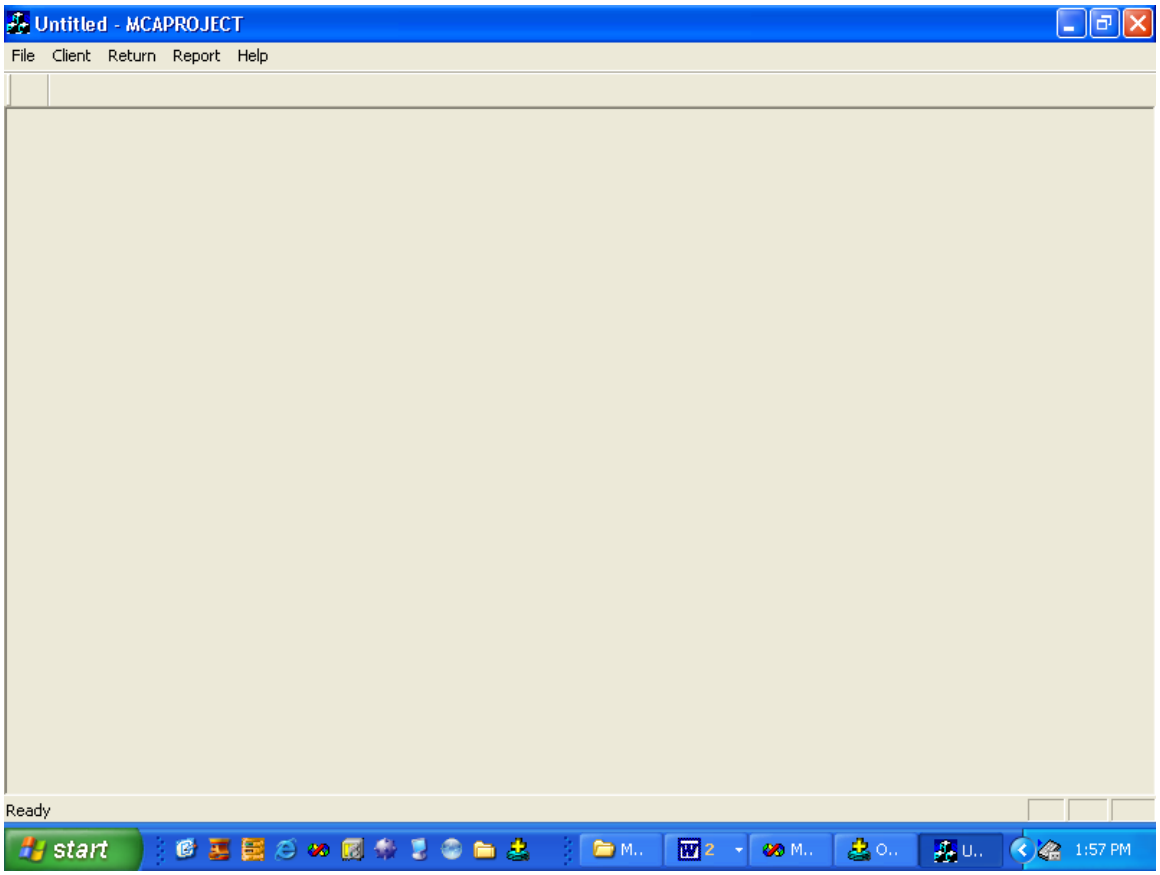
7. Design:

In this section we submit here some output, which are copied during run time through Keyboard <Print screen> command button.

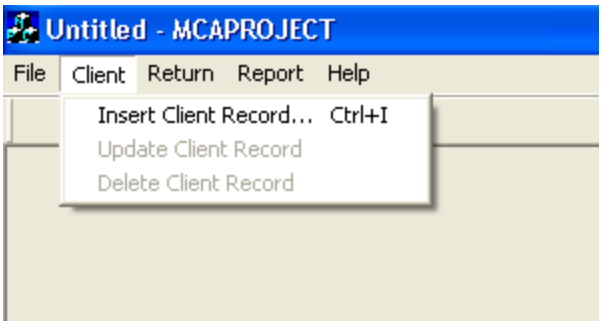
After execution of application program the SDI window appear as below. At this step user interact with system through LOGIN window. Only valid user who knows the USER ID and PASSWORD are enter into application. If user supplies wrong information the system will popup a message box. If cancels this LOGIN window, the application will automatically unload itself.



Valid user gets the application window after giving Valid USER ID and PASSWORD. The Main window is as below:



We divide application work in to three main menus. It is further divided into many submenus.



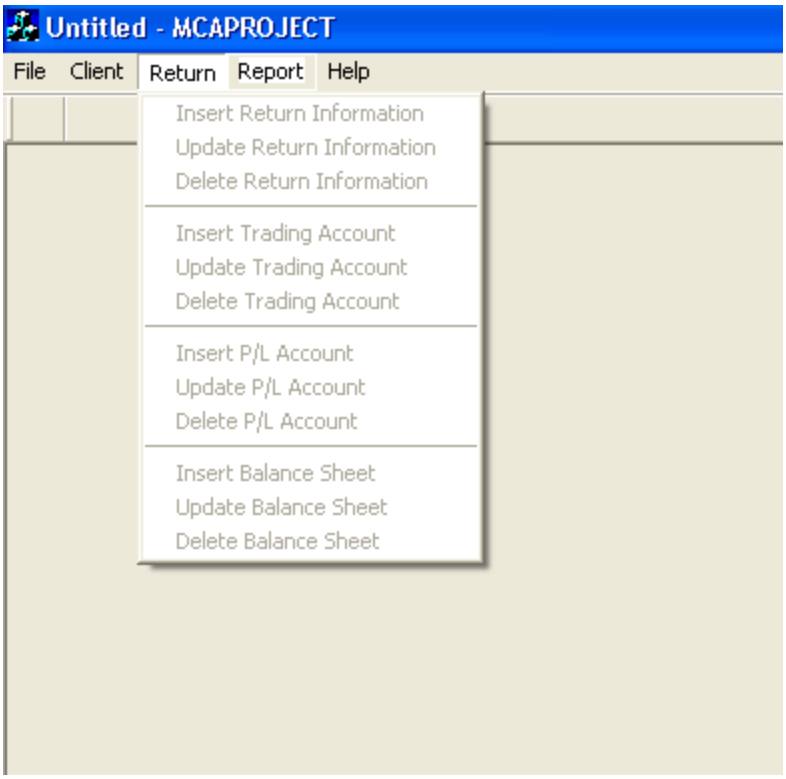
Above figure easily indicate that menu **Client** poses

1) Insert Client Record
2) Update Client Record
3) Delete Client Record

The Menu **Transaction** poses:

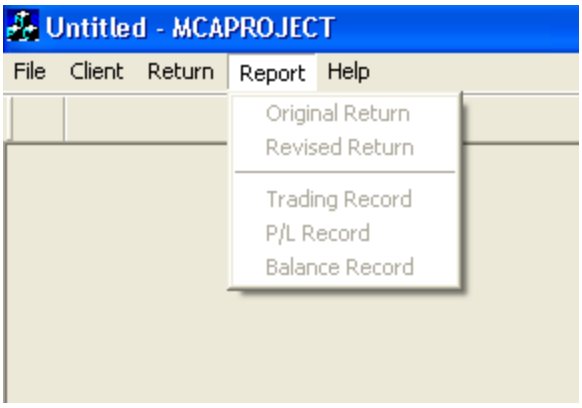
1) Insert Return Information
2) Update Return Information
3) Delete Return Information
4) Insert Trading Account
5) Update Trading Account
6) Delete Trading Account

7) Insert P/L Account
8) Update P/L Account
9) Delete P/L Account
10) Insert Balance Sheet
11) Update Balance Sheet
12) Delete Balance Sheet

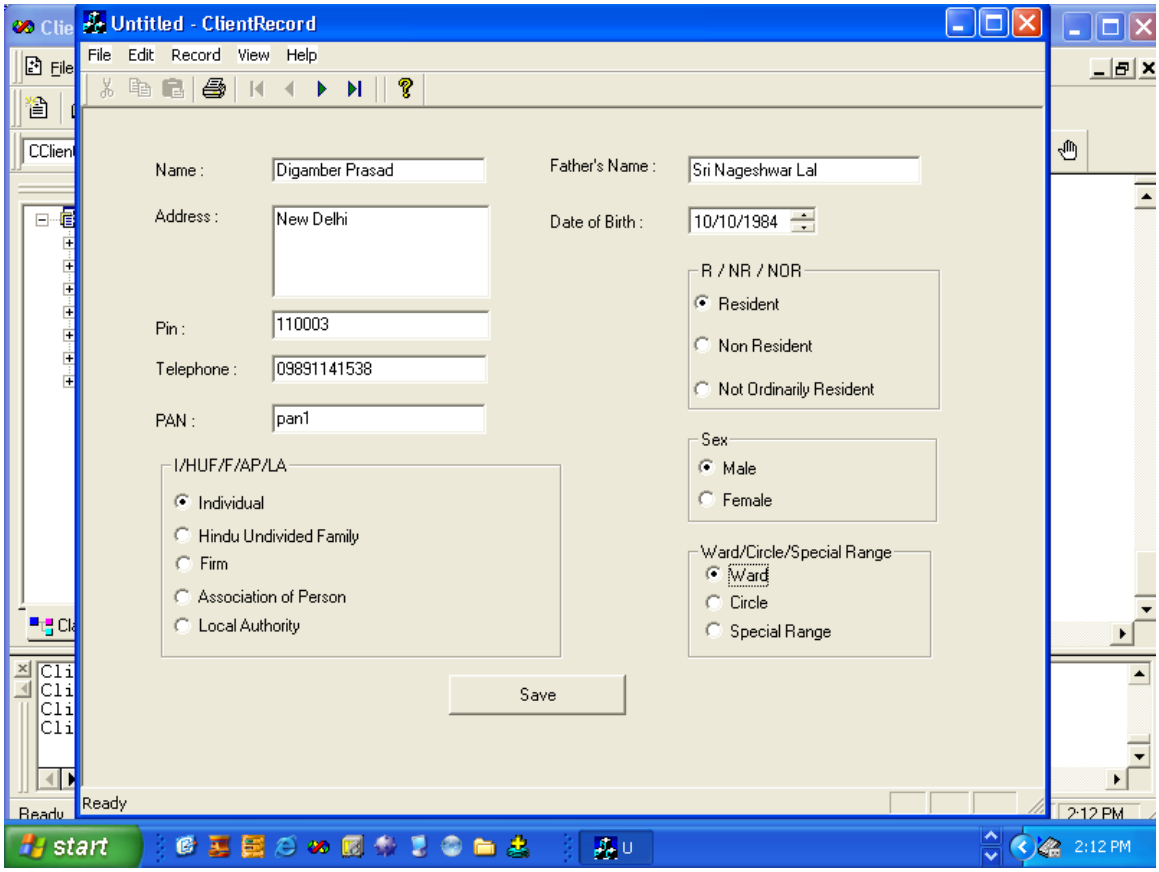


The Menu **Report** Poses

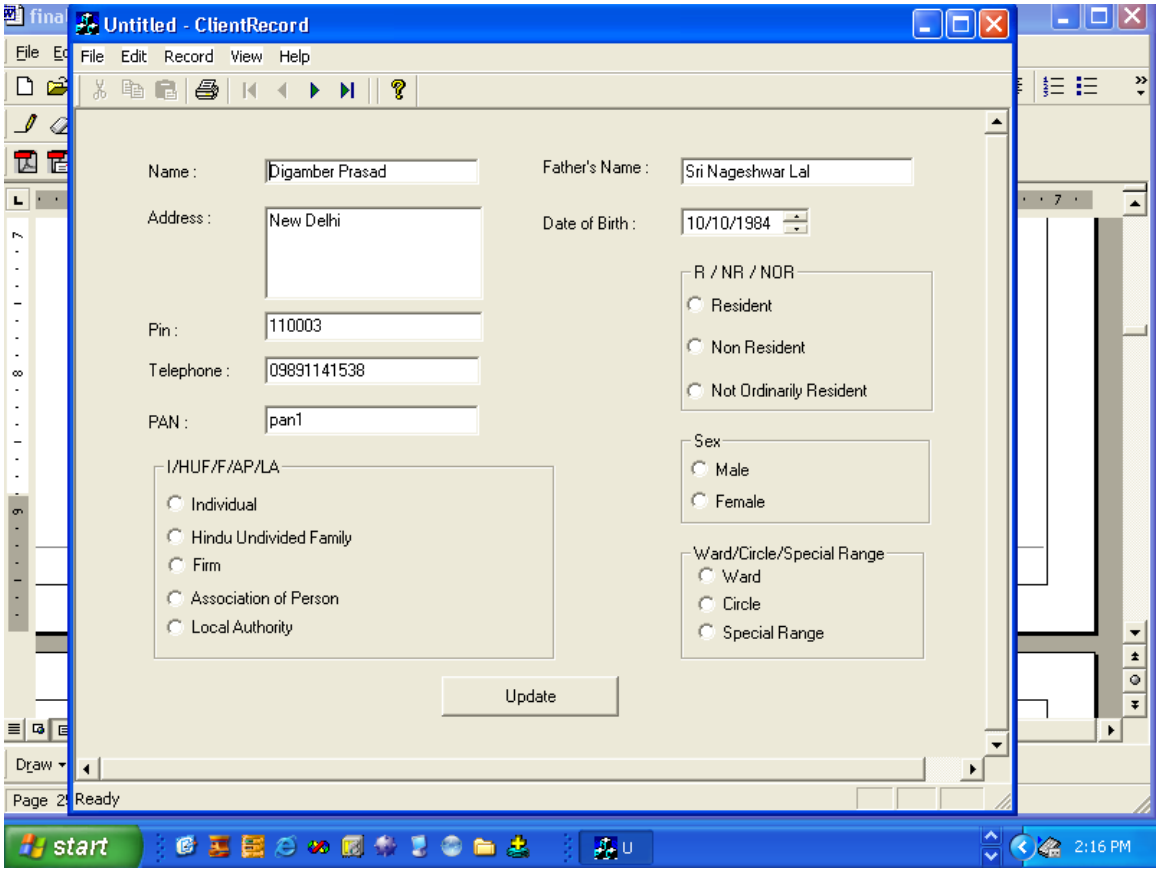
1) Original Return
2) Revised Return
3) Trading Record
4) P/L Record
5) Balance Record



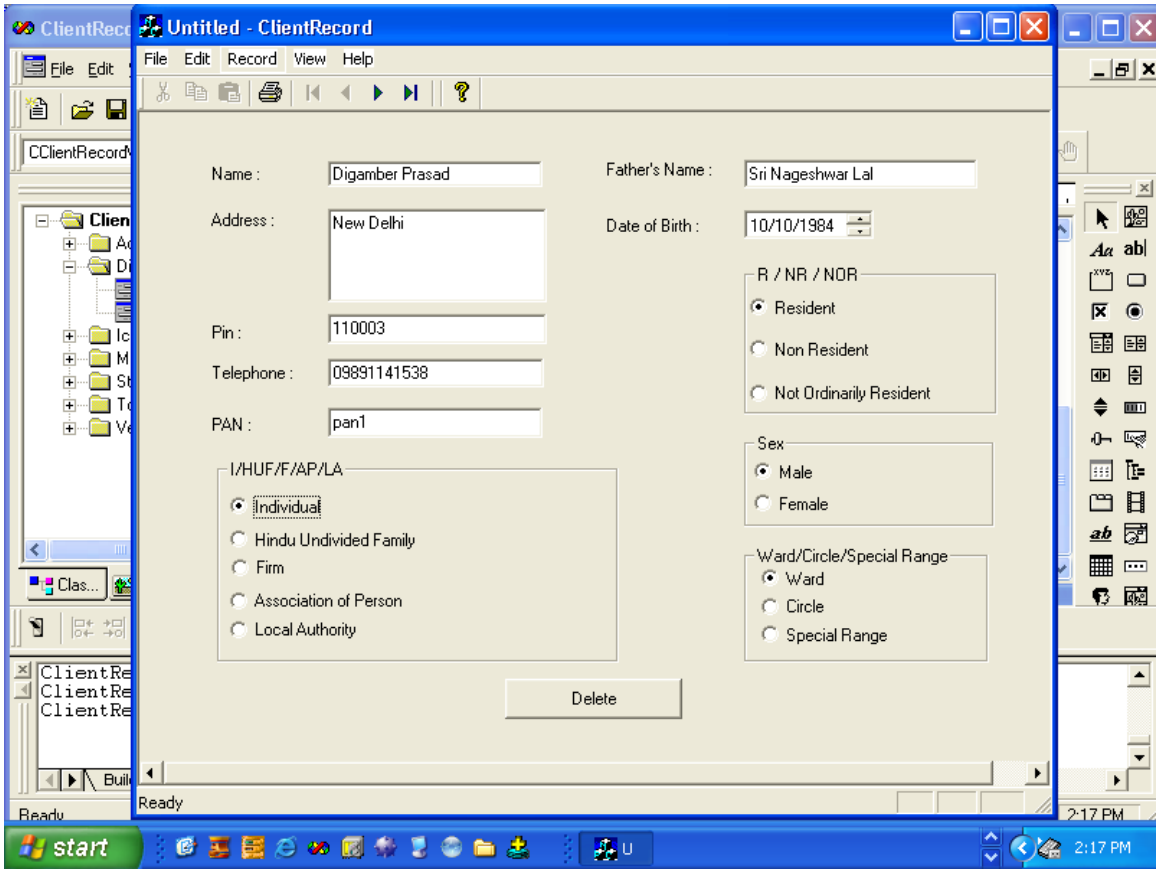
The form used for entering client information is as follows: -



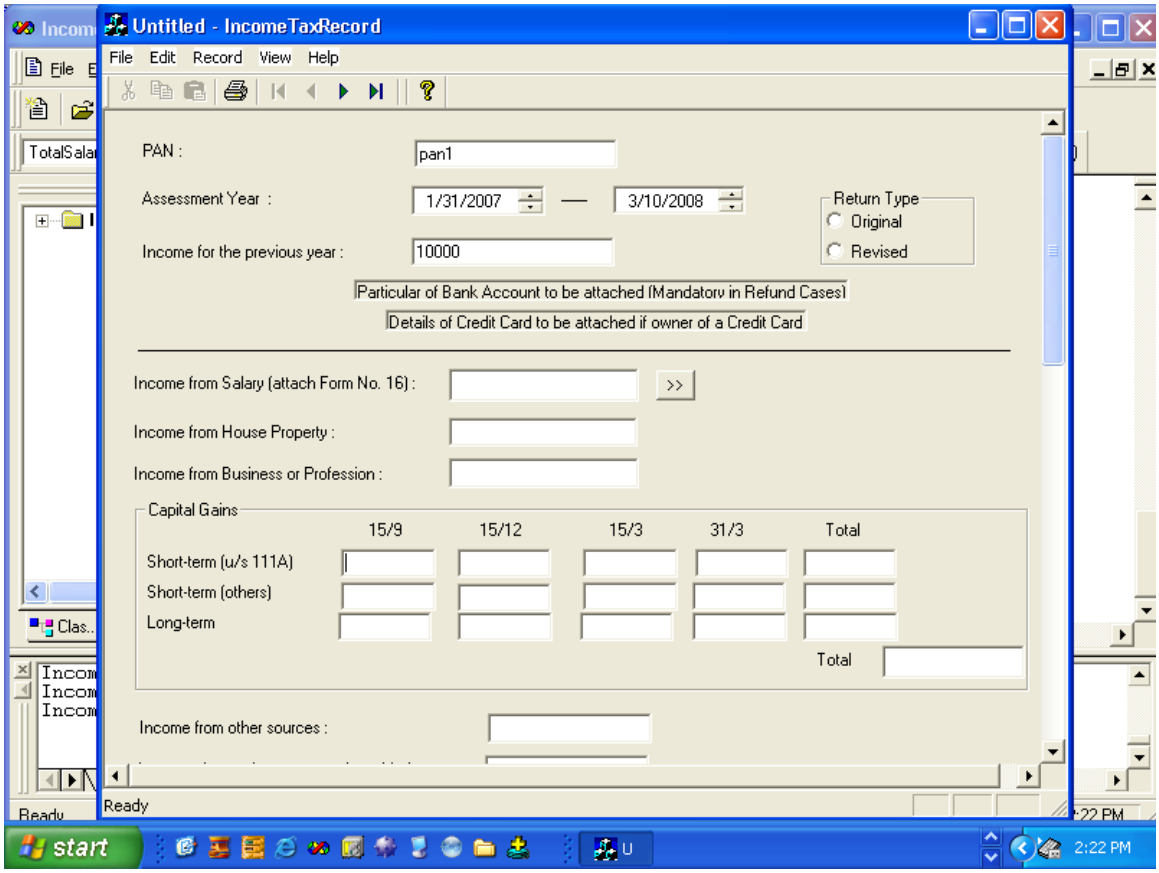
The form used for updating client information is as follows: -

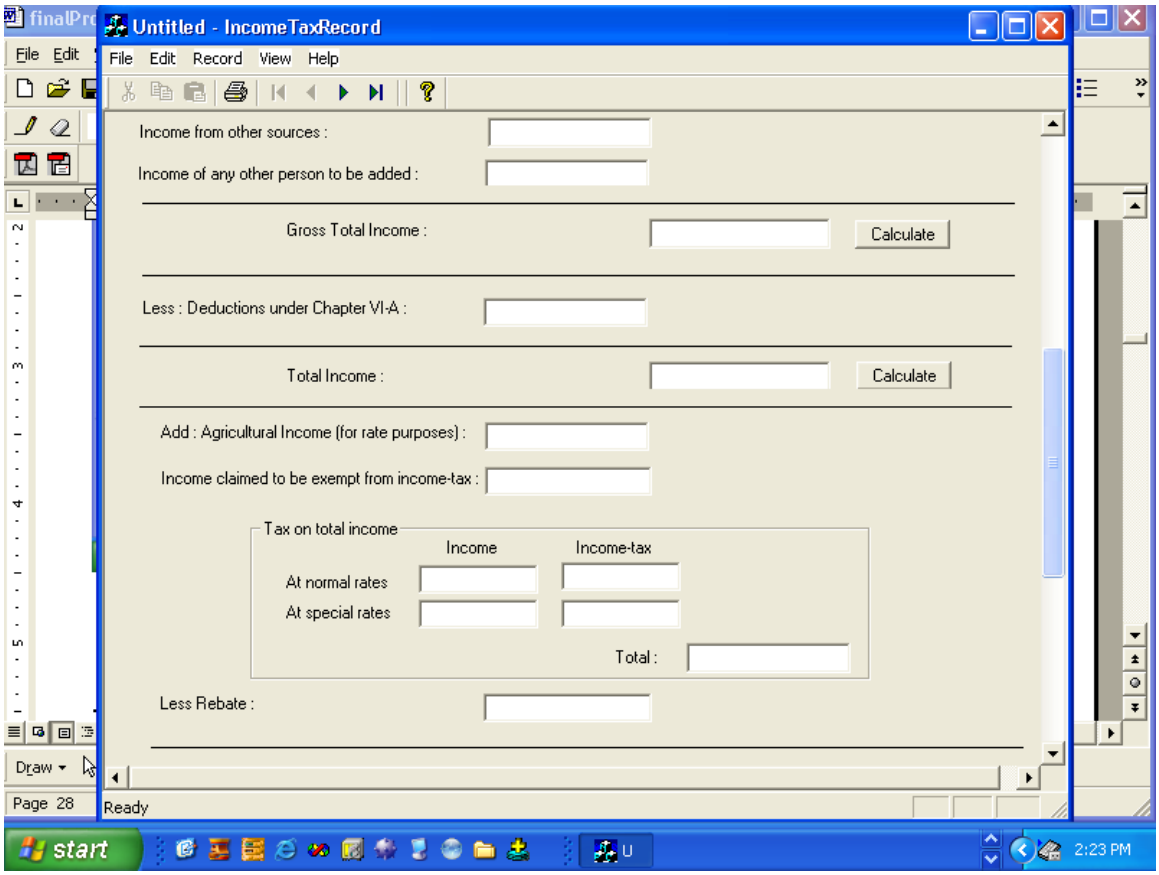


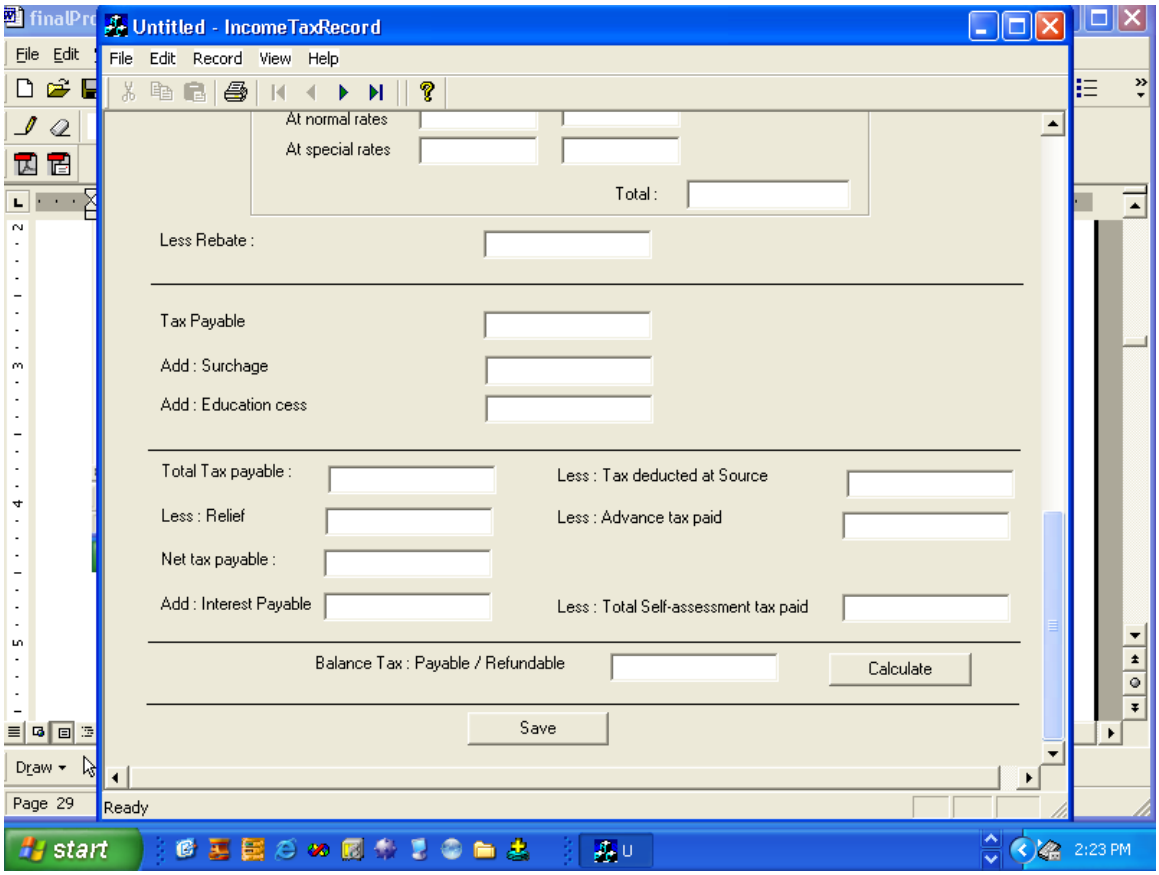
Form used for Deleting the client record is as follows: -



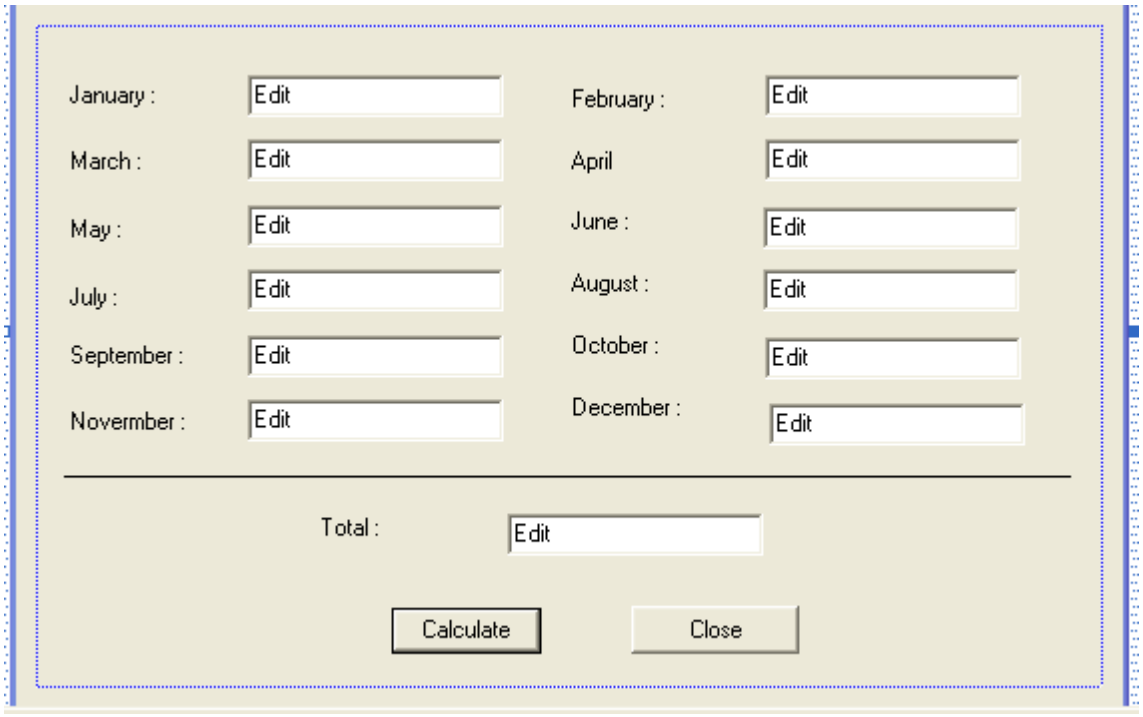
The form used to fill the income tax record is as follows: -



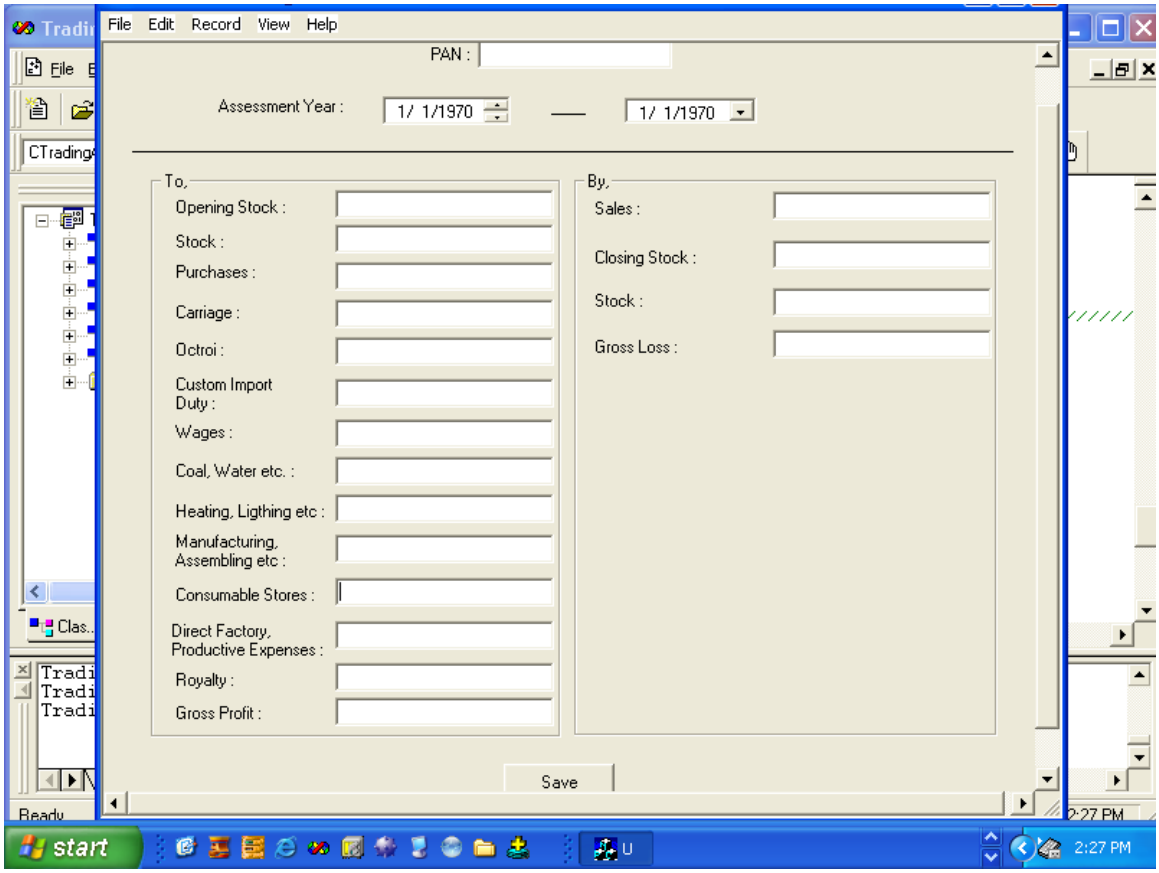

Untitled - IncomeTaxRecord
File Edit Record View Help
Income from other sources :
Income of any other person to be added :
Gross Total Income :
Calculate
Less : Deductions under Chapter VI-A :
Total Income :
Calculate
Add : Agricultural Income (for rate purposes) :
Income claimed to be exempt from income-tax :
Tax on total income
Income
Income-tax
At normal rates
At special rates
Total :
Less Rebate :
Ready
start
2:23 PM



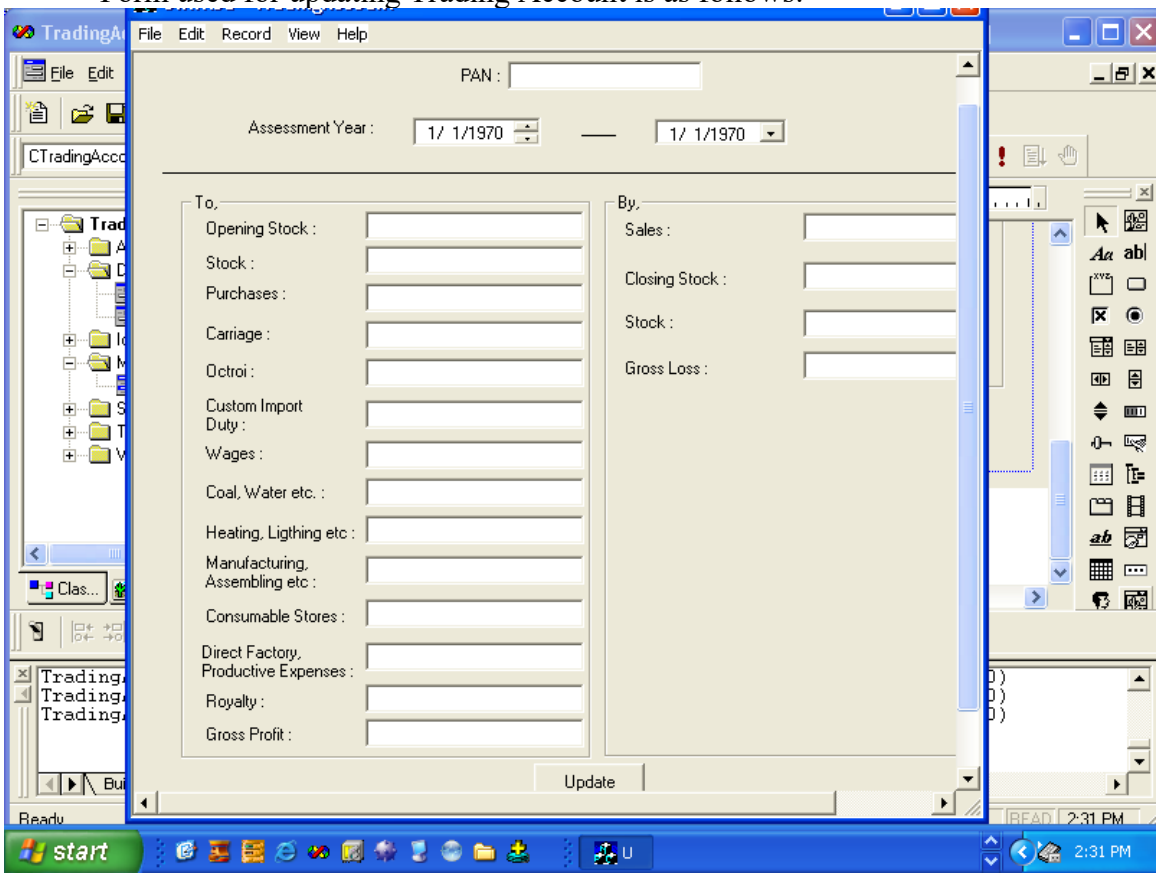
Form Used to calculate the total salary is as follows: -



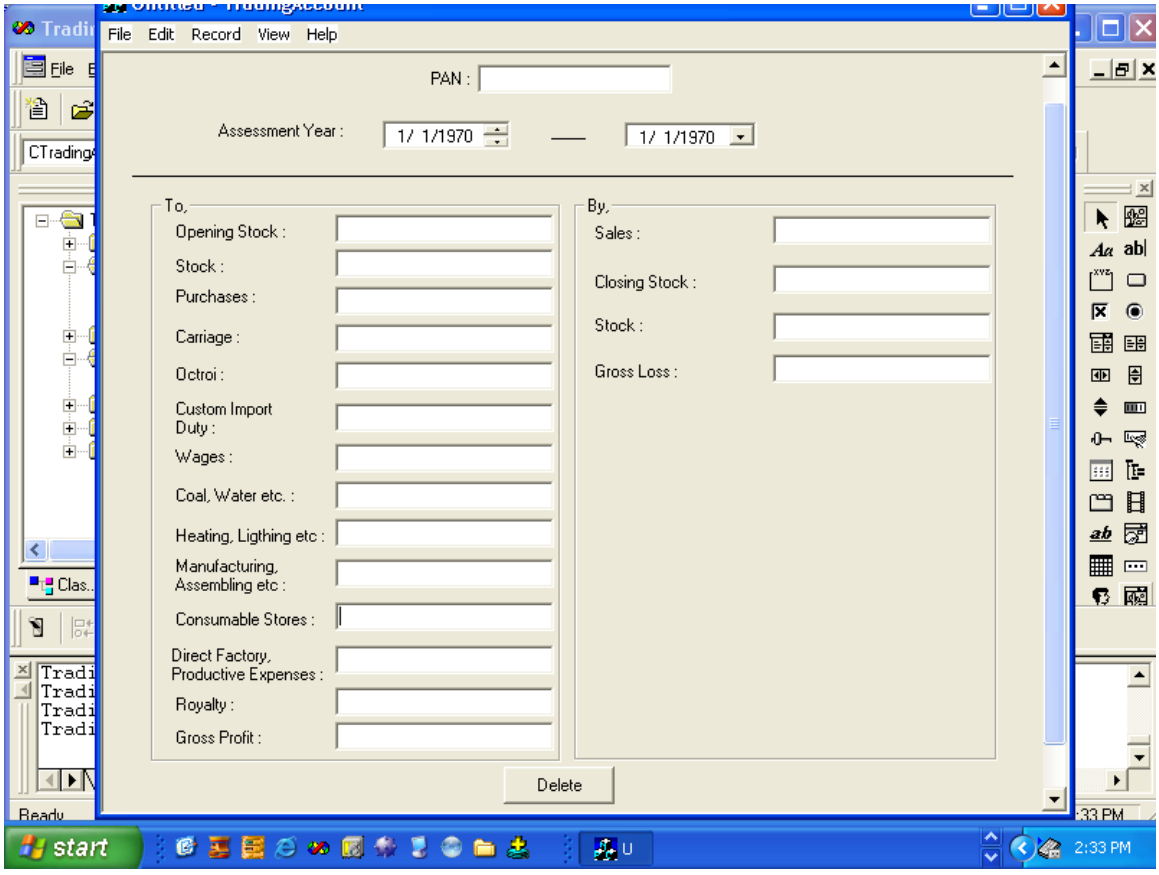
Form used for Trading Account is as follows: -



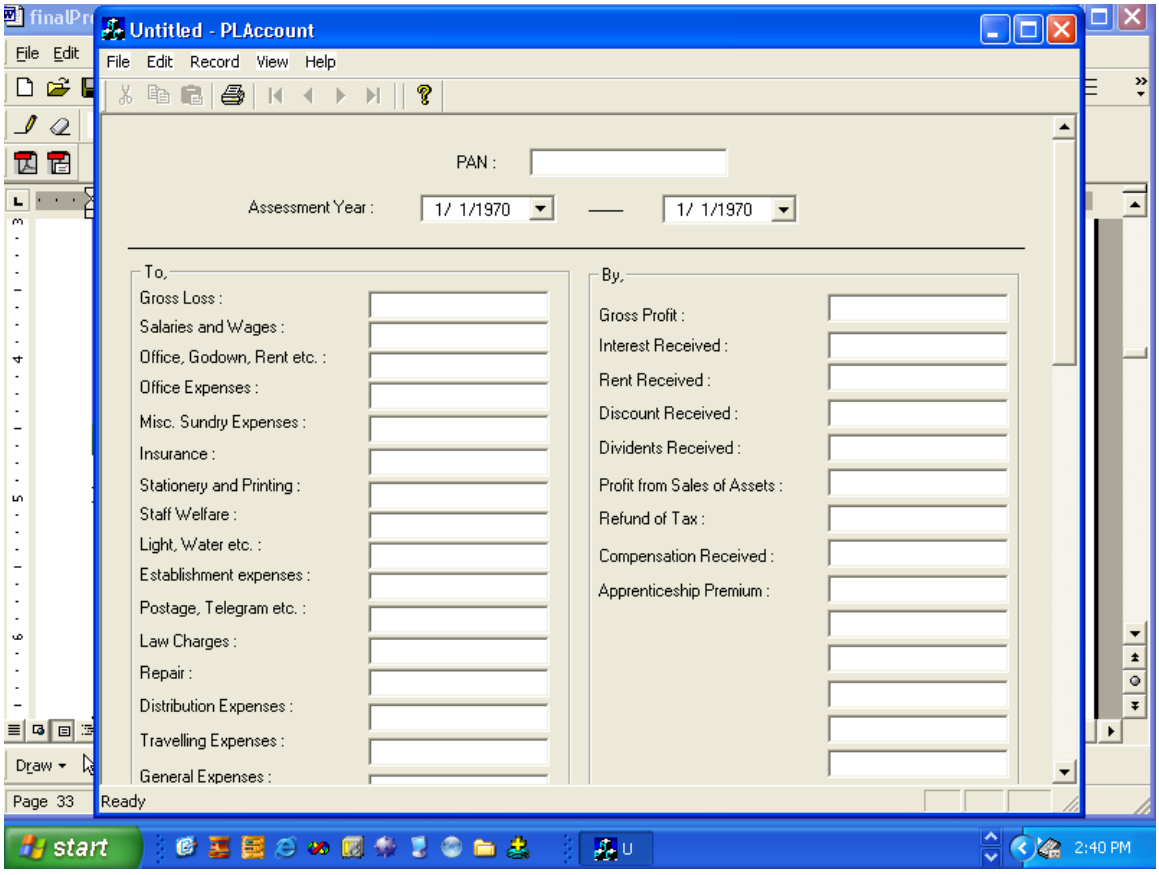
Form used for updating Trading Account is as follows: -

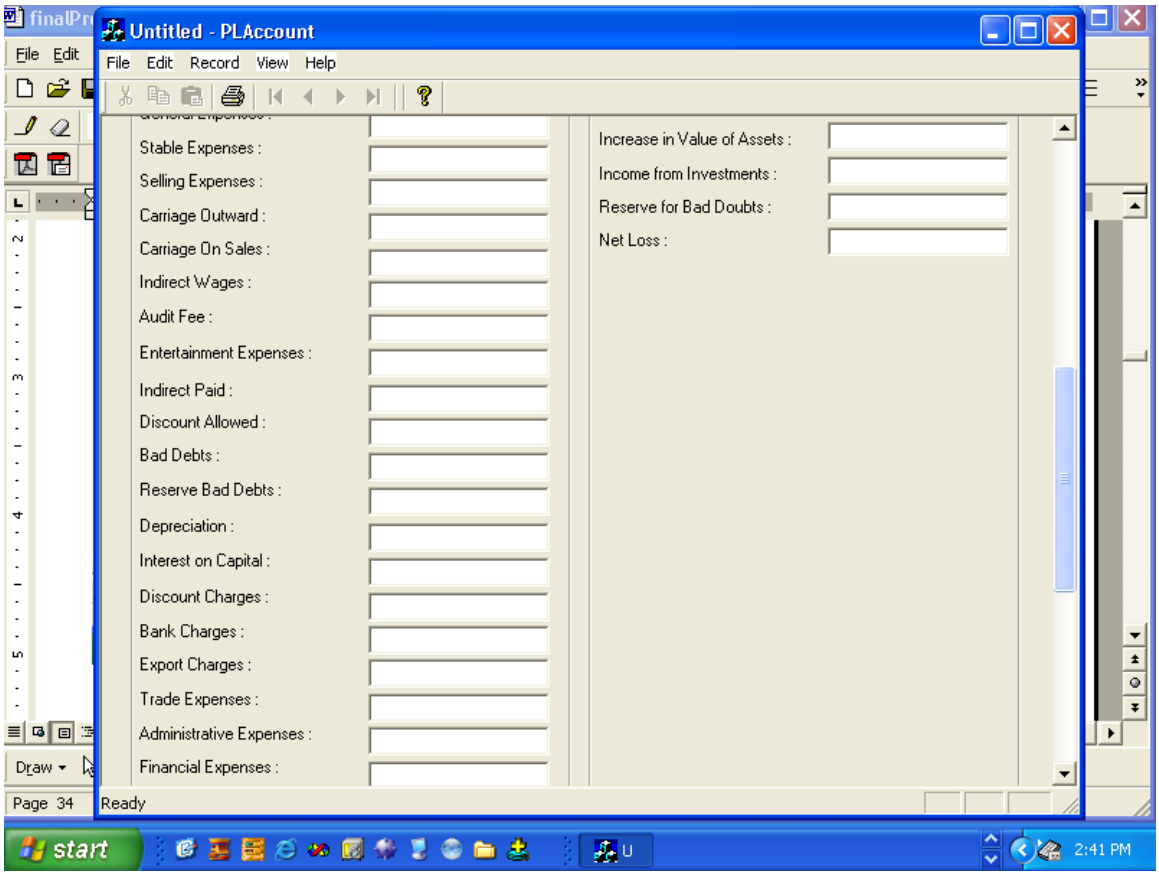


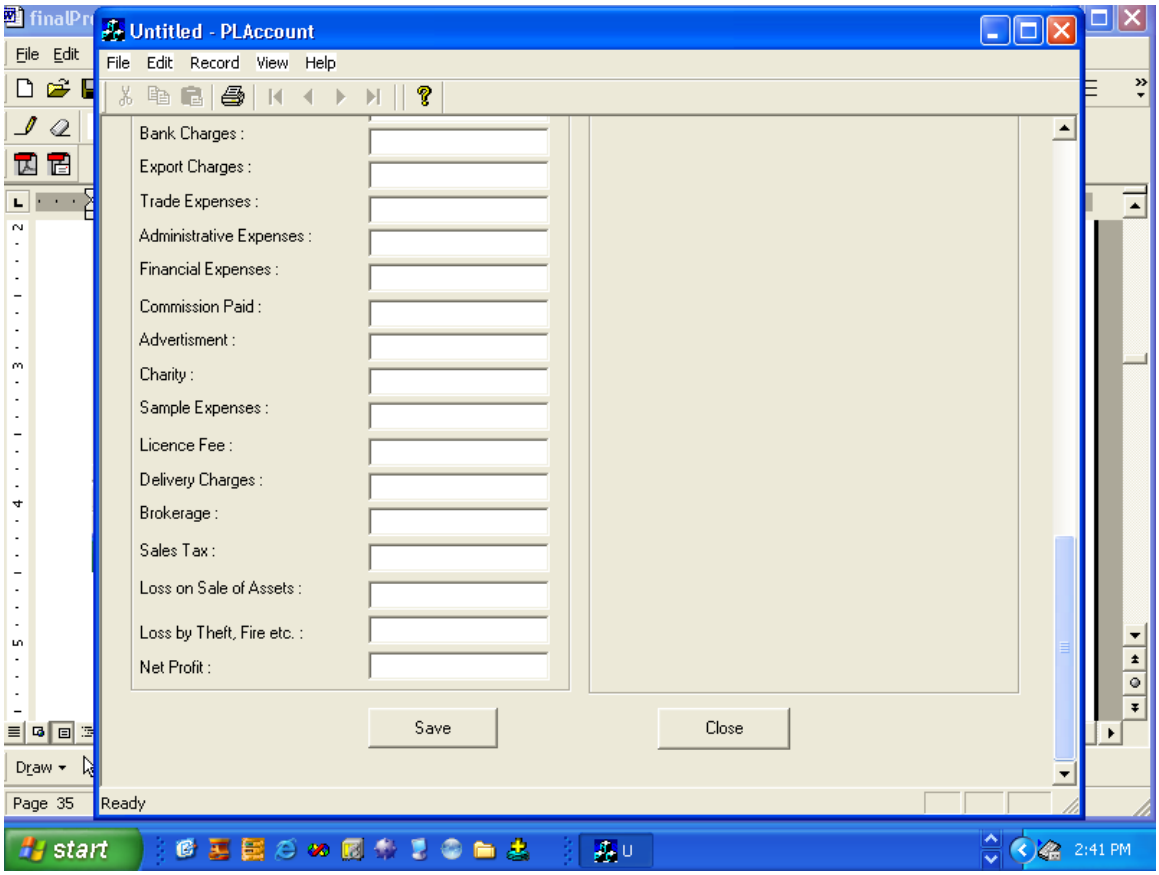
Form used for deleting trading account is as follows: -



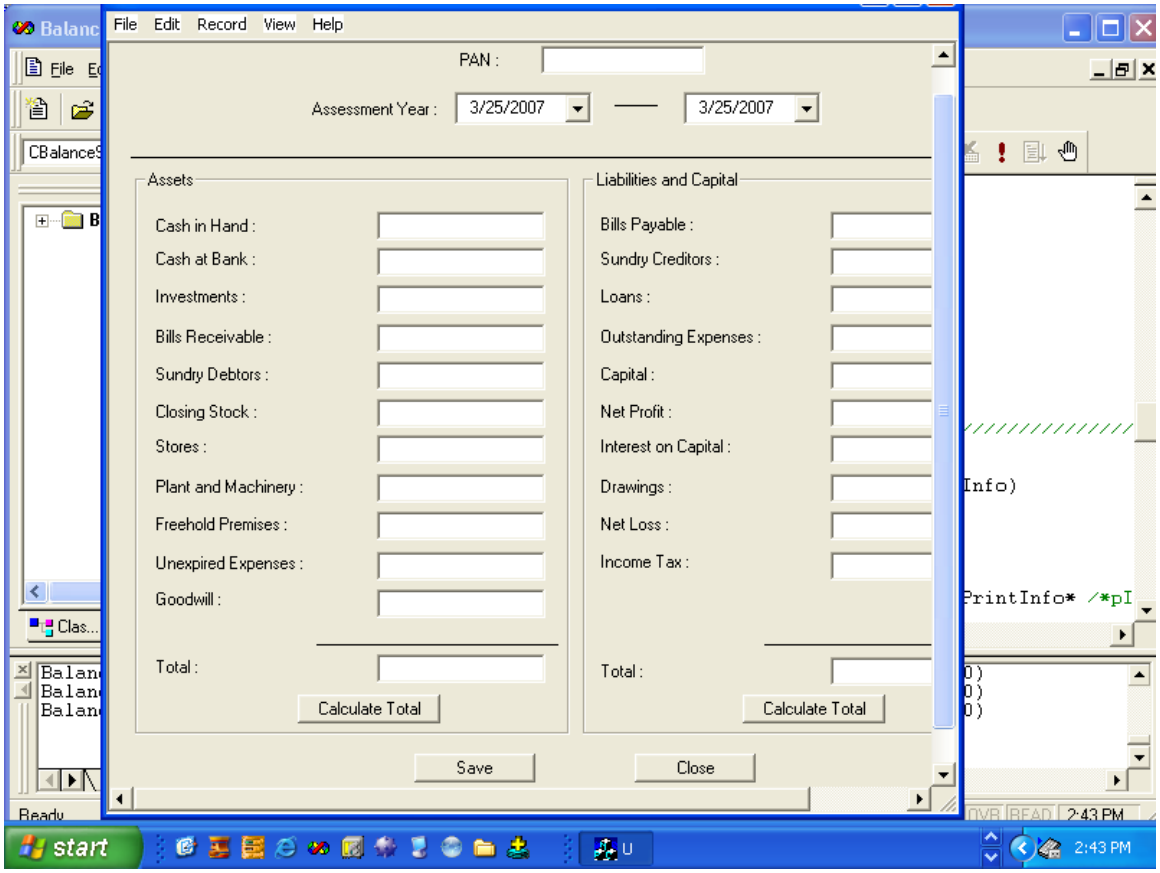
Form used for P/L Account is as follows: -



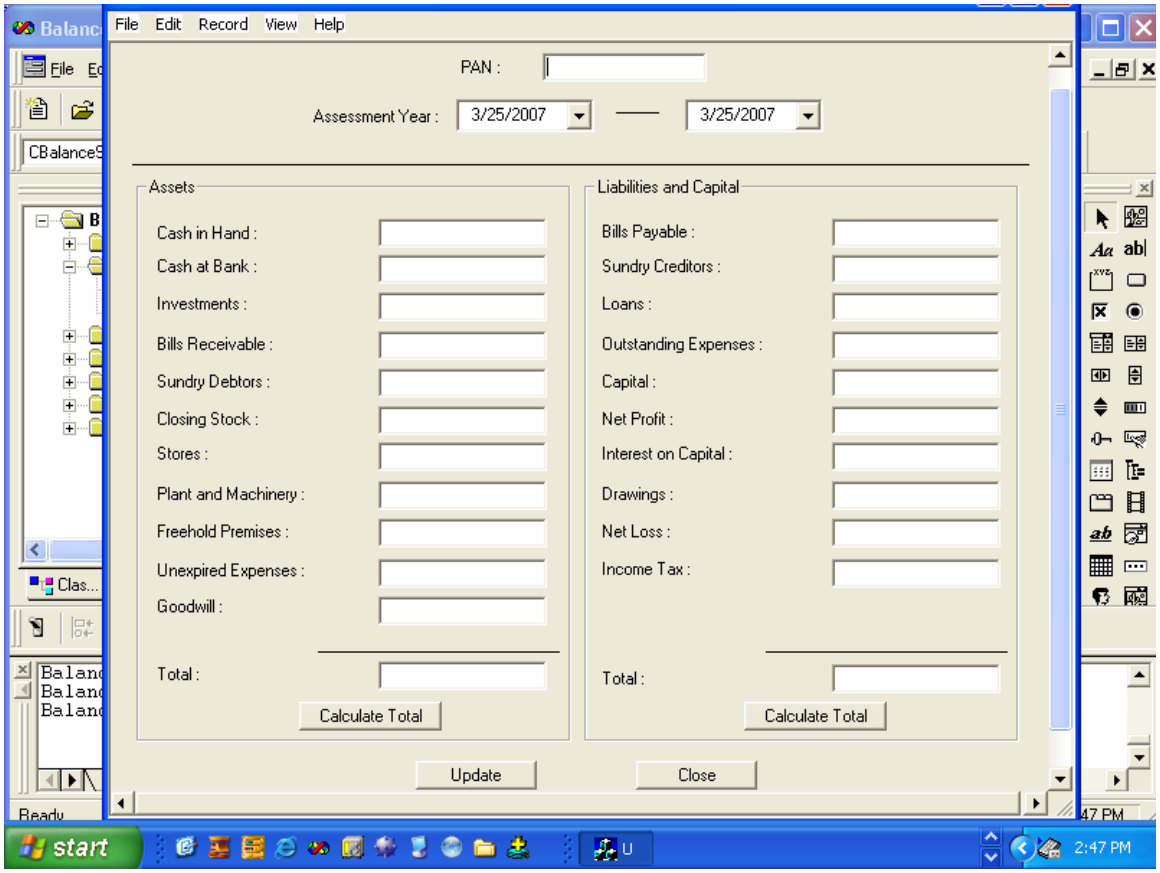

Untitled - PLAccount
File Edit Record View Help
Stable Expenses :
Selling Expenses :
Carriage Outward :
Carriage On Sales :
Indirect Wages :
Audit Fee :
Entertainment Expenses :
Indirect Paid :
Discount Allowed :
Bad Debts :
Reserve Bad Debts :
Depreciation :
Interest on Capital :
Discount Charges :
Bank Charges :
Export Charges :
Trade Expenses :
Administrative Expenses :
Financial Expenses :
Increase in Value of Assets :
Income from Investments :
Reserve for Bad Doubts :
Net Loss :
Ready
start
2:41 PM


Untitled - PLAccount
File Edit Record View Help
Bank Charges :
Export Charges :
Trade Expenses :
Administrative Expenses :
Financial Expenses :
Commission Paid :
Advertisment :
Charity :
Sample Expenses :
Licence Fee :
Delivery Charges :
Brokerage :
Sales Tax :
Loss on Sale of Assets :
Loss by Theft, Fire etc. :
Net Profit :
Save
Close
Ready
Page 35
start
2:41 PM

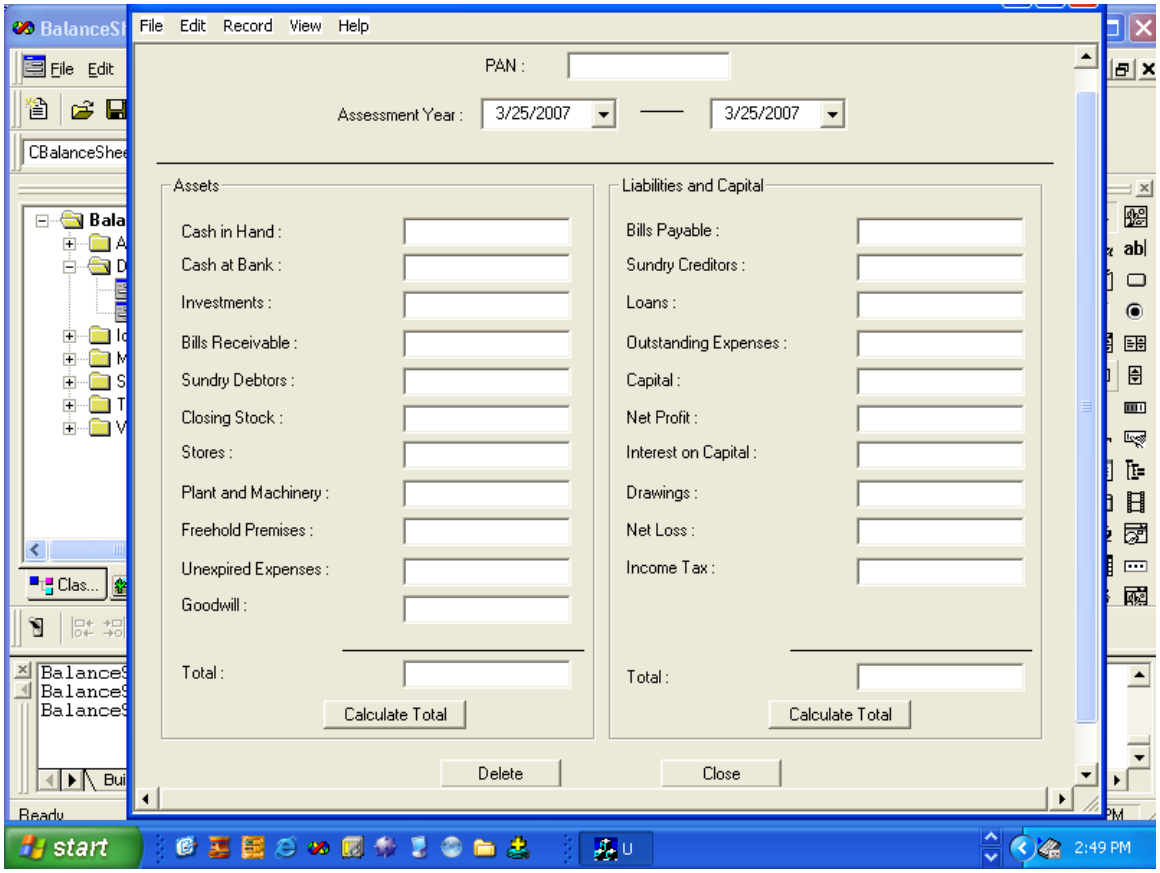
Form used for inserting Balance sheet is as follows: -



Form used for updating balance sheet is as follows :-



Form used for deleting balance sheet is as follows: -



8. Coding:

Codes of all the form are given below. Validation codes are not written here because they are describing in **"Validation of Code"** and **"Optimization of Code"** section.

In general we use following code to Insert, Update, Delete the database and for Display report.

Following code will be used for connecting with database:

```
CString CBalanceSheetSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CBalanceSheetSet::GetDefaultSQL()
{
	return
_T("[DIGAMBER].[BALANCE_SHEET],[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[INCOME
_TAX_RECORD]");
}
```

Following code will insert values into database:

```
void CBalanceSheetSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CBalanceSheetSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[BALANCE_SHEET].[PAN]"),
m_BALANCE_SHEET_PAN);
	RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
	RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
	RFX_Text(pFX, _T("[BILLS_PAYABLE]"), m_BILLS_PAYABLE);
	RFX_Text(pFX, _T("[SUNDRY_CREDITORS]"), m_SUNDRY_CREDITORS);
	RFX_Text(pFX, _T("[LOANS]"), m_LOANS);
	RFX_Text(pFX, _T("[OUTSTANDING_EXPENSES]"), m_OUTSTANDING_EXPENSES);
	RFX_Text(pFX, _T("[CAPITAL]"), m_CAPITAL);
	RFX_Text(pFX, _T("[NET_PROFIT]"), m_NET_PROFIT);
	RFX_Text(pFX, _T("[INTEREST_ON_CAPITAL]"), m_INTEREST_ON_CAPITAL);
	RFX_Text(pFX, _T("[DRAWINGS]"), m_DRAWINGS);
	RFX_Text(pFX, _T("[NET_LOSS]"), m_NET_LOSS);
	RFX_Text(pFX, _T("[INCOME_TAX]"), m_INCOME_TAX);
	RFX_Text(pFX, _T("[CASH_IN_HAND]"), m_CASH_IN_HAND);
	RFX_Text(pFX, _T("[CASH_AT_BANK]"), m_CASH_AT_BANK);
	RFX_Text(pFX, _T("[INVESTMENTS]"), m_INVESTMENTS);
	RFX_Text(pFX, _T("[BILLS_RECEIVABLE]"), m_BILLS_RECEIVABLE);
	RFX_Text(pFX, _T("[SUNDRY_DEBTORS]"), m_SUNDRY_DEBTORS);
	RFX_Text(pFX, _T("[CLOSING_STOCK]"), m_CLOSING_STOCK);
	RFX_Text(pFX, _T("[STORES]"), m_STORES);
```

```
	RFX_Text(pFX, _T("[PLANT_AND_MACHINERY]"), m_PLANT_AND_MACHINERY);
	RFX_Text(pFX, _T("[FREEHOLD_PREMISES]"), m_FREEHOLD_PREMISES);
	RFX_Text(pFX, _T("[UNEXPIRED_EXPENSES]"), m_UNEXPIRED_EXPENSES);
	RFX_Text(pFX, _T("[GOODWILL]"), m_GOODWILL);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	//}}AFX_FIELD_MAP
}
```

Following code will update the database:

```
CBalanceSheetSet::CBalanceSheetSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CBalanceSheetSet)
	m_BALANCE_SHEET_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_BILLS_PAYABLE = _T("");
	m_SUNDRY_CREDITORS = _T("");
	m_LOANS = _T("");
	m_OUTSTANDING_EXPENSES = _T("");
	m_CAPITAL = _T("");
	m_NET_PROFIT = _T("");
	m_INTEREST_ON_CAPITAL = _T("");
	m_DRAWINGS = _T("");
	m_NET_LOSS = _T("");
	m_INCOME_TAX = _T("");
	m_CASH_IN_HAND = _T("");
	m_CASH_AT_BANK = _T("");
	m_INVESTMENTS = _T("");
	m_BILLS_RECEIVABLE = _T("");
	m_SUNDRY_DEBTORS = _T("");
	m_CLOSING_STOCK = _T("");
	m_STORES = _T("");
	m_PLANT_AND_MACHINERY = _T("");
	m_FREEHOLD_PREMISES = _T("");
	m_UNEXPIRED_EXPENSES = _T("");
	m_GOODWILL = _T("");
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
```

```
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_nFields = 35;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}
```

Code of Main Menu:

```
// InsertClientRecord.cpp : implementation file
#include "stdafx.h"
#include "MCAPROJECT.h"
#include "InsertClientRecord.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
// CInsertClientRecord
IMPLEMENT_DYNAMIC(CInsertClientRecord, CRecordset)
CInsertClientRecord::CInsertClientRecord(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CInsertClientRecord)
	m_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_nFields = 11;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}


CString CInsertClientRecord::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CInsertClientRecord::GetDefaultSQL()
{
	return _T("[DIGAMBER].[CLIENT_RECORD]");
}

void CInsertClientRecord::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CInsertClientRecord)
```

```
		pFX->SetFieldType(CFieldExchange::outputColumn);
		RFX_Text(pFX, _T("[PAN]"), m_PAN);
		RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
		RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
		RFX_Date(pFX, _T("[DOB]"), m_DOB);
		RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
		RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
		RFX_Text(pFX, _T("[SEX]"), m_SEX);
		RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
		RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
		RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
		RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
		//}}AFX_FIELD_MAP
}
// CInsertClientRecord diagnostics
#ifdef _DEBUG
void CInsertClientRecord::AssertValid() const
{
		CRecordset::AssertValid();
}

void CInsertClientRecord::Dump(CDumpContext& dc) const
{
		CRecordset::Dump(dc);
}
#endif //_DEBUG
// MainFrm.cpp : implementation of the CMainFrame class
//

#include "stdafx.h"
#include "MCAPROJECT.h"
#include "MainFrm.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
// CMainFrame
IMPLEMENT_DYNCREATE(CMainFrame, CFrameWnd)
BEGIN_MESSAGE_MAP(CMainFrame, CFrameWnd)
		//{{AFX_MSG_MAP(CMainFrame)
		ON_WM_CREATE()
		ON_COMMAND(ID_CLIENT_INSERTCLIENTRECORD, OnClientInsertclientrecord)
		//}}AFX_MSG_MAP
END_MESSAGE_MAP()
static UINT indicators[] =
{
		ID_SEPARATOR,		// status line indicator
		ID_INDICATOR_CAPS,
		ID_INDICATOR_NUM,
		ID_INDICATOR_SCRL,
};
// CMainFrame construction/destruction
CMainFrame::CMainFrame()
```

```
{
	// TODO: add member initialization code here
}
CMainFrame::~CMainFrame()
{
}
int CMainFrame::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
	if (CFrameWnd::OnCreate(lpCreateStruct) == -1)
		return -1;

	if (!m_wndToolBar.CreateEx(this, TBSTYLE_FLAT, WS_CHILD | WS_VISIBLE | CBRS_TOP
		| CBRS_GRIPPER | CBRS_TOOLTIPS | CBRS_FLYBY | CBRS_SIZE_DYNAMIC) ||
		!m_wndToolBar.LoadToolBar(IDR_MAINFRAME))
	{
		TRACE0("Failed to create toolbar\n");
		return -1;	// fail to create
	}
	if (!m_wndStatusBar.Create(this) ||
		!m_wndStatusBar.SetIndicators(indicators,
		  sizeof(indicators)/sizeof(UINT)))
	{
		TRACE0("Failed to create status bar\n");
		return -1;	// fail to create
	}
	// TODO: Delete these three lines if you don't want the toolbar to
	//  be dockable
	m_wndToolBar.EnableDocking(CBRS_ALIGN_ANY);
	EnableDocking(CBRS_ALIGN_ANY);
	DockControlBar(&m_wndToolBar);
	return 0;
}
BOOL CMainFrame::PreCreateWindow(CREATESTRUCT& cs)
{
	if( !CFrameWnd::PreCreateWindow(cs) )
		return FALSE;
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs
	return TRUE;
}

// CMainFrame diagnostics
#ifdef _DEBUG
void CMainFrame::AssertValid() const
{
	CFrameWnd::AssertValid();
}
void CMainFrame::Dump(CDumpContext& dc) const
{
	CFrameWnd::Dump(dc);
}
#endif //_DEBUG
// CMainFrame message handlers
void CMainFrame::OnClientInsertclientrecord()
{
```

```
		// TODO: Add your command handler code here
		//exit(0);
		BOOL s;
		m_dCInsertClientRecord.m_PAN = 'pp';
		//if(m_dCInsertClientRecord.==IDOK){}

}
// MCAPROJECT.cpp : Defines the class behaviors for the application.
#include "stdafx.h"
#include "MCAPROJECT.h"
#include "MainFrm.h"
#include "MCAPROJECTSet.h"
#include "MCAPROJECTDoc.h"
#include "MCAPROJECTView.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
// CMCAPROJECTApp
BEGIN_MESSAGE_MAP(CMCAPROJECTApp, CWinApp)
		//{{AFX_MSG_MAP(CMCAPROJECTApp)
		ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
			// NOTE - the ClassWizard will add and remove mapping macros here.
			//    DO NOT EDIT what you see in these blocks of generated code!
		//}}AFX_MSG_MAP
		// Standard print setup command
		ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()
// CMCAPROJECTApp construction
CMCAPROJECTApp::CMCAPROJECTApp()
{
		// TODO: add construction code here,
		// Place all significant initialization in InitInstance
}
/////////////////////////////////////////////////////////////////////////////
// The one and only CMCAPROJECTApp object
CMCAPROJECTApp theApp;
/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTApp initialization
BOOL CMCAPROJECTApp::InitInstance()
{
		// Standard initialization
		// If you are not using these features and wish to reduce the size
		//  of your final executable, you should remove from the following
		//  the specific initialization routines you do not need.

#ifdef _AFXDLL
		Enable3dControls();			// Call this when using MFC in a shared DLL
#else
		Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif
		// Change the registry key under which our settings are stored.
		// TODO: You should modify this string to be something appropriate
		// such as the name of your company or organization.
```

```
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));
	LoadStdProfileSettings();  // Load standard INI file options (including MRU)
	// Register the application's document templates.  Document templates
	//  serve as the connection between documents, frame windows and views.
	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CMCAPROJECTDoc),
		RUNTIME_CLASS(CMainFrame),       // main SDI frame window
		RUNTIME_CLASS(CMCAPROJECTView));
	AddDocTemplate(pDocTemplate);
	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);
	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;
	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();
	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About
class CAboutDlg : public CDialog
{
public:
	CAboutDlg();
// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL
// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};
CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}
void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
```

```
	//}}AFX_DATA_MAP
}
BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()
// App command to run the dialog
void CMCAPROJECTApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
	aboutDlg.DoModal();
}
/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTApp message handlers

// MCAPROJECTDoc.cpp : implementation of the CMCAPROJECTDoc class
//

#include "stdafx.h"
#include "MCAPROJECT.h"

#include "MCAPROJECTSet.h"
#include "MCAPROJECTDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTDoc

IMPLEMENT_DYNCREATE(CMCAPROJECTDoc, CDocument)

BEGIN_MESSAGE_MAP(CMCAPROJECTDoc, CDocument)
	//{{AFX_MSG_MAP(CMCAPROJECTDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTDoc construction/destruction

CMCAPROJECTDoc::CMCAPROJECTDoc()
{
	// TODO: add one-time construction code here

}

CMCAPROJECTDoc::~CMCAPROJECTDoc()
{
}
```

```
BOOL CMCAPROJECTDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
}



/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTDoc diagnostics

#ifdef _DEBUG
void CMCAPROJECTDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CMCAPROJECTDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTDoc commands
// MCAPROJECTSet.cpp : implementation of the CMCAPROJECTSet class
//

#include "stdafx.h"
#include "MCAPROJECT.h"
#include "MCAPROJECTSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTSet implementation

IMPLEMENT_DYNAMIC(CMCAPROJECTSet, CRecordset)

CMCAPROJECTSet::CMCAPROJECTSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CMCAPROJECTSet)
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
```

```
}

CString CMCAPROJECTSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CMCAPROJECTSet::GetDefaultSQL()
{
	return
_T("[DIGAMBER].[BALANCE_SHEET],[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[INCOME
_TAX_RECORD],[DIGAMBER].[PL_ACCOUNT],[DIGAMBER].[TRADING_ACCOUNT]");
}

void CMCAPROJECTSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CMCAPROJECTSet)
		//}}AFX_FIELD_MAP
}

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTSet diagnostics

#ifdef _DEBUG
void CMCAPROJECTSet::AssertValid() const
{
	CRecordset::AssertValid();
}

void CMCAPROJECTSet::Dump(CDumpContext& dc) const
{
	CRecordset::Dump(dc);
}
#endif //_DEBUG

// MCAPROJECTView.cpp : implementation of the CMCAPROJECTView class
//

#include "stdafx.h"
#include "MCAPROJECT.h"

#include "MCAPROJECTSet.h"
#include "MCAPROJECTDoc.h"
#include "MCAPROJECTView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView

IMPLEMENT_DYNCREATE(CMCAPROJECTView, CRecordView)
```

```
BEGIN_MESSAGE_MAP(CMCAPROJECTView, CRecordView)
	//{{AFX_MSG_MAP(CMCAPROJECTView)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView construction/destruction

CMCAPROJECTView::CMCAPROJECTView()
	: CRecordView(CMCAPROJECTView::IDD)
{
	//{{AFX_DATA_INIT(CMCAPROJECTView)
		// NOTE: the ClassWizard will add member initialization here
	m_pSet = NULL;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CMCAPROJECTView::~CMCAPROJECTView()
{
}

void CMCAPROJECTView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CMCAPROJECTView)
		// NOTE: the ClassWizard will add DDX and DDV calls here
	//}}AFX_DATA_MAP
}

BOOL CMCAPROJECTView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs

	return CRecordView::PreCreateWindow(cs);
}

void CMCAPROJECTView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_mCAPROJECTSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}
```

```
/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView printing

BOOL CMCAPROJECTView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
}

void CMCAPROJECTView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
}

void CMCAPROJECTView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView diagnostics

#ifdef _DEBUG
void CMCAPROJECTView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CMCAPROJECTView::Dump(CDumpContext& dc) const
{
	CRecordView::Dump(dc);
}

CMCAPROJECTDoc* CMCAPROJECTView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CMCAPROJECTDoc)));
	return (CMCAPROJECTDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView database support
CRecordset* CMCAPROJECTView::OnGetRecordset()
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTView message handlers

// stdafx.cpp : source file that includes just the standard includes
//	MCAPROJECT.pch will be the pre-compiled header
//	stdafx.obj will contain the pre-compiled type information
```

```
#include "stdafx.h"

#if
!defined(AFX_INSERTCLIENTRECORD_H__B104479D_7C17_4C47_BD5B_E47BB1BABE23__INC
LUDED_)
#define
AFX_INSERTCLIENTRECORD_H__B104479D_7C17_4C47_BD5B_E47BB1BABE23__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
// InsertClientRecord.h : header file
//

/////////////////////////////////////////////////////////////////////////////
// CInsertClientRecord recordset

class CInsertClientRecord : public CRecordset
{
public:
	CInsertClientRecord(CDatabase* pDatabase = NULL);
	DECLARE_DYNAMIC(CInsertClientRecord)

// Field/Param Data
	//{{AFX_FIELD(CInsertClientRecord, CRecordset)
	CString  m_PAN;
	CString  m_CLIENT_NAME;
	CString  m_FATHERS_NAME;
	CTime   m_DOB;
	CString  m_ADDRESS;
	CString  m_TELEPHONE;
	CString  m_SEX;
	CString  m_INDV_HUF_FIRM_AOP_LA;
	CString  m_RESIDENT_NR_NOR;
	CString  m_WARD_CIRCLE_SPECIAL_RANGE;
	CString  m_PINCODE;
	//}}AFX_FIELD


// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CInsertClientRecord)
	public:
	virtual CString GetDefaultConnect();    // Default connection string
	virtual CString GetDefaultSQL();    // Default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);  // RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif
};
```

```
//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_INSERTCLIENTRECORD_H__B104479D_7C17_4C47_BD5B_E47BB1BABE23__INC
LUDED_)

// MainFrm.h : interface of the CMainFrame class
//
/////////////////////////////////////////////////////////////////////////////

#if !defined(AFX_MAINFRM_H__EB46766B_1BCE_4F08_B3F3_76782732CC5B__INCLUDED_)
#define AFX_MAINFRM_H__EB46766B_1BCE_4F08_B3F3_76782732CC5B__INCLUDED_

#include "InsertClientRecord.h"      // Added by ClassView
#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CMainFrame : public CFrameWnd
{

protected: // create from serialization only
        CMainFrame();
        DECLARE_DYNCREATE(CMainFrame)

// Attributes
public:

// Operations
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CMainFrame)
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CMainFrame();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:  // control bar embedded members
        CStatusBar  m_wndStatusBar;
        CToolBar    m_wndToolBar;

// Generated message map functions
protected:
        //{{AFX_MSG(CMainFrame)
        afx_msg int OnCreate(LPCREATESTRUCT lpCreateStruct);
        afx_msg void OnClientInsertclientrecord();
```

```
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
private:
	CInsertClientRecord m_dCInsertClientRecord;
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MAINFRM_H__EB46766B_1BCE_4F08_B3F3_76782732CC5B__INCLUDED_)

// MCAPROJECT.h : main header file for the MCAPROJECT application
//

#if !defined(AFX_MCAPROJECT_H__962C5B1B_E546_4E2F_A153_66A0B877BB20__INCLUDED_)
#define AFX_MCAPROJECT_H__962C5B1B_E546_4E2F_A153_66A0B877BB20__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
	#error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"		// main symbols

/////////////////////////////////////////////////////////////////////////////
// CMCAPROJECTApp:
// See MCAPROJECT.cpp for the implementation of this class
//

class CMCAPROJECTApp : public CWinApp
{
public:
	CMCAPROJECTApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CMCAPROJECTApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CMCAPROJECTApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};
```

```
/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MCAPROJECT_H__962C5B1B_E546_4E2F_A153_66A0B877BB20__INCLUDED_)

// MCAPROJECTDoc.h : interface of the CMCAPROJECTDoc class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_MCAPROJECTDOC_H__DEA89B3F_20F9_431E_AD39_8F95ECB30114__INCLUDED
_)
#define AFX_MCAPROJECTDOC_H__DEA89B3F_20F9_431E_AD39_8F95ECB30114__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "MCAPROJECTSet.h"


class CMCAPROJECTDoc : public CDocument
{
protected: // create from serialization only
	CMCAPROJECTDoc();
	DECLARE_DYNCREATE(CMCAPROJECTDoc)

// Attributes
public:
	CMCAPROJECTSet m_mCAPROJECTSet;

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CMCAPROJECTDoc)
	public:
	virtual BOOL OnNewDocument();
	//}}AFX_VIRTUAL

// Implementation
public:
	virtual ~CMCAPROJECTDoc();
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

protected:
```

```
// Generated message map functions
protected:
	//{{AFX_MSG(CMCAPROJECTDoc)
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MCAPROJECTDOC_H__DEA89B3F_20F9_431E_AD39_8F95ECB30114__INCLUDED
_)

// MCAPROJECTSet.h : interface of the CMCAPROJECTSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_MCAPROJECTSET_H__C014481A_F50C_4F78_80D1_4F501A077357__INCLUDED_)
#define AFX_MCAPROJECTSET_H__C014481A_F50C_4F78_80D1_4F501A077357__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CMCAPROJECTSet : public CRecordset
{
public:
	CMCAPROJECTSet(CDatabase* pDatabase = NULL);
	DECLARE_DYNAMIC(CMCAPROJECTSet)

// Field/Param Data
	//{{AFX_FIELD(CMCAPROJECTSet, CRecordset)

	//}}AFX_FIELD

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CMCAPROJECTSet)
	public:
	virtual CString GetDefaultConnect();	// Default connection string
	virtual CString GetDefaultSQL();	// default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);	// RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif
```

```
};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MCAPROJECTSET_H__C014481A_F50C_4F78_80D1_4F501A077357__INCLUDED_)
// MCAPROJECTView.h : interface of the CMCAPROJECTView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_MCAPROJECTVIEW_H__1FC265EC_0D82_4487_95A6_23950C8E6E8B__INCLUDED
_)
#define AFX_MCAPROJECTVIEW_H__1FC265EC_0D82_4487_95A6_23950C8E6E8B__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CMCAPROJECTSet;

class CMCAPROJECTView : public CRecordView
{
protected: // create from serialization only
        CMCAPROJECTView();
        DECLARE_DYNCREATE(CMCAPROJECTView)

public:
        //{{AFX_DATA(CMCAPROJECTView)
        enum{ IDD = IDD_MCAPROJECT_FORM };
        CMCAPROJECTSet* m_pSet;
                // NOTE: the ClassWizard will add data members here
        //}}AFX_DATA

// Attributes
public:
        CMCAPROJECTDoc* GetDocument();

// Operations
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CMCAPROJECTView)
        public:
        virtual CRecordset* OnGetRecordset();
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
        protected:
        virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
        virtual void OnInitialUpdate(); // called first time after construct
        virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
        virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
```

```
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CMCAPROJECTView();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CMCAPROJECTView)
                // NOTE - the ClassWizard will add and remove member functions here.
                //    DO NOT EDIT what you see in these blocks of generated code !
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in MCAPROJECTView.cpp
inline CMCAPROJECTDoc* CMCAPROJECTView::GetDocument()
   { return (CMCAPROJECTDoc*)m_pDocument; }
#endif

////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MCAPROJECTVIEW_H__1FC265EC_0D82_4487_95A6_23950C8E6E8B__INCLUDED
_)

//{{NO_DEPENDENCIES}}
// Microsoft Developer Studio generated include file.
// Used by MCAPROJECT.rc
//
#define IDD_ABOUTBOX                    100
#define IDD_MCAPROJECT_FORM             101
#define IDD_FORMVIEW                    102
#define IDP_FAILED_OPEN_DATABASE        103
#define IDR_MAINFRAME                   128
#define IDR_MCAPROTYPE                  129
#define IDC_CLIENT_PAN                  1000
#define IDC_BUTTON1                     1001
#define IDC_OK                          1001
#define ID_BUTTON32773                  32773
#define ID_CLIENT_INSERTCLIENTRECORD    32774
#define ID_CLIENT_UPDATECLIENTRECORD    32775
#define ID_CLIENT_DELETECLIENTRECORD    32776
#define ID_RETURN_INSERTRETURNINFORMATION 32777
#define ID_RETURN_UPDATERETURNINFORMATION 32778
#define ID_RETURN_DELETERETURNINFORMATION 32779
```

```
#define ID_RETURN_INSERTTRADINGACCOUNT  32780
#define ID_RETURN_UPDATETRADINGACCOUNT  32781
#define ID_RETURN_DELETETRADINGACCOUNT  32782
#define ID_RETURN_INSERTPLACCOUNT       32783
#define ID_RETURN_UPDATEPLACCOUNT       32784
#define ID_RETURN_DELETEPLACCOUNT       32785
#define ID_RETURN_INSERTBALANCESHEET    32786
#define ID_RETURN_UPDATEBALANCESHEET    32787
#define ID_RETURN_DELETEBALANCESHEET    32788
#define ID_REPORT_ORIGINALREPORT        32789
#define ID_REPORT_REVISEDRETURN         32790
#define ID_REPORT_TRADINGRETURN         32791
#define ID_REPORT_PLACCOUNT             32792
#define ID_REPORT_BALANCERECORD         32793

// Next default values for new objects
//
#ifdef APSTUDIO_INVOKED
#ifndef APSTUDIO_READONLY_SYMBOLS
#define _APS_3D_CONTROLS                1
#define _APS_NEXT_RESOURCE_VALUE        130
#define _APS_NEXT_COMMAND_VALUE         32794
#define _APS_NEXT_CONTROL_VALUE         1002
#define _APS_NEXT_SYMED_VALUE           101
#endif
#endif

// stdafx.h : include file for standard system include files,
//  or project specific include files that are used frequently, but
//      are changed infrequently
//

#if !defined(AFX_STDAFX_H__F5BC60E5_119B_43BF_AB0E_373190FB6F48__INCLUDED_)
#define AFX_STDAFX_H__F5BC60E5_119B_43BF_AB0E_373190FB6F48__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#define VC_EXTRALEAN                    // Exclude rarely-used stuff from Windows headers

#include <afxwin.h>         // MFC core and standard components
#include <afxext.h>         // MFC extensions
#include <afxdb.h>                      // MFC ODBC database classes
#include <afxdtctl.h>                   // MFC support for Internet Explorer 4 Common Controls
#ifndef _AFX_NO_AFXCMN_SUPPORT
#include <afxcmn.h>                     // MFC support for Windows Common Controls
#endif // _AFX_NO_AFXCMN_SUPPORT


//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif // !defined(AFX_STDAFX_H__F5BC60E5_119B_43BF_AB0E_373190FB6F48__INCLUDED_)
```

```
/*++

Copyright (c) 1997-1998  Microsoft Corporation

Module Name:

   basetsd.h

Abstract:

   Type definitions for the basic sized types.

Author:

   Jeff Havens (jhavens)   23-Oct-1997

Revision History:

--*/

#ifndef _BASETSD_H_
#define _BASETSD_H_

#ifdef __cplusplus
extern "C" {
#endif

//
// The following types are guaranteed to be signed and 32 bits wide.
//

typedef int LONG32, *PLONG32;
typedef int INT32, *PINT32;

//
// The following types are guaranteed to be unsigned and 32 bits wide.
//

typedef unsigned int ULONG32, *PULONG32;
typedef unsigned int DWORD32, *PDWORD32;
typedef unsigned int UINT32, *PUINT32;

//
// The INT_PTR is guaranteed to be the same size as a pointer.  Its
// size with change with pointer size (32/64).  It should be used
// anywhere that a pointer is cast to an integer type. UINT_PTR is
// the unsigned variation.
//
// HALF_PTR is half the size of a pointer it intended for use with
// within strcuture which contain a pointer and two small fields.
// UHALF_PTR is the unsigned variation.
//

#ifdef _WIN64
```

```
typedef __int64 INT_PTR, *PINT_PTR;
typedef unsigned __int64 UINT_PTR, *PUINT_PTR;

#define MAXINT_PTR (0x7fffffffffffffffI64)
#define MININT_PTR (0x8000000000000000I64)
#define MAXUINT_PTR (0xffffffffffffffffUI64)

typedef unsigned int UHALF_PTR, *PUHALF_PTR;
typedef int HALF_PTR, *PHALF_PTR;

#define MAXUHALF_PTR (0xffffffffUL)
#define MAXHALF_PTR (0x7fffffffL)
#define MINHALF_PTR (0x80000000L)

#pragma warning(disable:4311)   // type cast truncation

#if !defined(__midl)
__inline
unsigned long
HandleToUlong(
   void *h
   )
{
   return((unsigned long) h );
}

__inline
unsigned long
PtrToUlong(
   void  *p
   )
{
   return((unsigned long) p );
}

__inline
unsigned short
PtrToUshort(
   void  *p
   )
{
   return((unsigned short) p );
}

__inline
long
PtrToLong(
   void  *p
   )
{
   return((long) p );
}

__inline
short
```

```
PtrToShort(
   void  *p
   )
{
   return((short) p );
}
#endif
#pragma warning(3:4311)   // type cast truncation

#else


typedef long INT_PTR, *PINT_PTR;
typedef unsigned long UINT_PTR, *PUINT_PTR;

#define MAXINT_PTR (0x7fffffffL)
#define MININT_PTR (0x80000000L)
#define MAXUINT_PTR (0xffffffffUL)

typedef unsigned short UHALF_PTR, *PUHALF_PTR;
typedef short HALF_PTR, *PHALF_PTR;

#define MAXUHALF_PTR 0xffff
#define MAXHALF_PTR 0x7fff
#define MINHALF_PTR 0x8000

#define HandleToUlong( h ) ((ULONG) (h) )
#define PtrToUlong( p ) ((ULONG) (p) )
#define PtrToLong( p ) ((LONG) (p) )
#define PtrToUshort( p ) ((unsigned short) (p) )
#define PtrToShort( p ) ((short) (p) )

#endif

//
// SIZE_T used for counts or ranges which need to span the range of
// of a pointer.  SSIZE_T is the signed variation.
//

typedef UINT_PTR SIZE_T, *PSIZE_T;
typedef INT_PTR SSIZE_T, *PSSIZE_T;

//
// The following types are guaranteed to be signed and 64 bits wide.
//

typedef __int64 LONG64, *PLONG64;
typedef __int64 INT64, *PINT64;


//
// The following types are guaranteed to be unsigned and 64 bits wide.
//

typedef unsigned __int64 ULONG64, *PULONG64;
```

```
typedef unsigned __int64 DWORD64, *PDWORD64;
typedef unsigned __int64 UINT64, *PUINT64;

#ifdef __cplusplus
}
#endif

#endif // _BASETSD_H_gfd
```

Code of Client Record Form:

```
// ClientRecord.cpp : Defines the class behaviors for the application.
//

#include "stdafx.h"
#include "ClientRecord.h"

#include "MainFrm.h"
#include "ClientRecordSet.h"
#include "ClientRecordDoc.h"
#include "ClientRecordView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CClientRecordApp

BEGIN_MESSAGE_MAP(CClientRecordApp, CWinApp)
	//{{AFX_MSG_MAP(CClientRecordApp)
	ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard print setup command
	ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CClientRecordApp construction

CClientRecordApp::CClientRecordApp()
{
	// TODO: add construction code here,
	// Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CClientRecordApp object

CClientRecordApp theApp;
```

```
/////////////////////////////////////////////////////////////////////////////
// CClientRecordApp initialization

BOOL CClientRecordApp::InitInstance()
{
	AfxEnableControlContainer();

	// Standard initialization
	// If you are not using these features and wish to reduce the size
	//  of your final executable, you should remove from the following
	//  the specific initialization routines you do not need.

#ifdef _AFXDLL
	Enable3dControls();			// Call this when using MFC in a shared DLL
#else
	Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif

	// Change the registry key under which our settings are stored.
	// TODO: You should modify this string to be something appropriate
	// such as the name of your company or organization.
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));

	LoadStdProfileSettings();  // Load standard INI file options (including MRU)

	// Register the application's document templates.  Document templates
	//  serve as the connection between documents, frame windows and views.

	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CClientRecordDoc),
		RUNTIME_CLASS(CMainFrame),	// main SDI frame window
		RUNTIME_CLASS(CClientRecordView));
	AddDocTemplate(pDocTemplate);

	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);

	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;

	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();

	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About
```

```
class CAboutDlg : public CDialog
{
public:
	CAboutDlg();

// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA

	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
	//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CClientRecordApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
	aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CClientRecordApp message handlers

// ClientRecordDoc.cpp : implementation of the CClientRecordDoc class
```

```
//

#include "stdafx.h"
#include "ClientRecord.h"

#include "ClientRecordSet.h"
#include "ClientRecordDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CClientRecordDoc

IMPLEMENT_DYNCREATE(CClientRecordDoc, CDocument)

BEGIN_MESSAGE_MAP(CClientRecordDoc, CDocument)
	//{{AFX_MSG_MAP(CClientRecordDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CClientRecordDoc construction/destruction

CClientRecordDoc::CClientRecordDoc()
{
	// TODO: add one-time construction code here

}

CClientRecordDoc::~CClientRecordDoc()
{
}

BOOL CClientRecordDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
}



/////////////////////////////////////////////////////////////////////////////
// CClientRecordDoc diagnostics
```

```
#ifdef _DEBUG
void CClientRecordDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CClientRecordDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CClientRecordDoc commands

// ClientRecordSet.cpp : implementation of the CClientRecordSet class
//

#include "stdafx.h"
#include "ClientRecord.h"
#include "ClientRecordSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CClientRecordSet implementation

IMPLEMENT_DYNAMIC(CClientRecordSet, CRecordset)

CClientRecordSet::CClientRecordSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CClientRecordSet)
	m_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_nFields = 11;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}

CString CClientRecordSet::GetDefaultConnect()
{
```

```
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CClientRecordSet::GetDefaultSQL()
{
	return _T("[DIGAMBER].[CLIENT_RECORD]");
}

void CClientRecordSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CClientRecordSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[PAN]"), m_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	//}}AFX_FIELD_MAP
}

/////////////////////////////////////////////////////////////////////////////
// CClientRecordSet diagnostics

#ifdef _DEBUG
void CClientRecordSet::AssertValid() const
{
	CRecordset::AssertValid();
}

void CClientRecordSet::Dump(CDumpContext& dc) const
{
	CRecordset::Dump(dc);
}
#endif //_DEBUG

// ClientRecordView.cpp : implementation of the CClientRecordView class
//

#include "stdafx.h"
#include "ClientRecord.h"

#include "ClientRecordSet.h"
#include "ClientRecordDoc.h"
#include "ClientRecordView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
```

```
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CClientRecordView

IMPLEMENT_DYNCREATE(CClientRecordView, CRecordView)

BEGIN_MESSAGE_MAP(CClientRecordView, CRecordView)
	//{{AFX_MSG_MAP(CClientRecordView)
	ON_COMMAND(ID_RECORD_ADD, OnRecordAdd)
	ON_COMMAND(ID_RECORD_DELETE, OnRecordDelete)
	ON_BN_CLICKED(IDC_SAVE, OnSave)
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CClientRecordView construction/destruction

CClientRecordView::CClientRecordView()
	: CRecordView(CClientRecordView::IDD)
{
	//{{AFX_DATA_INIT(CClientRecordView)
	m_pSet = NULL;
	m_iWARD = -1;
	m_iRESIDENT = -1;
	m_iSEX = -1;
	m_iINDIVIDUAL = -1;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CClientRecordView::~CClientRecordView()
{
}

void CClientRecordView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CClientRecordView)
	DDX_Radio(pDX, IDC_WARD, m_iWARD);
	DDX_Radio(pDX, IDC_RESIDENT, m_iRESIDENT);
	DDX_Radio(pDX, IDC_MALE, m_iSEX);
	DDX_FieldText(pDX, IDC_CLIENT_NAME, m_pSet->m_CLIENT_NAME, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_CLIENT_NAME, 30);
	DDX_FieldText(pDX, IDC_FATHER_NAME, m_pSet->m_FATHERS_NAME, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_FATHERS_NAME, 30);
	DDX_FieldText(pDX, IDC_PAN, m_pSet->m_PAN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_PAN, 10);
	DDX_FieldText(pDX, IDC_PIN, m_pSet->m_PINCODE, m_pSet);
```

```
	DDV_MaxChars(pDX, m_pSet->m_PINCODE, 6);
	DDX_FieldText(pDX, IDC_TELEPHONE, m_pSet->m_TELEPHONE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TELEPHONE, 15);
	DDX_DateTimeCtrl(pDX, IDC_DOB, m_pSet->m_DOB);
	DDX_FieldText(pDX, IDC_ADDRESS, m_pSet->m_ADDRESS, m_pSet);
	DDX_Radio(pDX, IDC_INDIVIDUAL, m_iINDIVIDUAL);
	//}}AFX_DATA_MAP
}

BOOL CClientRecordView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs

	return CRecordView::PreCreateWindow(cs);
}

void CClientRecordView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_clientRecordSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}

/////////////////////////////////////////////////////////////////////////////
// CClientRecordView printing

BOOL CClientRecordView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
}

void CClientRecordView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
}

void CClientRecordView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CClientRecordView diagnostics

#ifdef _DEBUG
void CClientRecordView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CClientRecordView::Dump(CDumpContext& dc) const
```

```
{
	CRecordView::Dump(dc);
}

CClientRecordDoc* CClientRecordView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CClientRecordDoc)));
	return (CClientRecordDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CClientRecordView database support
CRecordset* CClientRecordView::OnGetRecordset()
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CClientRecordView message handlers

void CClientRecordView::OnRecordAdd()
{
	// TODO: Add your command handler code here
	m_pSet->AddNew();

	UpdateData(FALSE);

}

void CClientRecordView::OnRecordDelete()
{
	// TODO: Add your command handler code here
	m_pSet->Delete();
	m_pSet->Requery();
	m_pSet->MoveNext();
	if(m_pSet->IsEOF())
		m_pSet->MoveLast();

	if(m_pSet->IsBOF())
		m_pSet->SetFieldNull(NULL);

	UpdateData(FALSE);


}

void CClientRecordView::OnSave()
{
	// TODO: Add your control notification handler code here
	UpdateData(true);

	if(m_pSet->CanUpdate()) {
		m_pSet->Update();
```

```
	}

	m_pSet->Requery();

	UpdateData(false);
}

// ClientRecord.h : main header file for the CLIENTRECORD application
//

#if
!defined(AFX_CLIENTRECORD_H__7CA0CDF6_14D2_4AE8_BF68_327DFE44AF3B__INCLUDED_
)
#define AFX_CLIENTRECORD_H__7CA0CDF6_14D2_4AE8_BF68_327DFE44AF3B__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
	#error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"		// main symbols

/////////////////////////////////////////////////////////////////////////////
// CClientRecordApp:
// See ClientRecord.cpp for the implementation of this class
//

class CClientRecordApp : public CWinApp
{
public:
	CClientRecordApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CClientRecordApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CClientRecordApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
```

```
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_CLIENTRECORD_H__7CA0CDF6_14D2_4AE8_BF68_327DFE44AF3B__INCLUDED_
)

// ClientRecordDoc.h : interface of the CClientRecordDoc class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_CLIENTRECORDDOC_H__A94F9B41_1ABA_44EF_A120_18EEA9C6197D__INCLUD
ED_)
#define
AFX_CLIENTRECORDDOC_H__A94F9B41_1ABA_44EF_A120_18EEA9C6197D__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "ClientRecordSet.h"


class CClientRecordDoc : public CDocument
{
protected: // create from serialization only
	CClientRecordDoc();
	DECLARE_DYNCREATE(CClientRecordDoc)

// Attributes
public:
	CClientRecordSet m_clientRecordSet;

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CClientRecordDoc)
	public:
	virtual BOOL OnNewDocument();
	//}}AFX_VIRTUAL

// Implementation
public:
	virtual ~CClientRecordDoc();
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
	//{{AFX_MSG(CClientRecordDoc)
```

```
                // NOTE - the ClassWizard will add and remove member functions here.
                //    DO NOT EDIT what you see in these blocks of generated code !
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_CLIENTRECORDDOC_H__A94F9B41_1ABA_44EF_A120_18EEA9C6197D__INCLUD
ED_)

// ClientRecordSet.h : interface of the CClientRecordSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_CLIENTRECORDSET_H__AF6AB8BF_26E0_431C_A251_37AC92761914__INCLUDE
D_)
#define
AFX_CLIENTRECORDSET_H__AF6AB8BF_26E0_431C_A251_37AC92761914__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CClientRecordSet : public CRecordset
{
public:
        CClientRecordSet(CDatabase* pDatabase = NULL);
        DECLARE_DYNAMIC(CClientRecordSet)

// Field/Param Data
        //{{AFX_FIELD(CClientRecordSet, CRecordset)
        CString  m_PAN;
        CString  m_CLIENT_NAME;
        CString  m_FATHERS_NAME;
        CTime   m_DOB;
        CString  m_ADDRESS;
        CString  m_TELEPHONE;
        CString  m_SEX;
        CString  m_INDV_HUF_FIRM_AOP_LA;
        CString  m_RESIDENT_NR_NOR;
        CString  m_WARD_CIRCLE_SPECIAL_RANGE;
        CString  m_PINCODE;
        //}}AFX_FIELD

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CClientRecordSet)
        public:
        virtual CString GetDefaultConnect();        // Default connection string
```

```
	virtual CString GetDefaultSQL();	// default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);		// RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_CLIENTRECORDSET_H__AF6AB8BF_26E0_431C_A251_37AC92761914__INCLUDE
D_)

// ClientRecordView.h : interface of the CClientRecordView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_CLIENTRECORDVIEW_H__50EA76F4_A769_4744_B6DA_47D9E3F3E87F__INCLUD
ED_)
#define
AFX_CLIENTRECORDVIEW_H__50EA76F4_A769_4744_B6DA_47D9E3F3E87F__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CClientRecordSet;

class CClientRecordView : public CRecordView
{
protected: // create from serialization only
	CClientRecordView();
	DECLARE_DYNCREATE(CClientRecordView)

public:
	//{{AFX_DATA(CClientRecordView)
	enum { IDD = IDD_CLIENTRECORD_FORM };
	CClientRecordSet* m_pSet;
	int		m_iWARD;
	int		m_iRESIDENT;
	int		m_iSEX;
	int		m_iINDIVIDUAL;
	//}}AFX_DATA

// Attributes
public:
	CClientRecordDoc* GetDocument();
```

```
// Operations
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CClientRecordView)
        public:
        virtual CRecordset* OnGetRecordset();
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
        protected:
        virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
        virtual void OnInitialUpdate(); // called first time after construct
        virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
        virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CClientRecordView();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CClientRecordView)
        afx_msg void OnRecordAdd();
        afx_msg void OnRecordDelete();
        afx_msg void OnSave();
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in ClientRecordView.cpp
inline CClientRecordDoc* CClientRecordView::GetDocument()
   { return (CClientRecordDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_CLIENTRECORDVIEW_H__50EA76F4_A769_4744_B6DA_47D9E3F3E87F__INCLUD
ED_)
// MainFrm.h : interface of the CMainFrame class
//
/////////////////////////////////////////////////////////////////////////////

#if !defined(AFX_MAINFRM_H__4D01D3C1_6D15_4596_A2BF_E16B3B09D833__INCLUDED_)
```

```
#define AFX_MAINFRM_H__4D01D3C1_6D15_4596_A2BF_E16B3B09D833__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CMainFrame : public CFrameWnd
{

protected: // create from serialization only
	CMainFrame();
	DECLARE_DYNCREATE(CMainFrame)

// Attributes
public:

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CMainFrame)
	virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
	//}}AFX_VIRTUAL

// Implementation
public:
	virtual ~CMainFrame();
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

protected:  // control bar embedded members
	CStatusBar  m_wndStatusBar;
	CToolBar    m_wndToolBar;

// Generated message map functions
protected:
	//{{AFX_MSG(CMainFrame)
	afx_msg int OnCreate(LPCREATESTRUCT lpCreateStruct);
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_MAINFRM_H__4D01D3C1_6D15_4596_A2BF_E16B3B09D833__INCLUDED_)
//{{NO_DEPENDENCIES}}
// Microsoft Developer Studio generated include file.
// Used by ClientRecord.rc
```

```
//
#define IDD_ABOUTBOX                    100
#define IDD_CLIENTRECORD_FORM           101
#define IDP_FAILED_OPEN_DATABASE        103
#define IDR_MAINFRAME                   128
#define IDR_CLIENTTYPE                  129
#define IDC_CLIENT_NAME                 1000
#define IDC_FATHER_NAME                 1001
#define IDC_ADDRESS                     1002
#define IDC_PIN                         1003
#define IDC_TELEPHONE                   1004
#define IDC_DOB                         1006
#define IDC_RESIDENT                    1007
#define IDC_NON_RESIDENT                1008
#define IDC_NOT_ORDINARLY_RESIDENT      1009
#define IDC_MALE                        1010
#define IDC_FEMALE                      1011
#define IDC_WARD                        1012
#define IDC_RADIO7                      1013
#define IDC_SPCL_RANGE                  1014
#define IDC_PAN                         1015
#define IDC_INDIVIDUAL                  1016
#define IDC_HUF                         1017
#define IDC_FIRM                        1018
#define IDC_AOP                         1019
#define IDC_LA                          1020
#define IDC_SAVE                        1021
#define ID_RECORD_ADD                   32771
#define ID_RECORD_DELETE                32772

// Next default values for new objects
//
#ifdef APSTUDIO_INVOKED
#ifndef APSTUDIO_READONLY_SYMBOLS
#define _APS_3D_CONTROLS                1
#define _APS_NEXT_RESOURCE_VALUE        130
#define _APS_NEXT_COMMAND_VALUE         32773
#define _APS_NEXT_CONTROL_VALUE         1022
#define _APS_NEXT_SYMED_VALUE           101
#endif
#endif
```

Code of Income Tax Record Form:

```
// IncomeTaxRecord.cpp : Defines the class behaviors for the application.
//

#include "stdafx.h"
#include "IncomeTaxRecord.h"

#include "MainFrm.h"
#include "IncomeTaxRecordSet.h"
#include "IncomeTaxRecordDoc.h"
#include "IncomeTaxRecordView.h"

#ifdef _DEBUG
```

```
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp

BEGIN_MESSAGE_MAP(CIncomeTaxRecordApp, CWinApp)
	//{{AFX_MSG_MAP(CIncomeTaxRecordApp)
	ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard print setup command
	ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp construction

CIncomeTaxRecordApp::CIncomeTaxRecordApp()
{
	// TODO: add construction code here,
	// Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CIncomeTaxRecordApp object

CIncomeTaxRecordApp theApp;

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp initialization

BOOL CIncomeTaxRecordApp::InitInstance()
{
	AfxEnableControlContainer();

	// Standard initialization
	// If you are not using these features and wish to reduce the size
	//  of your final executable, you should remove from the following
	//  the specific initialization routines you do not need.

#ifdef _AFXDLL
	Enable3dControls();			// Call this when using MFC in a shared DLL
#else
	Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif

	// Change the registry key under which our settings are stored.
	// TODO: You should modify this string to be something appropriate
	// such as the name of your company or organization.
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));
```

```
	LoadStdProfileSettings();  // Load standard INI file options (including MRU)

	// Register the application's document templates.  Document templates
	//  serve as the connection between documents, frame windows and views.

	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CIncomeTaxRecordDoc),
		RUNTIME_CLASS(CMainFrame),       // main SDI frame window
		RUNTIME_CLASS(CIncomeTaxRecordView));
	AddDocTemplate(pDocTemplate);

	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);

	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;

	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();

	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
	CAboutDlg();

// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA

	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};
```

```
CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
	//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CIncomeTaxRecordApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
	aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp message handlers

// IncomeTaxRecordDoc.cpp : implementation of the CIncomeTaxRecordDoc class
//

#include "stdafx.h"
#include "IncomeTaxRecord.h"

#include "IncomeTaxRecordSet.h"
#include "IncomeTaxRecordDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordDoc

IMPLEMENT_DYNCREATE(CIncomeTaxRecordDoc, CDocument)

BEGIN_MESSAGE_MAP(CIncomeTaxRecordDoc, CDocument)
	//{{AFX_MSG_MAP(CIncomeTaxRecordDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
```

```
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordDoc construction/destruction

CIncomeTaxRecordDoc::CIncomeTaxRecordDoc()
{
	// TODO: add one-time construction code here

}

CIncomeTaxRecordDoc::~CIncomeTaxRecordDoc()
{
}

BOOL CIncomeTaxRecordDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
}



/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordDoc diagnostics

#ifdef _DEBUG
void CIncomeTaxRecordDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CIncomeTaxRecordDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordDoc commands

// IncomeTaxRecordSet.cpp : implementation of the CIncomeTaxRecordSet class
//

#include "stdafx.h"
#include "IncomeTaxRecord.h"
#include "IncomeTaxRecordSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
```

```
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

//////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordSet implementation

IMPLEMENT_DYNAMIC(CIncomeTaxRecordSet, CRecordset)

CIncomeTaxRecordSet::CIncomeTaxRecordSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CIncomeTaxRecordSet)
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_INCOME_TAX_RECORD_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_INCOME_FOR_PREV_YEAR = _T("");
	m_RETURN_ORIGINAL_REVISED = _T("");
	m_INCOME_FROM_SALARY = _T("");
	m_INCOME_FROM_HOUSE_PROPERTY = _T("");
	m_INC_FROM_BUS_OR_PROF = _T("");
	m_TOTAL_SHORT_TERM_GAIN = _T("");
	m_TOTAL_LONG_TERM_GAIN = _T("");
	m_TOTAL_CAPITAL_GAINS = _T("");
	m_INCOME_FROM_OTHER_SOURCES = _T("");
	m_INCM_OTHER_PERS_TO_ADDED = _T("");
	m_GROSS_TOTAL_INCOME = _T("");
	m_LESS_DEDUCTION_UNDER_VIA = _T("");
	m_TOTAL_INCOME = _T("");
	m_AGRICULTURAL_INCOME = _T("");
	m_INCOME_TO_BE_EXEMPTED = _T("");
	m_INCOME_AT_NORM_RATE = _T("");
	m_INCOME_TAX_NORM_RATE = _T("");
	m_INCOME_AT_SPCL_RATE = _T("");
	m_INCOME_TAX_SPCL_RATE = _T("");
	m_TAX_ON_TOTAL_INCOME = _T("");
	m_LESS_REBATE = _T("");
	m_TAX_PAYABLE = _T("");
	m_ADDITIONAL_SURCHARGE = _T("");
	m_TOTAL_TAX_PAYABLE = _T("");
	m_LESS_RELIEF = _T("");
	m_NET_TAX_PAYABLE = _T("");
	m_TAX_DEDUC_AT_SOURCE = _T("");
	m_ADVANCE_TAX_PAID = _T("");
```

```
	m_INTEREST_PAYABLE = _T("");
	m_SELF_ASSESS_BEFORE_31_MAY = _T("");
	m_SELF_ASSESS_AFT_31_MAY = _T("");
	m_TOT_SELF_ASSESS_TAX_PAID = _T("");
	m_BSR_CODE_OF_BANK_BRAN = _T("");
	m_DATE_OF_DEPOSIT = 0;
	m_SERIAL_NO_OF_CHALLAN = _T("");
	m_AMOUNT = _T("");
	m_NAME_OF_BANK_FOR_CHALLAN = _T("");
	m_BAL_TAX_PAYABLE_REFUND = _T("");
	m_BALANCE_TAX = _T("");
	m_ENCLOSURE1 = _T("");
	m_ENCLOSURE2 = _T("");
	m_ENCLOSURE3 = _T("");
	m_ENCLOSURE4 = _T("");
	m_ENCLOSURE5 = _T("");
	m_ENCLOSURE6 = _T("");
	m_nFields = 59;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}

CString CIncomeTaxRecordSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CIncomeTaxRecordSet::GetDefaultSQL()
{
	return _T("[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[INCOME_TAX_RECORD]");
}

void CIncomeTaxRecordSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CIncomeTaxRecordSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	RFX_Text(pFX, _T("[DIGAMBER].[INCOME_TAX_RECORD].[PAN]"),
m_INCOME_TAX_RECORD_PAN);
	RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
	RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
	RFX_Text(pFX, _T("[INCOME_FOR_PREV_YEAR]"), m_INCOME_FOR_PREV_YEAR);
```

```
	RFX_Text(pFX, _T("[RETURN_ORIGINAL_REVISED]"),
m_RETURN_ORIGINAL_REVISED);
	RFX_Text(pFX, _T("[INCOME_FROM_SALARY]"), m_INCOME_FROM_SALARY);
	RFX_Text(pFX, _T("[INCOME_FROM_HOUSE_PROPERTY]"),
m_INCOME_FROM_HOUSE_PROPERTY);
	RFX_Text(pFX, _T("[INC_FROM_BUS_OR_PROF]"), m_INC_FROM_BUS_OR_PROF);
	RFX_Text(pFX, _T("[TOTAL_SHORT_TERM_GAIN]"), m_TOTAL_SHORT_TERM_GAIN);
	RFX_Text(pFX, _T("[TOTAL_LONG_TERM_GAIN]"), m_TOTAL_LONG_TERM_GAIN);
	RFX_Text(pFX, _T("[TOTAL_CAPITAL_GAINS]"), m_TOTAL_CAPITAL_GAINS);
	RFX_Text(pFX, _T("[INCOME_FROM_OTHER_SOURCES]"),
m_INCOME_FROM_OTHER_SOURCES);
	RFX_Text(pFX, _T("[INCM_OTHER_PERS_TO_ADDED]"),
m_INCM_OTHER_PERS_TO_ADDED);
	RFX_Text(pFX, _T("[GROSS_TOTAL_INCOME]"), m_GROSS_TOTAL_INCOME);
	RFX_Text(pFX, _T("[LESS_DEDUCTION_UNDER_VIA]"),
m_LESS_DEDUCTION_UNDER_VIA);
	RFX_Text(pFX, _T("[TOTAL_INCOME]"), m_TOTAL_INCOME);
	RFX_Text(pFX, _T("[AGRICULTURAL_INCOME]"), m_AGRICULTURAL_INCOME);
	RFX_Text(pFX, _T("[INCOME_TO_BE_EXEMPTED]"), m_INCOME_TO_BE_EXEMPTED);
	RFX_Text(pFX, _T("[INCOME_AT_NORM_RATE]"), m_INCOME_AT_NORM_RATE);
	RFX_Text(pFX, _T("[INCOME_TAX_NORM_RATE]"), m_INCOME_TAX_NORM_RATE);
	RFX_Text(pFX, _T("[INCOME_AT_SPCL_RATE]"), m_INCOME_AT_SPCL_RATE);
	RFX_Text(pFX, _T("[INCOME_TAX_SPCL_RATE]"), m_INCOME_TAX_SPCL_RATE);
	RFX_Text(pFX, _T("[TAX_ON_TOTAL_INCOME]"), m_TAX_ON_TOTAL_INCOME);
	RFX_Text(pFX, _T("[LESS_REBATE]"), m_LESS_REBATE);
	RFX_Text(pFX, _T("[TAX_PAYABLE]"), m_TAX_PAYABLE);
	RFX_Text(pFX, _T("[ADDITIONAL_SURCHARGE]"), m_ADDITIONAL_SURCHARGE);
	RFX_Text(pFX, _T("[TOTAL_TAX_PAYABLE]"), m_TOTAL_TAX_PAYABLE);
	RFX_Text(pFX, _T("[LESS_RELIEF]"), m_LESS_RELIEF);
	RFX_Text(pFX, _T("[NET_TAX_PAYABLE]"), m_NET_TAX_PAYABLE);
	RFX_Text(pFX, _T("[TAX_DEDUC_AT_SOURCE]"), m_TAX_DEDUC_AT_SOURCE);
	RFX_Text(pFX, _T("[ADVANCE_TAX_PAID]"), m_ADVANCE_TAX_PAID);
	RFX_Text(pFX, _T("[INTEREST_PAYABLE]"), m_INTEREST_PAYABLE);
	RFX_Text(pFX, _T("[SELF_ASSESS_BEFORE_31_MAY]"),
m_SELF_ASSESS_BEFORE_31_MAY);
	RFX_Text(pFX, _T("[SELF_ASSESS_AFT_31_MAY]"), m_SELF_ASSESS_AFT_31_MAY);
	RFX_Text(pFX, _T("[TOT_SELF_ASSESS_TAX_PAID]"),
m_TOT_SELF_ASSESS_TAX_PAID);
	RFX_Text(pFX, _T("[BSR_CODE_OF_BANK_BRAN]"), m_BSR_CODE_OF_BANK_BRAN);
	RFX_Date(pFX, _T("[DATE_OF_DEPOSIT]"), m_DATE_OF_DEPOSIT);
	RFX_Text(pFX, _T("[SERIAL_NO_OF_CHALLAN]"), m_SERIAL_NO_OF_CHALLAN);
	RFX_Text(pFX, _T("[AMOUNT]"), m_AMOUNT);
	RFX_Text(pFX, _T("[NAME_OF_BANK_FOR_CHALLAN]"),
m_NAME_OF_BANK_FOR_CHALLAN);
	RFX_Text(pFX, _T("[BAL_TAX_PAYABLE_REFUND]"),
m_BAL_TAX_PAYABLE_REFUND);
	RFX_Text(pFX, _T("[BALANCE_TAX]"), m_BALANCE_TAX);
	RFX_Text(pFX, _T("[ENCLOSURE1]"), m_ENCLOSURE1);
	RFX_Text(pFX, _T("[ENCLOSURE2]"), m_ENCLOSURE2);
	RFX_Text(pFX, _T("[ENCLOSURE3]"), m_ENCLOSURE3);
	RFX_Text(pFX, _T("[ENCLOSURE4]"), m_ENCLOSURE4);
	RFX_Text(pFX, _T("[ENCLOSURE5]"), m_ENCLOSURE5);
	RFX_Text(pFX, _T("[ENCLOSURE6]"), m_ENCLOSURE6);
	//}}AFX_FIELD_MAP
}
```

```
/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordSet diagnostics

#ifdef _DEBUG
void CIncomeTaxRecordSet::AssertValid() const
{
	CRecordset::AssertValid();
}

void CIncomeTaxRecordSet::Dump(CDumpContext& dc) const
{
	CRecordset::Dump(dc);
}
#endif //_DEBUG

// IncomeTaxRecordView.cpp : implementation of the CIncomeTaxRecordView class
//

#include "stdafx.h"
#include "IncomeTaxRecord.h"

#include "IncomeTaxRecordSet.h"
#include "IncomeTaxRecordDoc.h"
#include "IncomeTaxRecordView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView

IMPLEMENT_DYNCREATE(CIncomeTaxRecordView, CRecordView)

BEGIN_MESSAGE_MAP(CIncomeTaxRecordView, CRecordView)
	//{{AFX_MSG_MAP(CIncomeTaxRecordView)
	ON_BN_CLICKED(IDC_SAVE, OnSave)
	ON_COMMAND(ID_RECORD_ADDRECORD, OnRecordAddrecord)
	ON_COMMAND(ID_RECORD_DELETERECORD, OnRecordDeleterecord)
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView construction/destruction

CIncomeTaxRecordView::CIncomeTaxRecordView()
	: CRecordView(CIncomeTaxRecordView::IDD)
{
```

```
	//{{AFX_DATA_INIT(CIncomeTaxRecordView)
	m_pSet = NULL;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CIncomeTaxRecordView::~CIncomeTaxRecordView()
{
}

void CIncomeTaxRecordView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CIncomeTaxRecordView)
	DDX_FieldText(pDX, IDC_AGR_INCOME, m_pSet->m_AGRICULTURAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_AGRICULTURAL_INCOME, 15);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR1, m_pSet->m_ASSES_YEAR_1);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR2, m_pSet->m_ASSES_YEAR_2);
	DDX_FieldText(pDX, IDC_PAN, m_pSet->m_INCOME_TAX_RECORD_PAN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_RECORD_PAN, 10);
	DDX_FieldText(pDX, IDC_ADD_SURCHARGE, m_pSet->m_ADDITIONAL_SURCHARGE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_ADDITIONAL_SURCHARGE, 15);
	DDX_FieldText(pDX, IDC_BALANCE_TAX, m_pSet->m_BALANCE_TAX, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BALANCE_TAX, 15);
	DDX_FieldText(pDX, IDC_CAP_GAIN_TOTAL, m_pSet->m_TOTAL_CAPITAL_GAINS,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_CAPITAL_GAINS, 15);
	DDX_FieldText(pDX, IDC_GROSS_TOT_INC, m_pSet->m_GROSS_TOTAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_GROSS_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_INC_EXMPT, m_pSet->m_INCOME_TO_BE_EXEMPTED,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TO_BE_EXEMPTED, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_BUS_PROF, m_pSet-
>m_INC_FROM_BUS_OR_PROF, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INC_FROM_BUS_OR_PROF, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_OTH_SRC, m_pSet-
>m_INCOME_FROM_OTHER_SOURCES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_OTHER_SOURCES, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_SAL, m_pSet->m_INCOME_FROM_SALARY,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_SALARY, 15);
	DDX_FieldText(pDX, IDC_INC_HOUSE_PROP, m_pSet-
>m_INCOME_FROM_HOUSE_PROPERTY, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_HOUSE_PROPERTY, 15);
	DDX_FieldText(pDX, IDC_INC_NOR_RATE, m_pSet->m_INCOME_AT_NORM_RATE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_AT_NORM_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_OTH_PERS, m_pSet-
>m_INCM_OTHER_PERS_TO_ADDED, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCM_OTHER_PERS_TO_ADDED, 15);
```

```
	DDX_FieldText(pDX, IDC_INC_SPCL_RATE, m_pSet->m_INCOME_AT_SPCL_RATE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_AT_SPCL_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_TAX_NOR_RATE, m_pSet-
>m_INCOME_TAX_NORM_RATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_NORM_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_TAX_SPCL_RATE, m_pSet-
>m_INCOME_TAX_SPCL_RATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_SPCL_RATE, 15);
	DDX_FieldText(pDX, IDC_LESS_ADV_TAX_PAID, m_pSet->m_ADVANCE_TAX_PAID,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_ADVANCE_TAX_PAID, 15);
	DDX_FieldText(pDX, IDC_LESS_DEDUCT_VI_A, m_pSet-
>m_LESS_DEDUCTION_UNDER_VIA, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_DEDUCTION_UNDER_VIA, 15);
	DDX_FieldText(pDX, IDC_LESS_REBATE, m_pSet->m_LESS_REBATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_REBATE, 15);
	DDX_FieldText(pDX, IDC_LESS_RELIEF, m_pSet->m_LESS_RELIEF, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_RELIEF, 15);
	DDX_FieldText(pDX, IDC_LESS_TAX_DED_SOURCE, m_pSet-
>m_TAX_DEDUC_AT_SOURCE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_DEDUC_AT_SOURCE, 15);
	DDX_FieldText(pDX, IDC_LESS_TOT_SELF_ASSESS_TAX_PAID, m_pSet-
>m_TOT_SELF_ASSESS_TAX_PAID, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOT_SELF_ASSESS_TAX_PAID, 15);
	DDX_FieldText(pDX, IDC_LONG_TERM_TOT, m_pSet->m_TOTAL_LONG_TERM_GAIN,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_LONG_TERM_GAIN, 15);
	DDX_FieldText(pDX, IDC_NET_TAX_PAYABLE, m_pSet->m_NET_TAX_PAYABLE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_NET_TAX_PAYABLE, 15);
	DDX_FieldText(pDX, IDC_PREV_YR_INC, m_pSet->m_INCOME_FOR_PREV_YEAR,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FOR_PREV_YEAR, 15);
	DDX_FieldText(pDX, IDC_SHORT_TERM_TOTAL, m_pSet-
>m_TOTAL_SHORT_TERM_GAIN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_SHORT_TERM_GAIN, 15);
	DDX_FieldText(pDX, IDC_TAX_ON_TOT_INC, m_pSet->m_TAX_ON_TOTAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_ON_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_TAX_PAYABLE, m_pSet->m_TAX_PAYABLE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_PAYABLE, 15);
	DDX_FieldText(pDX, IDC_TOT_INCOME, m_pSet->m_TOTAL_INCOME, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_TOT_TAX_PAYABLE, m_pSet->m_TOTAL_TAX_PAYABLE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_TAX_PAYABLE, 15);
	//}}AFX_DATA_MAP
}

BOOL CIncomeTaxRecordView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs
```

```
	return CRecordView::PreCreateWindow(cs);
}

void CIncomeTaxRecordView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_incomeTaxRecordSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView printing

BOOL CIncomeTaxRecordView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
}

void CIncomeTaxRecordView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
}

void CIncomeTaxRecordView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView diagnostics

#ifdef _DEBUG
void CIncomeTaxRecordView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CIncomeTaxRecordView::Dump(CDumpContext& dc) const
{
	CRecordView::Dump(dc);
}

CIncomeTaxRecordDoc* CIncomeTaxRecordView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CIncomeTaxRecordDoc)));
	return (CIncomeTaxRecordDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView database support
CRecordset* CIncomeTaxRecordView::OnGetRecordset()
```

```
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordView message handlers

void CIncomeTaxRecordView::OnSave()
{
	// TODO: Add your control notification handler code here
	UpdateData(true);

	if(m_pSet->CanUpdate()) {
		m_pSet->Update();
	}

	m_pSet->Requery();

	UpdateData(false);

}

void CIncomeTaxRecordView::OnRecordAddrecord()
{
	// TODO: Add your command handler code here
		m_pSet->AddNew();

	UpdateData(FALSE);

}

void CIncomeTaxRecordView::OnRecordDeleterecord()
{
	// TODO: Add your command handler code here
	m_pSet->Delete();
	m_pSet->Requery();
	m_pSet->MoveNext();
	if(m_pSet->IsEOF())
		m_pSet->MoveLast();

	if(m_pSet->IsBOF())
		m_pSet->SetFieldNull(NULL);

	UpdateData(FALSE);
}

// TotalSalary.cpp : implementation file
//

#include "stdafx.h"
#include "IncomeTaxRecord.h"
#include "TotalSalary.h"

#ifdef _DEBUG
```

```
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// TotalSalary dialog


TotalSalary::TotalSalary(CWnd* pParent /*=NULL*/)
	: CDialog(TotalSalary::IDD, pParent)
{
	//{{AFX_DATA_INIT(TotalSalary)
	m_dDecember = 0.0;
	m_dApril = 0.0;
	m_dAugust = 0.0;
	m_dFebruary = 0.0;
	m_dJanuary = 0.0;
	m_dJuly = 0.0;
	m_dJune = 0.0;
	m_dMarch = 0.0;
	m_dMay = 0.0;
	m_dNovember = 0.0;
	m_dOctober = 0.0;
	m_dSeptember = 0.0;
	m_dTotalSalary = 0.0;
	//}}AFX_DATA_INIT
}


void TotalSalary::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(TotalSalary)
	DDX_Text(pDX, IDC_DEC, m_dDecember);
	DDV_MinMaxDouble(pDX, m_dDecember, 0., 9999999999999.);
	DDX_Text(pDX, IDC_APRIL, m_dApril);
	DDV_MinMaxDouble(pDX, m_dApril, 0., 9999999999999.);
	DDX_Text(pDX, IDC_AUG, m_dAugust);
	DDV_MinMaxDouble(pDX, m_dAugust, 0., 9999999999999.);
	DDX_Text(pDX, IDC_FEB, m_dFebruary);
	DDV_MinMaxDouble(pDX, m_dFebruary, 0., 9999999999999.);
	DDX_Text(pDX, IDC_JAN, m_dJanuary);
	DDV_MinMaxDouble(pDX, m_dJanuary, 0., 9999999999999.);
	DDX_Text(pDX, IDC_JULY, m_dJuly);
	DDV_MinMaxDouble(pDX, m_dJuly, 0., 9999999999999.);
	DDX_Text(pDX, IDC_JUNE, m_dJune);
	DDV_MinMaxDouble(pDX, m_dJune, 0., 9999999999999.);
	DDX_Text(pDX, IDC_MARCH, m_dMarch);
	DDV_MinMaxDouble(pDX, m_dMarch, 0., 9999999999999.);
	DDX_Text(pDX, IDC_MAY, m_dMay);
	DDV_MinMaxDouble(pDX, m_dMay, 0., 9999999999999.);
	DDX_Text(pDX, IDC_NOV, m_dNovember);
	DDV_MinMaxDouble(pDX, m_dNovember, 0., 9999999999999.);
	DDX_Text(pDX, IDC_OCT, m_dOctober);
```

```
        DDV_MinMaxDouble(pDX, m_dOctober, 0., 9999999999999.);
        DDX_Text(pDX, IDC_SEP, m_dSeptember);
        DDV_MinMaxDouble(pDX, m_dSeptember, 0., 9999999999999.);
        DDX_Text(pDX, IDC_TOTAL_SALARY, m_dTotalSalary);
        //}}AFX_DATA_MAP
}


BEGIN_MESSAGE_MAP(TotalSalary, CDialog)
        //{{AFX_MSG_MAP(TotalSalary)
        ON_BN_CLICKED(IDCALCULATETOTAL, OnCalculatetotal)
        ON_BN_CLICKED(IDC_CLOSE, OnClose)
        //}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// TotalSalary message handlers

void TotalSalary::OnCalculatetotal()
{
        // TODO: Add your control notification handler code here
        m_dTotalSalary   = m_dJanuary + m_dFebruary + m_dMarch
                                                + m_dApril + m_dMay + m_dJune
                                                + m_dJuly + m_dAugust + m_dSeptember
                                                + m_dOctober + m_dNovember + m_dDecember;
        UpdateData(FALSE);

}

void TotalSalary::OnClose()
{
        // TODO: Add your control notification handler code here
//      close();

}

// IncomeTaxRecord.h : main header file for the INCOMETAXRECORD application
//

#if
!defined(AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLU
DED_)
#define
AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
        #error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"       // main symbols
```

```
/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp:
// See IncomeTaxRecord.cpp for the implementation of this class
//

class CIncomeTaxRecordApp : public CWinApp
{
public:
	CIncomeTaxRecordApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CIncomeTaxRecordApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CIncomeTaxRecordApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLU
DED_)

// IncomeTaxRecord.h : main header file for the INCOMETAXRECORD application
//

#if
!defined(AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLU
DED_)
#define
AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
	#error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"       // main symbols
```

```
/////////////////////////////////////////////////////////////////////////////
// CIncomeTaxRecordApp:
// See IncomeTaxRecord.cpp for the implementation of this class
//

class CIncomeTaxRecordApp : public CWinApp
{
public:
	CIncomeTaxRecordApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CIncomeTaxRecordApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CIncomeTaxRecordApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_INCOMETAXRECORD_H__451A608F_C2BB_4A11_A8D3_AAB22178771B__INCLU
DED_)

// IncomeTaxRecordSet.h : interface of the CIncomeTaxRecordSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_INCOMETAXRECORDSET_H__A228429C_C1BD_432C_B864_7634A7D7B2E6__INC
LUDED_)
#define
AFX_INCOMETAXRECORDSET_H__A228429C_C1BD_432C_B864_7634A7D7B2E6__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CIncomeTaxRecordSet : public CRecordset
{
public:
	CIncomeTaxRecordSet(CDatabase* pDatabase = NULL);
	DECLARE_DYNAMIC(CIncomeTaxRecordSet)
```

```
// Field/Param Data
	//{{AFX_FIELD(CIncomeTaxRecordSet, CRecordset)
	CString  m_CLIENT_RECORD_PAN;
	CString  m_CLIENT_NAME;
	CString  m_FATHERS_NAME;
	CTime   m_DOB;
	CString  m_ADDRESS;
	CString  m_TELEPHONE;
	CString  m_SEX;
	CString  m_INDV_HUF_FIRM_AOP_LA;
	CString  m_RESIDENT_NR_NOR;
	CString  m_WARD_CIRCLE_SPECIAL_RANGE;
	CString  m_PINCODE;
	CString  m_INCOME_TAX_RECORD_PAN;
	CTime   m_ASSES_YEAR_1;
	CTime   m_ASSES_YEAR_2;
	CString  m_INCOME_FOR_PREV_YEAR;
	CString  m_RETURN_ORIGINAL_REVISED;
	CString  m_INCOME_FROM_SALARY;
	CString  m_INCOME_FROM_HOUSE_PROPERTY;
	CString  m_INC_FROM_BUS_OR_PROF;
	CString  m_TOTAL_SHORT_TERM_GAIN;
	CString  m_TOTAL_LONG_TERM_GAIN;
	CString  m_TOTAL_CAPITAL_GAINS;
	CString  m_INCOME_FROM_OTHER_SOURCES;
	CString  m_INCM_OTHER_PERS_TO_ADDED;
	CString  m_GROSS_TOTAL_INCOME;
	CString  m_LESS_DEDUCTION_UNDER_VIA;
	CString  m_TOTAL_INCOME;
	CString  m_AGRICULTURAL_INCOME;
	CString  m_INCOME_TO_BE_EXEMPTED;
	CString  m_INCOME_AT_NORM_RATE;
	CString  m_INCOME_TAX_NORM_RATE;
	CString  m_INCOME_AT_SPCL_RATE;
	CString  m_INCOME_TAX_SPCL_RATE;
	CString  m_TAX_ON_TOTAL_INCOME;
	CString  m_LESS_REBATE;
	CString  m_TAX_PAYABLE;
	CString  m_ADDITIONAL_SURCHARGE;
	CString  m_TOTAL_TAX_PAYABLE;
	CString  m_LESS_RELIEF;
	CString  m_NET_TAX_PAYABLE;
	CString  m_TAX_DEDUC_AT_SOURCE;
	CString  m_ADVANCE_TAX_PAID;
	CString  m_INTEREST_PAYABLE;
	CString  m_SELF_ASSESS_BEFORE_31_MAY;
	CString  m_SELF_ASSESS_AFT_31_MAY;
	CString  m_TOT_SELF_ASSESS_TAX_PAID;
	CString  m_BSR_CODE_OF_BANK_BRAN;
	CTime   m_DATE_OF_DEPOSIT;
	CString  m_SERIAL_NO_OF_CHALLAN;
	CString  m_AMOUNT;
	CString  m_NAME_OF_BANK_FOR_CHALLAN;
	CString  m_BAL_TAX_PAYABLE_REFUND;
```

```
	CString	m_BALANCE_TAX;
	CString	m_ENCLOSURE1;
	CString	m_ENCLOSURE2;
	CString	m_ENCLOSURE3;
	CString	m_ENCLOSURE4;
	CString	m_ENCLOSURE5;
	CString	m_ENCLOSURE6;
	//}}AFX_FIELD

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CIncomeTaxRecordSet)
	public:
	virtual CString GetDefaultConnect();	// Default connection string
	virtual CString GetDefaultSQL();	// default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);	// RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_INCOMETAXRECORDSET_H__A228429C_C1BD_432C_B864_7634A7D7B2E6__INC
LUDED_)
// IncomeTaxRecordView.h : interface of the CIncomeTaxRecordView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_INCOMETAXRECORDVIEW_H__65CCAB6E_AED6_4058_A1FC_0B9A3BE57344__I
NCLUDED_)
#define
AFX_INCOMETAXRECORDVIEW_H__65CCAB6E_AED6_4058_A1FC_0B9A3BE57344__INCLUD
ED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CIncomeTaxRecordSet;

class CIncomeTaxRecordView : public CRecordView
{
protected: // create from serialization only
	CIncomeTaxRecordView();
	DECLARE_DYNCREATE(CIncomeTaxRecordView)
```

```
public:
	//{{AFX_DATA(CIncomeTaxRecordView)
	enum { IDD = IDD_INCOMETAXRECORD_FORM };
	CIncomeTaxRecordSet* m_pSet;
	//}}AFX_DATA

// Attributes
public:
	CIncomeTaxRecordDoc* GetDocument();

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CIncomeTaxRecordView)
	public:
	virtual CRecordset* OnGetRecordset();
	virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	virtual void OnInitialUpdate(); // called first time after construct
	virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
	virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
	virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
	//}}AFX_VIRTUAL

// Implementation
public:
	virtual ~CIncomeTaxRecordView();
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
	//{{AFX_MSG(CIncomeTaxRecordView)
	afx_msg void OnSave();
	afx_msg void OnRecordAddrecord();
	afx_msg void OnRecordDeleterecord();
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in IncomeTaxRecordView.cpp
inline CIncomeTaxRecordDoc* CIncomeTaxRecordView::GetDocument()
   { return (CIncomeTaxRecordDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
```

```
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_INCOMETAXRECORDVIEW_H__65CCAB6E_AED6_4058_A1FC_0B9A3BE57344__I
NCLUDED_)

#if
!defined(AFX_TOTALSALARY_H__7BB69527_D486_4E3C_A7D5_B9B5755E7685__INCLUDED_)
#define AFX_TOTALSALARY_H__7BB69527_D486_4E3C_A7D5_B9B5755E7685__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
// TotalSalary.h : header file
//

/////////////////////////////////////////////////////////////////////////////
// TotalSalary dialog

class TotalSalary : public CDialog
{
// Construction
public:
	TotalSalary(CWnd* pParent = NULL);   // standard constructor

// Dialog Data
	//{{AFX_DATA(TotalSalary)
	enum { IDD = IDD_TOTAL_SALARY };
	double	m_dDecember;
	double	m_dApril;
	double	m_dAugust;
	double	m_dFebruary;
	double	m_dJanuary;
	double	m_dJuly;
	double	m_dJune;
	double	m_dMarch;
	double	m_dMay;
	double	m_dNovember;
	double	m_dOctober;
	double	m_dSeptember;
	double	m_dTotalSalary;
	//}}AFX_DATA


// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(TotalSalary)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:

	// Generated message map functions
```

```
	//{{AFX_MSG(TotalSalary)
	afx_msg void OnCalculatetotal();
	afx_msg void OnClose();
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_TOTALSALARY_H__7BB69527_D486_4E3C_A7D5_B9B5755E7685__INCLUDED_)
```

Code of Trading Account Form:

```
// TradingAccount.cpp : Defines the class behaviors for the application.
//

#include "stdafx.h"
#include "TradingAccount.h"

#include "MainFrm.h"
#include "TradingAccountSet.h"
#include "TradingAccountDoc.h"
#include "TradingAccountView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountApp

BEGIN_MESSAGE_MAP(CTradingAccountApp, CWinApp)
	//{{AFX_MSG_MAP(CTradingAccountApp)
	ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard print setup command
	ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountApp construction

CTradingAccountApp::CTradingAccountApp()
{
	// TODO: add construction code here,
```

```
	// Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CTradingAccountApp object

CTradingAccountApp theApp;

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountApp initialization

BOOL CTradingAccountApp::InitInstance()
{
	AfxEnableControlContainer();

	// Standard initialization
	// If you are not using these features and wish to reduce the size
	//  of your final executable, you should remove from the following
	//  the specific initialization routines you do not need.

#ifdef _AFXDLL
	Enable3dControls();			// Call this when using MFC in a shared DLL
#else
	Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif

	// Change the registry key under which our settings are stored.
	// TODO: You should modify this string to be something appropriate
	// such as the name of your company or organization.
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));

	LoadStdProfileSettings();  // Load standard INI file options (including MRU)

	// Register the application's document templates.  Document templates
	//  serve as the connection between documents, frame windows and views.

	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CTradingAccountDoc),
		RUNTIME_CLASS(CMainFrame),       // main SDI frame window
		RUNTIME_CLASS(CTradingAccountView));
	AddDocTemplate(pDocTemplate);

	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);

	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;

	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();
```

```
	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
	CAboutDlg();

// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA

	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
	//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CTradingAccountApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
```

```
	aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountApp message handlers

// TradingAccountDoc.cpp : implementation of the CTradingAccountDoc class
//

#include "stdafx.h"
#include "TradingAccount.h"

#include "TradingAccountSet.h"
#include "TradingAccountDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountDoc

IMPLEMENT_DYNCREATE(CTradingAccountDoc, CDocument)

BEGIN_MESSAGE_MAP(CTradingAccountDoc, CDocument)
	//{{AFX_MSG_MAP(CTradingAccountDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountDoc construction/destruction

CTradingAccountDoc::CTradingAccountDoc()
{
	// TODO: add one-time construction code here

}

CTradingAccountDoc::~CTradingAccountDoc()
{
}

BOOL CTradingAccountDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
```

```
}



/////////////////////////////////////////////////////////////////////////////
// CTradingAccountDoc diagnostics

#ifdef _DEBUG
void CTradingAccountDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CTradingAccountDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountDoc commands

// TradingAccountSet.cpp : implementation of the CTradingAccountSet class
//

#include "stdafx.h"
#include "TradingAccount.h"
#include "TradingAccountSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountSet implementation

IMPLEMENT_DYNAMIC(CTradingAccountSet, CRecordset)

CTradingAccountSet::CTradingAccountSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CTradingAccountSet)
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
```

```
	m_TRADING_ACCOUNT_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_TO_OPENING_STOCK = _T("");
	m_TO_STOCK = _T("");
	m_TO_PURCHASES = _T("");
	m_TO_CARRIAGE = _T("");
	m_TO_OCTROI = _T("");
	m_TO_IMPORT_DUTY_CUSTOMS = _T("");
	m_TO_WAGES = _T("");
	m_TO_COAL_WATER_GAS = _T("");
	m_TO_HEATING_LIGHTING_POWER = _T("");
	m_TO_MANU_ASSEM_EXPEN = _T("");
	m_TO_CONSUMABLE_STORES = _T("");
	m_TO_DRCT_FACT_PROD_EXP = _T("");
	m_TO_ROYALTY = _T("");
	m_TO_GROSS_PROFIT = _T("");
	m_BY_SALES = _T("");
	m_BY_CLOSING_STOCK = _T("");
	m_BY_STOCK = _T("");
	m_BY_GROSS_LOSS = _T("");
	m_nFields = 32;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}

CString CTradingAccountSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CTradingAccountSet::GetDefaultSQL()
{
	return _T("[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[TRADING_ACCOUNT]");
}

void CTradingAccountSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CTradingAccountSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	RFX_Text(pFX, _T("[DIGAMBER].[TRADING_ACCOUNT].[PAN]"),
m_TRADING_ACCOUNT_PAN);
```

```
	RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
	RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
	RFX_Text(pFX, _T("[TO_OPENING_STOCK]"), m_TO_OPENING_STOCK);
	RFX_Text(pFX, _T("[TO_STOCK]"), m_TO_STOCK);
	RFX_Text(pFX, _T("[TO_PURCHASES]"), m_TO_PURCHASES);
	RFX_Text(pFX, _T("[TO_CARRIAGE]"), m_TO_CARRIAGE);
	RFX_Text(pFX, _T("[TO_OCTROI]"), m_TO_OCTROI);
	RFX_Text(pFX, _T("[TO_IMPORT_DUTY_CUSTOMS]"),
m_TO_IMPORT_DUTY_CUSTOMS);
	RFX_Text(pFX, _T("[TO_WAGES]"), m_TO_WAGES);
	RFX_Text(pFX, _T("[TO_COAL_WATER_GAS]"), m_TO_COAL_WATER_GAS);
	RFX_Text(pFX, _T("[TO_HEATING_LIGHTING_POWER]"),
m_TO_HEATING_LIGHTING_POWER);
	RFX_Text(pFX, _T("[TO_MANU_ASSEM_EXPEN]"), m_TO_MANU_ASSEM_EXPEN);
	RFX_Text(pFX, _T("[TO_CONSUMABLE_STORES]"), m_TO_CONSUMABLE_STORES);
	RFX_Text(pFX, _T("[TO_DRCT_FACT_PROD_EXP]"), m_TO_DRCT_FACT_PROD_EXP);
	RFX_Text(pFX, _T("[TO_ROYALTY]"), m_TO_ROYALTY);
	RFX_Text(pFX, _T("[TO_GROSS_PROFIT]"), m_TO_GROSS_PROFIT);
	RFX_Text(pFX, _T("[BY_SALES]"), m_BY_SALES);
	RFX_Text(pFX, _T("[BY_CLOSING_STOCK]"), m_BY_CLOSING_STOCK);
	RFX_Text(pFX, _T("[BY_STOCK]"), m_BY_STOCK);
	RFX_Text(pFX, _T("[BY_GROSS_LOSS]"), m_BY_GROSS_LOSS);
	//}}AFX_FIELD_MAP
}

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountSet diagnostics

#ifdef _DEBUG
void CTradingAccountSet::AssertValid() const
{
	CRecordset::AssertValid();
}

void CTradingAccountSet::Dump(CDumpContext& dc) const
{
	CRecordset::Dump(dc);
}
#endif // _DEBUG
// TradingAccountView.cpp : implementation of the CTradingAccountView class
//

#include "stdafx.h"
#include "TradingAccount.h"

#include "TradingAccountSet.h"
#include "TradingAccountDoc.h"
#include "TradingAccountView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
```

```
/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView

IMPLEMENT_DYNCREATE(CTradingAccountView, CRecordView)

BEGIN_MESSAGE_MAP(CTradingAccountView, CRecordView)
	//{{AFX_MSG_MAP(CTradingAccountView)
	ON_COMMAND(ID_RECORD_ADDRECORD, OnRecordAddrecord)
	ON_COMMAND(ID_RECORD_DELETERRECORD, OnRecordDeleterrecord)
	ON_BN_CLICKED(IDC_SAVE, OnSave)
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView construction/destruction

CTradingAccountView::CTradingAccountView()
	: CRecordView(CTradingAccountView::IDD)
{
	//{{AFX_DATA_INIT(CTradingAccountView)
	m_pSet = NULL;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CTradingAccountView::~CTradingAccountView()
{
}

void CTradingAccountView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CTradingAccountView)
	DDX_FieldText(pDX, IDC_ASSEMBLY, m_pSet->m_TO_MANU_ASSEM_EXPEN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_MANU_ASSEM_EXPEN, 15);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR1, m_pSet->m_ASSES_YEAR_1);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR2, m_pSet->m_ASSES_YEAR_2);
	DDX_FieldText(pDX, IDC_BY_STOCK, m_pSet->m_BY_STOCK, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BY_STOCK, 15);
	DDX_FieldText(pDX, IDC_CARRIAGE, m_pSet->m_TO_CARRIAGE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_CARRIAGE, 15);
	DDX_FieldText(pDX, IDC_CLOSING_STOCK, m_pSet->m_BY_CLOSING_STOCK, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BY_CLOSING_STOCK, 15);
	DDX_FieldText(pDX, IDC_COAL_WATER, m_pSet->m_TO_COAL_WATER_GAS, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_COAL_WATER_GAS, 15);
	DDX_FieldText(pDX, IDC_CONSUMABLE, m_pSet->m_TO_CONSUMABLE_STORES,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_CONSUMABLE_STORES, 15);
	DDX_FieldText(pDX, IDC_CUST_IMP_DUTY, m_pSet-
>m_TO_IMPORT_DUTY_CUSTOMS, m_pSet);
```

```
	DDV_MaxChars(pDX, m_pSet->m_TO_IMPORT_DUTY_CUSTOMS, 15);
	DDX_FieldText(pDX, IDC_FACTORY_PRODUCTIVE, m_pSet-
>m_TO_DRCT_FACT_PROD_EXP, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_DRCT_FACT_PROD_EXP, 15);
	DDX_FieldText(pDX, IDC_GROSS_LOSS, m_pSet->m_BY_GROSS_LOSS, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BY_GROSS_LOSS, 15);
	DDX_FieldText(pDX, IDC_GROSS_PROFIT, m_pSet->m_TO_GROSS_PROFIT, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_GROSS_PROFIT, 15);
	DDX_FieldText(pDX, IDC_HEATING, m_pSet->m_TO_HEATING_LIGHTING_POWER,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_HEATING_LIGHTING_POWER, 15);
	DDX_FieldText(pDX, IDC_OCTROI, m_pSet->m_TO_OCTROI, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_OCTROI, 15);
	DDX_FieldText(pDX, IDC_OPENING_STOCK, m_pSet->m_TO_OPENING_STOCK,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_OPENING_STOCK, 15);
	DDX_FieldText(pDX, IDC_PAN, m_pSet->m_TRADING_ACCOUNT_PAN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TRADING_ACCOUNT_PAN, 15);
	DDX_FieldText(pDX, IDC_PURCHASE, m_pSet->m_TO_PURCHASES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_PURCHASES, 15);
	DDX_FieldText(pDX, IDC_ROYALTY, m_pSet->m_TO_ROYALTY, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_ROYALTY, 15);
	DDX_FieldText(pDX, IDC_SALES, m_pSet->m_BY_SALES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BY_SALES, 15);
	DDX_FieldText(pDX, IDC_STOCK, m_pSet->m_TO_STOCK, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_STOCK, 15);
	DDX_FieldText(pDX, IDC_WAGES, m_pSet->m_TO_WAGES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_WAGES, 15);
	//}}AFX_DATA_MAP
}

BOOL CTradingAccountView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs

	return CRecordView::PreCreateWindow(cs);
}

void CTradingAccountView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_tradingAccountSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView printing

BOOL CTradingAccountView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
```

```
}

void CTradingAccountView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
}

void CTradingAccountView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView diagnostics

#ifdef _DEBUG
void CTradingAccountView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CTradingAccountView::Dump(CDumpContext& dc) const
{
	CRecordView::Dump(dc);
}

CTradingAccountDoc* CTradingAccountView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CTradingAccountDoc)));
	return (CTradingAccountDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView database support
CRecordset* CTradingAccountView::OnGetRecordset()
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CTradingAccountView message handlers

void CTradingAccountView::OnRecordAddrecord()
{
	// TODO: Add your command handler code here
	m_pSet->AddNew();

	UpdateData(FALSE);

}

void CTradingAccountView::OnRecordDeleterrecord()
{
```

```
	// TODO: Add your command handler code here
	m_pSet->Delete();
	m_pSet->Requery();
	m_pSet->MoveNext();
	if(m_pSet->IsEOF())
		m_pSet->MoveLast();

	if(m_pSet->IsBOF())
		m_pSet->SetFieldNull(NULL);

	UpdateData(FALSE);

}

void CTradingAccountView::OnSave()
{
	// TODO: Add your control notification handler code here
	UpdateData(true);

	if(m_pSet->CanUpdate()) {
		m_pSet->Update();
	}

	m_pSet->Requery();

	UpdateData(false);

}
// TradingAccount.h : main header file for the TRADINGACCOUNT application
//

#if
!defined(AFX_TRADINGACCOUNT_H__BB646FA6_606B_48F5_A655_3B5F61ADC832__INCLUDE
D_)
#define
AFX_TRADINGACCOUNT_H__BB646FA6_606B_48F5_A655_3B5F61ADC832__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
	#error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"       // main symbols

/////////////////////////////////////////////////////////////////////////////
// CTradingAccountApp:
// See TradingAccount.cpp for the implementation of this class
//

class CTradingAccountApp : public CWinApp
{
public:
```

```
	CTradingAccountApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CTradingAccountApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CTradingAccountApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_TRADINGACCOUNT_H__BB646FA6_606B_48F5_A655_3B5F61ADC832__INCLUDE
D_)

// TradingAccountDoc.h : interface of the CTradingAccountDoc class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_TRADINGACCOUNTDOC_H__81BBF620_64DA_4A40_AE73_8EC8CAC030DC__INC
LUDED_)
#define
AFX_TRADINGACCOUNTDOC_H__81BBF620_64DA_4A40_AE73_8EC8CAC030DC__INCLUDED
_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "TradingAccountSet.h"


class CTradingAccountDoc : public CDocument
{
protected: // create from serialization only
	CTradingAccountDoc();
	DECLARE_DYNCREATE(CTradingAccountDoc)

// Attributes
public:
	CTradingAccountSet m_tradingAccountSet;
```

```
// Operations
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CTradingAccountDoc)
        public:
        virtual BOOL OnNewDocument();
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CTradingAccountDoc();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CTradingAccountDoc)
                // NOTE - the ClassWizard will add and remove member functions here.
                //    DO NOT EDIT what you see in these blocks of generated code !
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_TRADINGACCOUNTDOC_H__81BBF620_64DA_4A40_AE73_8EC8CAC030DC__INC
LUDED_)
// TradingAccountSet.h : interface of the CTradingAccountSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_TRADINGACCOUNTSET_H__0E9AD8CA_E99B_4C82_BACC_17CAA861C419__INC
LUDED_)
#define
AFX_TRADINGACCOUNTSET_H__0E9AD8CA_E99B_4C82_BACC_17CAA861C419__INCLUDED
_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CTradingAccountSet : public CRecordset
{
public:
```

```
	CTradingAccountSet(CDatabase* pDatabase = NULL);
	DECLARE_DYNAMIC(CTradingAccountSet)

// Field/Param Data
	//{{AFX_FIELD(CTradingAccountSet, CRecordset)
	CString  m_CLIENT_RECORD_PAN;
	CString  m_CLIENT_NAME;
	CString  m_FATHERS_NAME;
	CTime   m_DOB;
	CString  m_ADDRESS;
	CString  m_TELEPHONE;
	CString  m_SEX;
	CString  m_INDV_HUF_FIRM_AOP_LA;
	CString  m_RESIDENT_NR_NOR;
	CString  m_WARD_CIRCLE_SPECIAL_RANGE;
	CString  m_PINCODE;
	CString  m_TRADING_ACCOUNT_PAN;
	CTime   m_ASSES_YEAR_1;
	CTime   m_ASSES_YEAR_2;
	CString  m_TO_OPENING_STOCK;
	CString  m_TO_STOCK;
	CString  m_TO_PURCHASES;
	CString  m_TO_CARRIAGE;
	CString  m_TO_OCTROI;
	CString  m_TO_IMPORT_DUTY_CUSTOMS;
	CString  m_TO_WAGES;
	CString  m_TO_COAL_WATER_GAS;
	CString  m_TO_HEATING_LIGHTING_POWER;
	CString  m_TO_MANU_ASSEM_EXPEN;
	CString  m_TO_CONSUMABLE_STORES;
	CString  m_TO_DRCT_FACT_PROD_EXP;
	CString  m_TO_ROYALTY;
	CString  m_TO_GROSS_PROFIT;
	CString  m_BY_SALES;
	CString  m_BY_CLOSING_STOCK;
	CString  m_BY_STOCK;
	CString  m_BY_GROSS_LOSS;
	//}}AFX_FIELD

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CTradingAccountSet)
	public:
	virtual CString GetDefaultConnect();		// Default connection string
	virtual CString GetDefaultSQL();	// default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);		// RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

};
```

```
//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_TRADINGACCOUNTSET_H__0E9AD8CA_E99B_4C82_BACC_17CAA861C419__INC
LUDED_)

// TradingAccountView.h : interface of the CTradingAccountView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_TRADINGACCOUNTVIEW_H__175CF536_80AC_446B_93D2_BA228631CAD1__INC
LUDED_)
#define
AFX_TRADINGACCOUNTVIEW_H__175CF536_80AC_446B_93D2_BA228631CAD1__INCLUDED
_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CTradingAccountSet;

class CTradingAccountView : public CRecordView
{
protected: // create from serialization only
	CTradingAccountView();
	DECLARE_DYNCREATE(CTradingAccountView)

public:
	//{{AFX_DATA(CTradingAccountView)
	enum { IDD = IDD_TRADINGACCOUNT_FORM };
	CTradingAccountSet* m_pSet;
	//}}AFX_DATA

// Attributes
public:
	CTradingAccountDoc* GetDocument();

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CTradingAccountView)
	public:
	virtual CRecordset* OnGetRecordset();
	virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	virtual void OnInitialUpdate(); // called first time after construct
	virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
	virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
```

```
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CTradingAccountView();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CTradingAccountView)
        afx_msg void OnRecordAddrecord();
        afx_msg void OnRecordDeleterrecord();
        afx_msg void OnSave();
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in TradingAccountView.cpp
inline CTradingAccountDoc* CTradingAccountView::GetDocument()
   { return (CTradingAccountDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_TRADINGACCOUNTVIEW_H__175CF536_80AC_446B_93D2_BA228631CAD1__INC
LUDED_)
```

Code of P/L Account Form:

```
// PLAccount.cpp : Defines the class behaviors for the application.
//

#include "stdafx.h"
#include "PLAccount.h"

#include "MainFrm.h"
#include "PLAccountSet.h"
#include "PLAccountDoc.h"
#include "PLAccountView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
```

```
#endif

/////////////////////////////////////////////////////////////////////////////
// CPLAccountApp

BEGIN_MESSAGE_MAP(CPLAccountApp, CWinApp)
	//{{AFX_MSG_MAP(CPLAccountApp)
	ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard print setup command
	ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CPLAccountApp construction

CPLAccountApp::CPLAccountApp()
{
	// TODO: add construction code here,
	// Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CPLAccountApp object

CPLAccountApp theApp;

/////////////////////////////////////////////////////////////////////////////
// CPLAccountApp initialization

BOOL CPLAccountApp::InitInstance()
{
	AfxEnableControlContainer();

	// Standard initialization
	// If you are not using these features and wish to reduce the size
	//  of your final executable, you should remove from the following
	//  the specific initialization routines you do not need.

#ifdef _AFXDLL
	Enable3dControls();			// Call this when using MFC in a shared DLL
#else
	Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif

	// Change the registry key under which our settings are stored.
	// TODO: You should modify this string to be something appropriate
	// such as the name of your company or organization.
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));

	LoadStdProfileSettings();  // Load standard INI file options (including MRU)

	// Register the application's document templates.  Document templates
```

```
	//  serve as the connection between documents, frame windows and views.

	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CPLAccountDoc),
		RUNTIME_CLASS(CMainFrame),	// main SDI frame window
		RUNTIME_CLASS(CPLAccountView));
	AddDocTemplate(pDocTemplate);

	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);

	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;

	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();

	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
	CAboutDlg();

// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA

	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
```

```
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
	//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CPLAccountApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
	aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CPLAccountApp message handlers

// PLAccountDoc.cpp : implementation of the CPLAccountDoc class
//

#include "stdafx.h"
#include "PLAccount.h"

#include "PLAccountSet.h"
#include "PLAccountDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CPLAccountDoc

IMPLEMENT_DYNCREATE(CPLAccountDoc, CDocument)

BEGIN_MESSAGE_MAP(CPLAccountDoc, CDocument)
	//{{AFX_MSG_MAP(CPLAccountDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
```

```
// CPLAccountDoc construction/destruction

CPLAccountDoc::CPLAccountDoc()
{
	// TODO: add one-time construction code here

}

CPLAccountDoc::~CPLAccountDoc()
{
}

BOOL CPLAccountDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
}



/////////////////////////////////////////////////////////////////////////////
// CPLAccountDoc diagnostics

#ifdef _DEBUG
void CPLAccountDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CPLAccountDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CPLAccountDoc commands

// PLAccountSet.cpp : implementation of the CPLAccountSet class
//

#include "stdafx.h"
#include "PLAccount.h"
#include "PLAccountSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
```

```
/////////////////////////////////////////////////////////////////////////////
// CPLAccountSet implementation

IMPLEMENT_DYNAMIC(CPLAccountSet, CRecordset)

CPLAccountSet::CPLAccountSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CPLAccountSet)
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_PL_ACCOUNT_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_TO_GROSS_LOSS = _T("");
	m_TO_SALARIES_WAGES = _T("");
	m_TO_OFFICE_GODOWN_RENT = _T("");
	m_TO_OFFICE_EXPENSES = _T("");
	m_TO_MISC_SUNDRY_EXPENSE = _T("");
	m_TO_INSURANCE = _T("");
	m_TO_STATION_PRINT = _T("");
	m_TO_STAFF_WELF_EXPENSE = _T("");
	m_TO_LIGHT_WATER_ELECT = _T("");
	m_TO_ESTAB_EXPENSE = _T("");
	m_TO_POST_TLGRM_FAX_COUR_PH = _T("");
	m_TO_LAW_CHARGES = _T("");
	m_TO_REPAIRS = _T("");
	m_TO_DISTRIBUTION_EXPENSES = _T("");
	m_TO_TRAVEL_EXPENSE = _T("");
	m_TO_GENERAL_EXPENSES = _T("");
	m_TO_STABLE_EXPENSES = _T("");
	m_TO_SELLING_EXPENSES = _T("");
	m_TO_CARRIAGE_OUTWARD = _T("");
	m_TO_CARRIAGE_ON_SALES = _T("");
	m_TO_INDIRECT_WAGES = _T("");
	m_TO_AUDIT_FEES = _T("");
	m_TO_ENTERTAIN_EXPENSES = _T("");
	m_TO_INTEREST_PAID = _T("");
	m_TO_DISCOUNT_ALLOWED = _T("");
	m_TO_BAD_DEBTS = _T("");
	m_TO_RESERVE_FOR_BAD_DEBTS = _T("");
	m_TO_DEPRECIATION = _T("");
	m_TO_INTEREST_ON_CAPITAL = _T("");
	m_TO_DISCOUNTING_CHARGES = _T("");
	m_TO_BANK_CHARGES = _T("");
```

```
	m_TO_EXPORT_CHARGES = _T("");
	m_TO_TRADE_EXPENSES = _T("");
	m_TO_ADMIN_EXPENSES = _T("");
	m_TO_FINANCIAL_EXPENSES = _T("");
	m_TO_COMMISSION_PAID = _T("");
	m_TO_ADVERTISEMENT = _T("");
	m_TO_CHARITY = _T("");
	m_TO_SAMPLE_EXPENSES = _T("");
	m_TO_LICENCE_FEE = _T("");
	m_TO_DELIVERY_CHARGES = _T("");
	m_TO_BROKERAGE = _T("");
	m_TO_SALES_TAX = _T("");
	m_TO_LOSS_ON_SALE_OF_ASSET = _T("");
	m_TO_LOSS_BY_THEFT_ACCIDENT = _T("");
	m_TO_NET_PROFIT = _T("");
	m_BY_GROSS_PROFIT = _T("");
	m_BY_INTEREST_RECEIVED = _T("");
	m_BY_RENT_RECEIVED = _T("");
	m_BY_DISCOUNT_RECEIVED = _T("");
	m_BY_DIVIDENDS_RECEIVED = _T("");
	m_BY_PROF_FROM_SALE_OF_ASSET = _T("");
	m_BY_REFUND_OF_TAX = _T("");
	m_BY_COMPENSAT_RECEIVED = _T("");
	m_BY_APPRENTICESHIP_PREMIUM = _T("");
	m_BY_DIFFER_IN_EXCHANGE = _T("");
	m_BY_INTEREST_ON_DRAWINGS = _T("");
	m_BY_DISCOUNT_ON_CREDITORS = _T("");
	m_BY_BAD_DEBTS_RECOVERED = _T("");
	m_BY_MISC_RECEIPTS = _T("");
	m_BY_INC_IN_VALUE_OF_ASSET = _T("");
	m_BY_INCOME_FROM_INVESTMENT = _T("");
	m_BY_RES_FOR_BAD_DOUBTS = _T("");
	m_BY_NET_LOSS = _T("");
	m_nFields = 78;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}

CString CPLAccountSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CPLAccountSet::GetDefaultSQL()
{
	return _T("[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[PL_ACCOUNT]");
}

void CPLAccountSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CPLAccountSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
```

```
        RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
        RFX_Date(pFX, _T("[DOB]"), m_DOB);
        RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
        RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
        RFX_Text(pFX, _T("[SEX]"), m_SEX);
        RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
        RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
        RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
        RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
        RFX_Text(pFX, _T("[DIGAMBER].[PL_ACCOUNT].[PAN]"), m_PL_ACCOUNT_PAN);
        RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
        RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
        RFX_Text(pFX, _T("[TO_GROSS_LOSS]"), m_TO_GROSS_LOSS);
        RFX_Text(pFX, _T("[TO_SALARIES_WAGES]"), m_TO_SALARIES_WAGES);
        RFX_Text(pFX, _T("[TO_OFFICE_GODOWN_RENT]"), m_TO_OFFICE_GODOWN_RENT);
        RFX_Text(pFX, _T("[TO_OFFICE_EXPENSES]"), m_TO_OFFICE_EXPENSES);
        RFX_Text(pFX, _T("[TO_MISC_SUNDRY_EXPENSE]"),
m_TO_MISC_SUNDRY_EXPENSE);
        RFX_Text(pFX, _T("[TO_INSURANCE]"), m_TO_INSURANCE);
        RFX_Text(pFX, _T("[TO_STATION_PRINT]"), m_TO_STATION_PRINT);
        RFX_Text(pFX, _T("[TO_STAFF_WELF_EXPENSE]"), m_TO_STAFF_WELF_EXPENSE);
        RFX_Text(pFX, _T("[TO_LIGHT_WATER_ELECT]"), m_TO_LIGHT_WATER_ELECT);
        RFX_Text(pFX, _T("[TO_ESTAB_EXPENSE]"), m_TO_ESTAB_EXPENSE);
        RFX_Text(pFX, _T("[TO_POST_TLGRM_FAX_COUR_PH]"),
m_TO_POST_TLGRM_FAX_COUR_PH);
        RFX_Text(pFX, _T("[TO_LAW_CHARGES]"), m_TO_LAW_CHARGES);
        RFX_Text(pFX, _T("[TO_REPAIRS]"), m_TO_REPAIRS);
        RFX_Text(pFX, _T("[TO_DISTRIBUTION_EXPENSES]"),
m_TO_DISTRIBUTION_EXPENSES);
        RFX_Text(pFX, _T("[TO_TRAVEL_EXPENSE]"), m_TO_TRAVEL_EXPENSE);
        RFX_Text(pFX, _T("[TO_GENERAL_EXPENSES]"), m_TO_GENERAL_EXPENSES);
        RFX_Text(pFX, _T("[TO_STABLE_EXPENSES]"), m_TO_STABLE_EXPENSES);
        RFX_Text(pFX, _T("[TO_SELLING_EXPENSES]"), m_TO_SELLING_EXPENSES);
        RFX_Text(pFX, _T("[TO_CARRIAGE_OUTWARD]"), m_TO_CARRIAGE_OUTWARD);
        RFX_Text(pFX, _T("[TO_CARRIAGE_ON_SALES]"), m_TO_CARRIAGE_ON_SALES);
        RFX_Text(pFX, _T("[TO_INDIRECT_WAGES]"), m_TO_INDIRECT_WAGES);
        RFX_Text(pFX, _T("[TO_AUDIT_FEES]"), m_TO_AUDIT_FEES);
        RFX_Text(pFX, _T("[TO_ENTERTAIN_EXPENSES]"), m_TO_ENTERTAIN_EXPENSES);
        RFX_Text(pFX, _T("[TO_INTEREST_PAID]"), m_TO_INTEREST_PAID);
        RFX_Text(pFX, _T("[TO_DISCOUNT_ALLOWED]"), m_TO_DISCOUNT_ALLOWED);
        RFX_Text(pFX, _T("[TO_BAD_DEBTS]"), m_TO_BAD_DEBTS);
        RFX_Text(pFX, _T("[TO_RESERVE_FOR_BAD_DEBTS]"),
m_TO_RESERVE_FOR_BAD_DEBTS);
        RFX_Text(pFX, _T("[TO_DEPRECIATION]"), m_TO_DEPRECIATION);
        RFX_Text(pFX, _T("[TO_INTEREST_ON_CAPITAL]"), m_TO_INTEREST_ON_CAPITAL);
        RFX_Text(pFX, _T("[TO_DISCOUNTING_CHARGES]"),
m_TO_DISCOUNTING_CHARGES);
        RFX_Text(pFX, _T("[TO_BANK_CHARGES]"), m_TO_BANK_CHARGES);
        RFX_Text(pFX, _T("[TO_EXPORT_CHARGES]"), m_TO_EXPORT_CHARGES);
        RFX_Text(pFX, _T("[TO_TRADE_EXPENSES]"), m_TO_TRADE_EXPENSES);
        RFX_Text(pFX, _T("[TO_ADMIN_EXPENSES]"), m_TO_ADMIN_EXPENSES);
        RFX_Text(pFX, _T("[TO_FINANCIAL_EXPENSES]"), m_TO_FINANCIAL_EXPENSES);
        RFX_Text(pFX, _T("[TO_COMMISSION_PAID]"), m_TO_COMMISSION_PAID);
        RFX_Text(pFX, _T("[TO_ADVERTISEMENT]"), m_TO_ADVERTISEMENT);
```

```
		RFX_Text(pFX, _T("[TO_CHARITY]"), m_TO_CHARITY);
		RFX_Text(pFX, _T("[TO_SAMPLE_EXPENSES]"), m_TO_SAMPLE_EXPENSES);
		RFX_Text(pFX, _T("[TO_LICENCE_FEE]"), m_TO_LICENCE_FEE);
		RFX_Text(pFX, _T("[TO_DELIVERY_CHARGES]"), m_TO_DELIVERY_CHARGES);
		RFX_Text(pFX, _T("[TO_BROKERAGE]"), m_TO_BROKERAGE);
		RFX_Text(pFX, _T("[TO_SALES_TAX]"), m_TO_SALES_TAX);
		RFX_Text(pFX, _T("[TO_LOSS_ON_SALE_OF_ASSET]"),
m_TO_LOSS_ON_SALE_OF_ASSET);
		RFX_Text(pFX, _T("[TO_LOSS_BY_THEFT_ACCIDENT]"),
m_TO_LOSS_BY_THEFT_ACCIDENT);
		RFX_Text(pFX, _T("[TO_NET_PROFIT]"), m_TO_NET_PROFIT);
		RFX_Text(pFX, _T("[BY_GROSS_PROFIT]"), m_BY_GROSS_PROFIT);
		RFX_Text(pFX, _T("[BY_INTEREST_RECEIVED]"), m_BY_INTEREST_RECEIVED);
		RFX_Text(pFX, _T("[BY_RENT_RECEIVED]"), m_BY_RENT_RECEIVED);
		RFX_Text(pFX, _T("[BY_DISCOUNT_RECEIVED]"), m_BY_DISCOUNT_RECEIVED);
		RFX_Text(pFX, _T("[BY_DIVIDENDS_RECEIVED]"), m_BY_DIVIDENDS_RECEIVED);
		RFX_Text(pFX, _T("[BY_PROF_FROM_SALE_OF_ASSET]"),
m_BY_PROF_FROM_SALE_OF_ASSET);
		RFX_Text(pFX, _T("[BY_REFUND_OF_TAX]"), m_BY_REFUND_OF_TAX);
		RFX_Text(pFX, _T("[BY_COMPENSAT_RECEIVED]"), m_BY_COMPENSAT_RECEIVED);
		RFX_Text(pFX, _T("[BY_APPRENTICESHIP_PREMIUM]"),
m_BY_APPRENTICESHIP_PREMIUM);
		RFX_Text(pFX, _T("[BY_DIFFER_IN_EXCHANGE]"), m_BY_DIFFER_IN_EXCHANGE);
		RFX_Text(pFX, _T("[BY_INTEREST_ON_DRAWINGS]"),
m_BY_INTEREST_ON_DRAWINGS);
		RFX_Text(pFX, _T("[BY_DISCOUNT_ON_CREDITORS]"),
m_BY_DISCOUNT_ON_CREDITORS);
		RFX_Text(pFX, _T("[BY_BAD_DEBTS_RECOVERED]"),
m_BY_BAD_DEBTS_RECOVERED);
		RFX_Text(pFX, _T("[BY_MISC_RECEIPTS]"), m_BY_MISC_RECEIPTS);
		RFX_Text(pFX, _T("[BY_INC_IN_VALUE_OF_ASSET]"),
m_BY_INC_IN_VALUE_OF_ASSET);
		RFX_Text(pFX, _T("[BY_INCOME_FROM_INVESTMENT]"),
m_BY_INCOME_FROM_INVESTMENT);
		RFX_Text(pFX, _T("[BY_RES_FOR_BAD_DOUBTS]"), m_BY_RES_FOR_BAD_DOUBTS);
		RFX_Text(pFX, _T("[BY_NET_LOSS]"), m_BY_NET_LOSS);
		//}}AFX_FIELD_MAP
}

/////////////////////////////////////////////////////////////////////////////
// CPLAccountSet diagnostics

#ifdef _DEBUG
void CPLAccountSet::AssertValid() const
{
		CRecordset::AssertValid();
}

void CPLAccountSet::Dump(CDumpContext& dc) const
{
		CRecordset::Dump(dc);
}
#endif //_DEBUG

// PLAccountView.cpp : implementation of the CPLAccountView class
```

```
//

#include "stdafx.h"
#include "PLAccount.h"

#include "PLAccountSet.h"
#include "PLAccountDoc.h"
#include "PLAccountView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CPLAccountView

IMPLEMENT_DYNCREATE(CPLAccountView, CRecordView)

BEGIN_MESSAGE_MAP(CPLAccountView, CRecordView)
	//{{AFX_MSG_MAP(CPLAccountView)
	ON_COMMAND(ID_RECORD_ADDRECORD, OnRecordAddrecord)
	ON_COMMAND(ID_RECORD_DELETERECORD, OnRecordDeleterecord)
	ON_BN_CLICKED(IDC_SAVE, OnSave)
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CPLAccountView construction/destruction

CPLAccountView::CPLAccountView()
	: CRecordView(CPLAccountView::IDD)
{
	//{{AFX_DATA_INIT(CPLAccountView)
	m_pSet = NULL;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CPLAccountView::~CPLAccountView()
{
}

void CPLAccountView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CPLAccountView)
	DDX_FieldText(pDX, IDC_APPR_PREMIUM, m_pSet-
>m_BY_APPRENTICESHIP_PREMIUM, m_pSet);
```

```
        DDV_MaxChars(pDX, m_pSet->m_BY_APPRENTICESHIP_PREMIUM, 15);
        DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR1, m_pSet->m_ASSES_YEAR_1);
        DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR2, m_pSet->m_ASSES_YEAR_2);
        DDX_FieldText(pDX, IDC_BAD_DEBTS_RECOVERED, m_pSet-
>m_BY_BAD_DEBTS_RECOVERED, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_BAD_DEBTS_RECOVERED, 15);
        DDX_FieldText(pDX, IDC_COMP_RECEIVED, m_pSet->m_BY_COMPENSAT_RECEIVED,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_COMPENSAT_RECEIVED, 15);
        DDX_FieldText(pDX, IDC_DIFF_IN_EXCHANGE, m_pSet-
>m_BY_DIFFER_IN_EXCHANGE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_DIFFER_IN_EXCHANGE, 15);
        DDX_FieldText(pDX, IDC_DISC_ON_CREDITORS, m_pSet-
>m_BY_DISCOUNT_ON_CREDITORS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_DISCOUNT_ON_CREDITORS, 15);
        DDX_FieldText(pDX, IDC_DISCOUNT_RECEIVED, m_pSet-
>m_BY_DISCOUNT_RECEIVED, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_DISCOUNT_RECEIVED, 15);
        DDX_FieldText(pDX, IDC_DIVIDENT_RECEIVED, m_pSet-
>m_BY_DIVIDENDS_RECEIVED, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_DIVIDENDS_RECEIVED, 15);
        DDX_FieldText(pDX, IDC_GROSS_PROFIT, m_pSet->m_BY_GROSS_PROFIT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_GROSS_PROFIT, 15);
        DDX_FieldText(pDX, IDC_INC_FROM_INVESTMENTS, m_pSet-
>m_BY_INCOME_FROM_INVESTMENT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_INCOME_FROM_INVESTMENT, 15);
        DDX_FieldText(pDX, IDC_INC_IN_VALUE_OF_ASSETS, m_pSet-
>m_BY_INC_IN_VALUE_OF_ASSET, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_INC_IN_VALUE_OF_ASSET, 15);
        DDX_FieldText(pDX, IDC_INTEREST_ON_DRAWING, m_pSet-
>m_BY_INTEREST_ON_DRAWINGS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_INTEREST_ON_DRAWINGS, 15);
        DDX_FieldText(pDX, IDC_INTEREST_RECEIVED, m_pSet-
>m_BY_INTEREST_RECEIVED, m_pSet);
        DDX_FieldText(pDX, IDC_MISC_RECEIPTS, m_pSet->m_BY_MISC_RECEIPTS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_MISC_RECEIPTS, 15);
        DDX_FieldText(pDX, IDC_NET_LOSS, m_pSet->m_BY_NET_LOSS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_NET_LOSS, 15);
        DDX_FieldText(pDX, IDC_PAN, m_pSet->m_PL_ACCOUNT_PAN, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_PL_ACCOUNT_PAN, 15);
        DDX_FieldText(pDX, IDC_PROF_FROM_SALE_ASSETS, m_pSet-
>m_BY_PROF_FROM_SALE_OF_ASSET, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_PROF_FROM_SALE_OF_ASSET, 15);
        DDX_FieldText(pDX, IDC_REFUND_OF_TAX, m_pSet->m_BY_REFUND_OF_TAX,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_REFUND_OF_TAX, 15);
        DDX_FieldText(pDX, IDC_RENT_RECEIVED, m_pSet->m_BY_RENT_RECEIVED, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_RENT_RECEIVED, 15);
        DDX_FieldText(pDX, IDC_RESERVE_FOR_BAD_DEBTS, m_pSet-
>m_BY_RES_FOR_BAD_DOUBTS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_BY_RES_FOR_BAD_DOUBTS, 15);
        DDX_FieldText(pDX, IDC_ADMIN_EXPENSES, m_pSet->m_TO_ADMIN_EXPENSES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_ADMIN_EXPENSES, 15);
```

```
        DDX_FieldText(pDX, IDC_ADVERTISEMENT, m_pSet->m_TO_ADVERTISEMENT,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_ADVERTISEMENT, 15);
        DDX_FieldText(pDX, IDC_AUDIT_FEES, m_pSet->m_TO_AUDIT_FEES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_AUDIT_FEES, 15);
        DDX_FieldText(pDX, IDC_BAD_DEBTS, m_pSet->m_TO_BAD_DEBTS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_BAD_DEBTS, 15);
        DDX_FieldText(pDX, IDC_BROKERAGE, m_pSet->m_TO_BROKERAGE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_BROKERAGE, 15);
        DDX_FieldText(pDX, IDC_CARRIAGE_ON_SALES, m_pSet-
>m_TO_CARRIAGE_ON_SALES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_CARRIAGE_ON_SALES, 15);
        DDX_FieldText(pDX, IDC_CARRIAGE_OUTWARD, m_pSet-
>m_TO_CARRIAGE_OUTWARD, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_CARRIAGE_OUTWARD, 15);
        DDX_FieldText(pDX, IDC_CHARITY, m_pSet->m_TO_CHARITY, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_CHARITY, 15);
        DDX_FieldText(pDX, IDC_COMMISSION_PAID, m_pSet->m_TO_COMMISSION_PAID,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_COMMISSION_PAID, 15);
        DDX_FieldText(pDX, IDC_DELIVERY_CHARGES, m_pSet-
>m_TO_DELIVERY_CHARGES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_DELIVERY_CHARGES, 15);
        DDX_FieldText(pDX, IDC_DEPRECIATION, m_pSet->m_TO_DEPRECIATION, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_DEPRECIATION, 15);
        DDX_FieldText(pDX, IDC_DISCOUNT_ALLOWED, m_pSet-
>m_TO_DISCOUNT_ALLOWED, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_DISCOUNT_ALLOWED, 15);
        DDX_FieldText(pDX, IDC_DISCOUNT_CHARGES, m_pSet-
>m_TO_DISCOUNTING_CHARGES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_DISCOUNTING_CHARGES, 15);
        DDX_FieldText(pDX, IDC_DIST_EXP, m_pSet->m_TO_DISTRIBUTION_EXPENSES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_DISTRIBUTION_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_ENTERTAINMENT_EXP, m_pSet-
>m_TO_ENTERTAIN_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_ENTERTAIN_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_EST_EXP, m_pSet->m_TO_ESTAB_EXPENSE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_ESTAB_EXPENSE, 15);
        DDX_FieldText(pDX, IDC_EXPORT_CHARGES, m_pSet->m_TO_EXPORT_CHARGES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_EXPORT_CHARGES, 15);
        DDX_FieldText(pDX, IDC_FINAN_EXP, m_pSet->m_TO_FINANCIAL_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_FINANCIAL_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_GENERAL_EXP, m_pSet->m_TO_GENERAL_EXPENSES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_GENERAL_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_GROSS_LOSS, m_pSet->m_TO_GROSS_LOSS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_GROSS_LOSS, 15);
        DDX_FieldText(pDX, IDC_INDIRECT_WAGES, m_pSet->m_TO_INDIRECT_WAGES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_INDIRECT_WAGES, 15);
        DDX_FieldText(pDX, IDC_INSURANCE, m_pSet->m_TO_INSURANCE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_INSURANCE, 15);
```

```
        DDX_FieldText(pDX, IDC_INTEREST_ON_CAPITAL, m_pSet-
>m_TO_INTEREST_ON_CAPITAL, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_INTEREST_ON_CAPITAL, 15);
        DDX_FieldText(pDX, IDC_LAW_CHARGE, m_pSet->m_TO_LAW_CHARGES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_LAW_CHARGES, 15);
        DDX_FieldText(pDX, IDC_LICENCE_FEE, m_pSet->m_TO_LICENCE_FEE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_LICENCE_FEE, 15);
        DDX_FieldText(pDX, IDC_LIGHT_WATER, m_pSet->m_TO_LIGHT_WATER_ELECT,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_LIGHT_WATER_ELECT, 15);
        DDX_FieldText(pDX, IDC_LOSS_BY_THEFT, m_pSet-
>m_TO_LOSS_BY_THEFT_ACCIDENT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_LOSS_BY_THEFT_ACCIDENT, 15);
        DDX_FieldText(pDX, IDC_LOSS_ON_SALE_OF_ASSETS, m_pSet-
>m_TO_LOSS_ON_SALE_OF_ASSET, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_LOSS_ON_SALE_OF_ASSET, 15);
        DDX_FieldText(pDX, IDC_MISC_SUNDRY_EXP, m_pSet-
>m_TO_MISC_SUNDRY_EXPENSE, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_MISC_SUNDRY_EXPENSE, 15);
        DDX_FieldText(pDX, IDC_NET_PROFIT, m_pSet->m_TO_NET_PROFIT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_NET_PROFIT, 15);
        DDX_FieldText(pDX, IDC_OFFICE_EXP, m_pSet->m_TO_OFFICE_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_OFFICE_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_OFFICE_GODOWN_RENT, m_pSet-
>m_TO_OFFICE_GODOWN_RENT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_OFFICE_GODOWN_RENT, 15);
        DDX_FieldText(pDX, IDC_POSTAGE_TELEGRAM, m_pSet-
>m_TO_POST_TLGRM_FAX_COUR_PH, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_POST_TLGRM_FAX_COUR_PH, 15);
        DDX_FieldText(pDX, IDC_REPAIR, m_pSet->m_TO_REPAIRS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_REPAIRS, 15);
        DDX_FieldText(pDX, IDC_RESERVE_BAD_DEBTS, m_pSet-
>m_TO_RESERVE_FOR_BAD_DEBTS, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_RESERVE_FOR_BAD_DEBTS, 15);
        DDX_FieldText(pDX, IDC_SALARY_WAGE, m_pSet->m_TO_SALARIES_WAGES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_SALARIES_WAGES, 15);
        DDX_FieldText(pDX, IDC_SALES_TAX, m_pSet->m_TO_SALES_TAX, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_SALES_TAX, 15);
        DDX_FieldText(pDX, IDC_SAMPLE_EXP, m_pSet->m_TO_SAMPLE_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_SAMPLE_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_SELLING_EXP, m_pSet->m_TO_SELLING_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_SELLING_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_STABLE_EXP, m_pSet->m_TO_STABLE_EXPENSES, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_STABLE_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_STAFF_WELFARE, m_pSet->m_TO_STAFF_WELF_EXPENSE,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_STAFF_WELF_EXPENSE, 15);
        DDX_FieldText(pDX, IDC_STAT_PRINTIN, m_pSet->m_TO_STATION_PRINT, m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_STATION_PRINT, 15);
        DDX_FieldText(pDX, IDC_TRADE_EXPENSES, m_pSet->m_TO_TRADE_EXPENSES,
m_pSet);
        DDV_MaxChars(pDX, m_pSet->m_TO_TRADE_EXPENSES, 15);
        DDX_FieldText(pDX, IDC_TRAVELLING_EXP, m_pSet->m_TO_TRAVEL_EXPENSE,
m_pSet);
```

```
	DDV_MaxChars(pDX, m_pSet->m_TO_TRAVEL_EXPENSE, 15);
	DDX_FieldText(pDX, IDC_INDIRECT_PAID, m_pSet->m_TO_INTEREST_PAID, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_INTEREST_PAID, 15);
	DDX_FieldText(pDX, IDC_BANK_CHARGES, m_pSet->m_TO_BANK_CHARGES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TO_BANK_CHARGES, 15);
	//}}AFX_DATA_MAP
}

BOOL CPLAccountView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs

	return CRecordView::PreCreateWindow(cs);
}

void CPLAccountView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_pLAccountSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}

/////////////////////////////////////////////////////////////////////////////
// CPLAccountView printing

BOOL CPLAccountView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
}

void CPLAccountView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
}

void CPLAccountView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CPLAccountView diagnostics

#ifdef _DEBUG
void CPLAccountView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CPLAccountView::Dump(CDumpContext& dc) const
{
```

```
	CRecordView::Dump(dc);
}

CPLAccountDoc* CPLAccountView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CPLAccountDoc)));
	return (CPLAccountDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CPLAccountView database support
CRecordset* CPLAccountView::OnGetRecordset()
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CPLAccountView message handlers

void CPLAccountView::OnRecordAddrecord()
{
	// TODO: Add your command handler code here
	m_pSet->AddNew();

	UpdateData(FALSE);


}

void CPLAccountView::OnRecordDeleterecord()
{
	// TODO: Add your command handler code here
	m_pSet->Delete();
	m_pSet->Requery();
	m_pSet->MoveNext();
	if(m_pSet->IsEOF())
		m_pSet->MoveLast();

	if(m_pSet->IsBOF())
		m_pSet->SetFieldNull(NULL);

	UpdateData(FALSE);
}

void CPLAccountView::OnSave()
{
	// TODO: Add your control notification handler code here
	UpdateData(true);

	if(m_pSet->CanUpdate()) {
		m_pSet->Update();
	}
```

```
	m_pSet->Requery();

	UpdateData(false);

}

// PLAccount.h : main header file for the PLACCOUNT application
//

#if
!defined(AFX_PLACCOUNT_H__BE103D5C_A492_46DB_AEE3_BBEBF8048A42__INCLUDED_)
#define AFX_PLACCOUNT_H__BE103D5C_A492_46DB_AEE3_BBEBF8048A42__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
	#error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"		// main symbols

/////////////////////////////////////////////////////////////////////////////
// CPLAccountApp:
// See PLAccount.cpp for the implementation of this class
//

class CPLAccountApp : public CWinApp
{
public:
	CPLAccountApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CPLAccountApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CPLAccountApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.
```

```
#endif //
!defined(AFX_PLACCOUNT_H__BE103D5C_A492_46DB_AEE3_BBEBF8048A42__INCLUDED_)

// PLAccountDoc.h : interface of the CPLAccountDoc class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_PLACCOUNTDOC_H__2737FD7C_E0A3_41FF_84C1_0E1868731F7D__INCLUDED_)
#define AFX_PLACCOUNTDOC_H__2737FD7C_E0A3_41FF_84C1_0E1868731F7D__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "PLAccountSet.h"


class CPLAccountDoc : public CDocument
{
protected: // create from serialization only
	CPLAccountDoc();
	DECLARE_DYNCREATE(CPLAccountDoc)

// Attributes
public:
	CPLAccountSet m_pLAccountSet;

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CPLAccountDoc)
	public:
	virtual BOOL OnNewDocument();
	//}}AFX_VIRTUAL

// Implementation
public:
	virtual ~CPLAccountDoc();
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
	//{{AFX_MSG(CPLAccountDoc)
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};
```

```
/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_PLACCOUNTDOC_H__2737FD7C_E0A3_41FF_84C1_0E1868731F7D__INCLUDED_)

// PLAccountSet.h : interface of the CPLAccountSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_PLACCOUNTSET_H__C0F4F47F_E472_4DDB_96B9_7F57E6E3117B__INCLUDED_)
#define AFX_PLACCOUNTSET_H__C0F4F47F_E472_4DDB_96B9_7F57E6E3117B__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CPLAccountSet : public CRecordset
{
public:
	CPLAccountSet(CDatabase* pDatabase = NULL);
	DECLARE_DYNAMIC(CPLAccountSet)

// Field/Param Data
	//{{AFX_FIELD(CPLAccountSet, CRecordset)
	CString  m_CLIENT_RECORD_PAN;
	CString  m_CLIENT_NAME;
	CString  m_FATHERS_NAME;
	CTime   m_DOB;
	CString  m_ADDRESS;
	CString  m_TELEPHONE;
	CString  m_SEX;
	CString  m_INDV_HUF_FIRM_AOP_LA;
	CString  m_RESIDENT_NR_NOR;
	CString  m_WARD_CIRCLE_SPECIAL_RANGE;
	CString  m_PINCODE;
	CString  m_PL_ACCOUNT_PAN;
	CTime   m_ASSES_YEAR_1;
	CTime   m_ASSES_YEAR_2;
	CString  m_TO_GROSS_LOSS;
	CString  m_TO_SALARIES_WAGES;
	CString  m_TO_OFFICE_GODOWN_RENT;
	CString  m_TO_OFFICE_EXPENSES;
	CString  m_TO_MISC_SUNDRY_EXPENSE;
	CString  m_TO_INSURANCE;
	CString  m_TO_STATION_PRINT;
	CString  m_TO_STAFF_WELF_EXPENSE;
	CString  m_TO_LIGHT_WATER_ELECT;
	CString  m_TO_ESTAB_EXPENSE;
	CString  m_TO_POST_TLGRM_FAX_COUR_PH;
	CString  m_TO_LAW_CHARGES;
```

```
CString m_TO_REPAIRS;
CString m_TO_DISTRIBUTION_EXPENSES;
CString m_TO_TRAVEL_EXPENSE;
CString m_TO_GENERAL_EXPENSES;
CString m_TO_STABLE_EXPENSES;
CString m_TO_SELLING_EXPENSES;
CString m_TO_CARRIAGE_OUTWARD;
CString m_TO_CARRIAGE_ON_SALES;
CString m_TO_INDIRECT_WAGES;
CString m_TO_AUDIT_FEES;
CString m_TO_ENTERTAIN_EXPENSES;
CString m_TO_INTEREST_PAID;
CString m_TO_DISCOUNT_ALLOWED;
CString m_TO_BAD_DEBTS;
CString m_TO_RESERVE_FOR_BAD_DEBTS;
CString m_TO_DEPRECIATION;
CString m_TO_INTEREST_ON_CAPITAL;
CString m_TO_DISCOUNTING_CHARGES;
CString m_TO_BANK_CHARGES;
CString m_TO_EXPORT_CHARGES;
CString m_TO_TRADE_EXPENSES;
CString m_TO_ADMIN_EXPENSES;
CString m_TO_FINANCIAL_EXPENSES;
CString m_TO_COMMISSION_PAID;
CString m_TO_ADVERTISEMENT;
CString m_TO_CHARITY;
CString m_TO_SAMPLE_EXPENSES;
CString m_TO_LICENCE_FEE;
CString m_TO_DELIVERY_CHARGES;
CString m_TO_BROKERAGE;
CString m_TO_SALES_TAX;
CString m_TO_LOSS_ON_SALE_OF_ASSET;
CString m_TO_LOSS_BY_THEFT_ACCIDENT;
CString m_TO_NET_PROFIT;
CString m_BY_GROSS_PROFIT;
CString m_BY_INTEREST_RECEIVED;
CString m_BY_RENT_RECEIVED;
CString m_BY_DISCOUNT_RECEIVED;
CString m_BY_DIVIDENDS_RECEIVED;
CString m_BY_PROF_FROM_SALE_OF_ASSET;
CString m_BY_REFUND_OF_TAX;
CString m_BY_COMPENSAT_RECEIVED;
CString m_BY_APPRENTICESHIP_PREMIUM;
CString m_BY_DIFFER_IN_EXCHANGE;
CString m_BY_INTEREST_ON_DRAWINGS;
CString m_BY_DISCOUNT_ON_CREDITORS;
CString m_BY_BAD_DEBTS_RECOVERED;
CString m_BY_MISC_RECEIPTS;
CString m_BY_INC_IN_VALUE_OF_ASSET;
CString m_BY_INCOME_FROM_INVESTMENT;
CString m_BY_RES_FOR_BAD_DOUBTS;
CString m_BY_NET_LOSS;
//}}AFX_FIELD

// Overrides
```

```
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CPLAccountSet)
	public:
	virtual CString GetDefaultConnect();	// Default connection string
	virtual CString GetDefaultSQL();	// default SQL for Recordset
	virtual void DoFieldExchange(CFieldExchange* pFX);	// RFX support
	//}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
	virtual void AssertValid() const;
	virtual void Dump(CDumpContext& dc) const;
#endif

};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_PLACCOUNTSET_H__C0F4F47F_E472_4DDB_96B9_7F57E6E3117B__INCLUDED_)

// PLAccountView.h : interface of the CPLAccountView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_PLACCOUNTVIEW_H__AD511D4E_5C4E_4A28_A260_50C9B8886AA5__INCLUDE
D_)
#define
AFX_PLACCOUNTVIEW_H__AD511D4E_5C4E_4A28_A260_50C9B8886AA5__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CPLAccountSet;

class CPLAccountView : public CRecordView
{
protected: // create from serialization only
	CPLAccountView();
	DECLARE_DYNCREATE(CPLAccountView)

public:
	//{{AFX_DATA(CPLAccountView)
	enum { IDD = IDD_PLACCOUNT_FORM };
	CPLAccountSet* m_pSet;
	//}}AFX_DATA

// Attributes
public:
	CPLAccountDoc* GetDocument();

// Operations
```

```
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CPLAccountView)
        public:
        virtual CRecordset* OnGetRecordset();
        virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
        protected:
        virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
        virtual void OnInitialUpdate(); // called first time after construct
        virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
        virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CPLAccountView();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CPLAccountView)
        afx_msg void OnRecordAddrecord();
        afx_msg void OnRecordDeleterecord();
        afx_msg void OnSave();
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in PLAccountView.cpp
inline CPLAccountDoc* CPLAccountView::GetDocument()
   { return (CPLAccountDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_PLACCOUNTVIEW_H__AD511D4E_5C4E_4A28_A260_50C9B8886AA5__INCLUDE
D_)
```

Code of Balance Sheet Form:

```
// BalanceSheet.cpp : Defines the class behaviors for the application.
//
```

```
#include "stdafx.h"
#include "BalanceSheet.h"

#include "MainFrm.h"
#include "BalanceSheetSet.h"
#include "BalanceSheetDoc.h"
#include "BalanceSheetView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetApp

BEGIN_MESSAGE_MAP(CBalanceSheetApp, CWinApp)
	//{{AFX_MSG_MAP(CBalanceSheetApp)
	ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
	// Standard print setup command
	ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetApp construction

CBalanceSheetApp::CBalanceSheetApp()
{
	// TODO: add construction code here,
	// Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CBalanceSheetApp object

CBalanceSheetApp theApp;

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetApp initialization

BOOL CBalanceSheetApp::InitInstance()
{
	AfxEnableControlContainer();

	// Standard initialization
	// If you are not using these features and wish to reduce the size
	//  of your final executable, you should remove from the following
	//  the specific initialization routines you do not need.

#ifdef _AFXDLL
```

```
	Enable3dControls();			// Call this when using MFC in a shared DLL
#else
	Enable3dControlsStatic();	// Call this when linking to MFC statically
#endif

	// Change the registry key under which our settings are stored.
	// TODO: You should modify this string to be something appropriate
	// such as the name of your company or organization.
	SetRegistryKey(_T("Local AppWizard-Generated Applications"));

	LoadStdProfileSettings();  // Load standard INI file options (including MRU)

	// Register the application's document templates.  Document templates
	//  serve as the connection between documents, frame windows and views.

	CSingleDocTemplate* pDocTemplate;
	pDocTemplate = new CSingleDocTemplate(
		IDR_MAINFRAME,
		RUNTIME_CLASS(CBalanceSheetDoc),
		RUNTIME_CLASS(CMainFrame),	// main SDI frame window
		RUNTIME_CLASS(CBalanceSheetView));
	AddDocTemplate(pDocTemplate);

	// Parse command line for standard shell commands, DDE, file open
	CCommandLineInfo cmdInfo;
	ParseCommandLine(cmdInfo);

	// Dispatch commands specified on the command line
	if (!ProcessShellCommand(cmdInfo))
		return FALSE;

	// The one and only window has been initialized, so show and update it.
	m_pMainWnd->ShowWindow(SW_SHOW);
	m_pMainWnd->UpdateWindow();

	return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
	CAboutDlg();

// Dialog Data
	//{{AFX_DATA(CAboutDlg)
	enum { IDD = IDD_ABOUTBOX };
	//}}AFX_DATA

	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CAboutDlg)
	protected:
```

```
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
	//}}AFX_VIRTUAL

// Implementation
protected:
	//{{AFX_MSG(CAboutDlg)
		// No message handlers
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
	//{{AFX_DATA_INIT(CAboutDlg)
	//}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialog::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CAboutDlg)
	//}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
	//{{AFX_MSG_MAP(CAboutDlg)
		// No message handlers
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CBalanceSheetApp::OnAppAbout()
{
	CAboutDlg aboutDlg;
	aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetApp message handlers

// BalanceSheetDoc.cpp : implementation of the CBalanceSheetDoc class
//

#include "stdafx.h"
#include "BalanceSheet.h"

#include "BalanceSheetSet.h"
#include "BalanceSheetDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
```

```
/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetDoc

IMPLEMENT_DYNCREATE(CBalanceSheetDoc, CDocument)

BEGIN_MESSAGE_MAP(CBalanceSheetDoc, CDocument)
	//{{AFX_MSG_MAP(CBalanceSheetDoc)
		// NOTE - the ClassWizard will add and remove mapping macros here.
		//    DO NOT EDIT what you see in these blocks of generated code!
	//}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetDoc construction/destruction

CBalanceSheetDoc::CBalanceSheetDoc()
{
	// TODO: add one-time construction code here

}

CBalanceSheetDoc::~CBalanceSheetDoc()
{
}

BOOL CBalanceSheetDoc::OnNewDocument()
{
	if (!CDocument::OnNewDocument())
		return FALSE;

	// TODO: add reinitialization code here
	// (SDI documents will reuse this document)

	return TRUE;
}



/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetDoc diagnostics

#ifdef _DEBUG
void CBalanceSheetDoc::AssertValid() const
{
	CDocument::AssertValid();
}

void CBalanceSheetDoc::Dump(CDumpContext& dc) const
{
	CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetDoc commands
```

```
// BalanceSheetSet.cpp : implementation of the CBalanceSheetSet class
//

#include "stdafx.h"
#include "BalanceSheet.h"
#include "BalanceSheetSet.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

////////////////////////////////////////////////////////////////////////////
// CBalanceSheetSet implementation

IMPLEMENT_DYNAMIC(CBalanceSheetSet, CRecordset)

CBalanceSheetSet::CBalanceSheetSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CBalanceSheetSet)
	m_BALANCE_SHEET_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_BILLS_PAYABLE = _T("");
	m_SUNDRY_CREDITORS = _T("");
	m_LOANS = _T("");
	m_OUTSTANDING_EXPENSES = _T("");
	m_CAPITAL = _T("");
	m_NET_PROFIT = _T("");
	m_INTEREST_ON_CAPITAL = _T("");
	m_DRAWINGS = _T("");
	m_NET_LOSS = _T("");
	m_INCOME_TAX = _T("");
	m_CASH_IN_HAND = _T("");
	m_CASH_AT_BANK = _T("");
	m_INVESTMENTS = _T("");
	m_BILLS_RECEIVABLE = _T("");
	m_SUNDRY_DEBTORS = _T("");
	m_CLOSING_STOCK = _T("");
	m_STORES = _T("");
	m_PLANT_AND_MACHINERY = _T("");
	m_FREEHOLD_PREMISES = _T("");
	m_UNEXPIRED_EXPENSES = _T("");
	m_GOODWILL = _T("");
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
	m_SEX = _T("");
	m_INDV_HUF_FIRM_AOP_LA = _T("");
```

```
	m_RESIDENT_NR_NOR = _T("");
	m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
	m_PINCODE = _T("");
	m_nFields = 35;
	//}}AFX_FIELD_INIT
	m_nDefaultType = snapshot;
}

CString CBalanceSheetSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CBalanceSheetSet::GetDefaultSQL()
{
	return
_T("[DIGAMBER].[BALANCE_SHEET],[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[INCOME
_TAX_RECORD]");
}

void CBalanceSheetSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CBalanceSheetSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[BALANCE_SHEET].[PAN]"),
m_BALANCE_SHEET_PAN);
	RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
	RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
	RFX_Text(pFX, _T("[BILLS_PAYABLE]"), m_BILLS_PAYABLE);
	RFX_Text(pFX, _T("[SUNDRY_CREDITORS]"), m_SUNDRY_CREDITORS);
	RFX_Text(pFX, _T("[LOANS]"), m_LOANS);
	RFX_Text(pFX, _T("[OUTSTANDING_EXPENSES]"), m_OUTSTANDING_EXPENSES);
	RFX_Text(pFX, _T("[CAPITAL]"), m_CAPITAL);
	RFX_Text(pFX, _T("[NET_PROFIT]"), m_NET_PROFIT);
	RFX_Text(pFX, _T("[INTEREST_ON_CAPITAL]"), m_INTEREST_ON_CAPITAL);
	RFX_Text(pFX, _T("[DRAWINGS]"), m_DRAWINGS);
	RFX_Text(pFX, _T("[NET_LOSS]"), m_NET_LOSS);
	RFX_Text(pFX, _T("[INCOME_TAX]"), m_INCOME_TAX);
	RFX_Text(pFX, _T("[CASH_IN_HAND]"), m_CASH_IN_HAND);
	RFX_Text(pFX, _T("[CASH_AT_BANK]"), m_CASH_AT_BANK);
	RFX_Text(pFX, _T("[INVESTMENTS]"), m_INVESTMENTS);
	RFX_Text(pFX, _T("[BILLS_RECEIVABLE]"), m_BILLS_RECEIVABLE);
	RFX_Text(pFX, _T("[SUNDRY_DEBTORS]"), m_SUNDRY_DEBTORS);
	RFX_Text(pFX, _T("[CLOSING_STOCK]"), m_CLOSING_STOCK);
	RFX_Text(pFX, _T("[STORES]"), m_STORES);
	RFX_Text(pFX, _T("[PLANT_AND_MACHINERY]"), m_PLANT_AND_MACHINERY);
	RFX_Text(pFX, _T("[FREEHOLD_PREMISES]"), m_FREEHOLD_PREMISES);
	RFX_Text(pFX, _T("[UNEXPIRED_EXPENSES]"), m_UNEXPIRED_EXPENSES);
	RFX_Text(pFX, _T("[GOODWILL]"), m_GOODWILL);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
```

```
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	//}}AFX_FIELD_MAP
}

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetSet diagnostics

#ifdef _DEBUG
void CBalanceSheetSet::AssertValid() const
{
	CRecordset::AssertValid();
}

void CBalanceSheetSet::Dump(CDumpContext& dc) const
{
	CRecordset::Dump(dc);
}
#endif //_DEBUG

// BalanceSheetView.cpp : implementation of the CBalanceSheetView class
//

#include "stdafx.h"
#include "BalanceSheet.h"

#include "BalanceSheetSet.h"
#include "BalanceSheetDoc.h"
#include "BalanceSheetView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView

IMPLEMENT_DYNCREATE(CBalanceSheetView, CRecordView)

BEGIN_MESSAGE_MAP(CBalanceSheetView, CRecordView)
	//{{AFX_MSG_MAP(CBalanceSheetView)
	ON_COMMAND(ID_RECORD_ADDRECORD, OnRecordAddrecord)
	ON_COMMAND(ID_RECORD_DELETERECORD, OnRecordDeleterecord)
	ON_BN_CLICKED(IDC_SAVE, OnSave)
	ON_BN_CLICKED(IDC_CLOSE, OnClose)
	//}}AFX_MSG_MAP
	// Standard printing commands
	ON_COMMAND(ID_FILE_PRINT, CRecordView::OnFilePrint)
```

```
	ON_COMMAND(ID_FILE_PRINT_DIRECT, CRecordView::OnFilePrint)
	ON_COMMAND(ID_FILE_PRINT_PREVIEW, CRecordView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView construction/destruction

CBalanceSheetView::CBalanceSheetView()
	: CRecordView(CBalanceSheetView::IDD)
{
	//{{AFX_DATA_INIT(CBalanceSheetView)
	m_pSet = NULL;
	//}}AFX_DATA_INIT
	// TODO: add construction code here

}

CBalanceSheetView::~CBalanceSheetView()
{
}

void CBalanceSheetView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CBalanceSheetView)
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR1, m_pSet->m_ASSES_YEAR_1);
	DDX_FieldText(pDX, IDC_BILLS_PAYABLE, m_pSet->m_BILLS_PAYABLE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BILLS_PAYABLE, 15);
	DDX_FieldText(pDX, IDC_BILLS_RECEIVABLE, m_pSet->m_BILLS_RECEIVABLE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BILLS_RECEIVABLE, 15);
	DDX_FieldText(pDX, IDC_CAPITAL, m_pSet->m_CAPITAL, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_CAPITAL, 15);
	DDX_FieldText(pDX, IDC_CASH_AT_BANK, m_pSet->m_CASH_AT_BANK, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_CASH_AT_BANK, 15);
	DDX_FieldText(pDX, IDC_CASH_IN_HAND, m_pSet->m_CASH_IN_HAND, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_CASH_IN_HAND, 15);
	DDX_FieldText(pDX, IDC_CLOSING_STOCK, m_pSet->m_CLOSING_STOCK, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_CLOSING_STOCK, 15);
	DDX_FieldText(pDX, IDC_DRAWING, m_pSet->m_DRAWINGS, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_DRAWINGS, 15);
	DDX_FieldText(pDX, IDC_FREEHOLD_PREMISES, m_pSet->m_FREEHOLD_PREMISES,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_FREEHOLD_PREMISES, 15);
	DDX_FieldText(pDX, IDC_GOODWILL, m_pSet->m_GOODWILL, m_pSet);
	DDX_FieldText(pDX, IDC_INCOME_TAX, m_pSet->m_INCOME_TAX, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX, 15);
	DDX_FieldText(pDX, IDC_INTERSET_ON_CAPITAL, m_pSet-
>m_INTEREST_ON_CAPITAL, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INTEREST_ON_CAPITAL, 15);
	DDX_FieldText(pDX, IDC_INVESTMENTS, m_pSet->m_INVESTMENTS, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INVESTMENTS, 15);
	DDX_FieldText(pDX, IDC_LOANS, m_pSet->m_LOANS, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LOANS, 15);
	DDX_FieldText(pDX, IDC_NET_LOSS, m_pSet->m_NET_LOSS, m_pSet);
```

```
	DDV_MaxChars(pDX, m_pSet->m_NET_LOSS, 15);
	DDX_FieldText(pDX, IDC_NET_PROFIT, m_pSet->m_NET_PROFIT, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_NET_PROFIT, 15);
	DDX_FieldText(pDX, IDC_OUTSTANDING_EXPENSES, m_pSet-
>m_OUTSTANDING_EXPENSES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_OUTSTANDING_EXPENSES, 15);
	DDX_FieldText(pDX, IDC_PAN, m_pSet->m_BALANCE_SHEET_PAN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BALANCE_SHEET_PAN, 10);
	DDX_FieldText(pDX, IDC_PLANT_MACHINE, m_pSet->m_PLANT_AND_MACHINERY,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_PLANT_AND_MACHINERY, 15);
	DDX_FieldText(pDX, IDC_STORES, m_pSet->m_STORES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_STORES, 15);
	DDX_FieldText(pDX, IDC_SUNDRY_CREDITORS, m_pSet->m_SUNDRY_CREDITORS,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_SUNDRY_CREDITORS, 15);
	DDX_FieldText(pDX, IDC_SUNDRY_DEBTORES, m_pSet->m_SUNDRY_DEBTORS,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_SUNDRY_DEBTORS, 15);
	DDX_FieldText(pDX, IDC_UNEXPIRED_EXP, m_pSet->m_UNEXPIRED_EXPENSES,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_UNEXPIRED_EXPENSES, 15);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR2, m_pSet->m_ASSES_YEAR_2);
	//}}AFX_DATA_MAP
}

BOOL CBalanceSheetView::PreCreateWindow(CREATESTRUCT& cs)
{
	// TODO: Modify the Window class or styles here by modifying
	//  the CREATESTRUCT cs

	return CRecordView::PreCreateWindow(cs);
}

void CBalanceSheetView::OnInitialUpdate()
{
	m_pSet = &GetDocument()->m_balanceSheetSet;
	CRecordView::OnInitialUpdate();
	GetParentFrame()->RecalcLayout();
	ResizeParentToFit();

}

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView printing

BOOL CBalanceSheetView::OnPreparePrinting(CPrintInfo* pInfo)
{
	// default preparation
	return DoPreparePrinting(pInfo);
}

void CBalanceSheetView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add extra initialization before printing
```

```
}

void CBalanceSheetView::OnEndPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
{
	// TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView diagnostics

#ifdef _DEBUG
void CBalanceSheetView::AssertValid() const
{
	CRecordView::AssertValid();
}

void CBalanceSheetView::Dump(CDumpContext& dc) const
{
	CRecordView::Dump(dc);
}

CBalanceSheetDoc* CBalanceSheetView::GetDocument() // non-debug version is inline
{
	ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CBalanceSheetDoc)));
	return (CBalanceSheetDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView database support
CRecordset* CBalanceSheetView::OnGetRecordset()
{
	return m_pSet;
}


/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetView message handlers

void CBalanceSheetView::OnRecordAddrecord()
{
	// TODO: Add your command handler code here
	m_pSet->AddNew();

	UpdateData(FALSE);

}

void CBalanceSheetView::OnRecordDeleterecord()
{
	// TODO: Add your command handler code here
	m_pSet->Delete();
	m_pSet->Requery();
	m_pSet->MoveNext();
	if(m_pSet->IsEOF())
```

```
                m_pSet->MoveLast();

        if(m_pSet->IsBOF())
                m_pSet->SetFieldNull(NULL);

        UpdateData(FALSE);

}

void CBalanceSheetView::OnSave()
{
        // TODO: Add your control notification handler code here
        UpdateData(true);

        if(m_pSet->CanUpdate()) {
                m_pSet->Update();
        }

        m_pSet->Requery();

        UpdateData(false);

}

void CBalanceSheetView::OnClose()
{
        // TODO: Add your control notification handler code here

}

// BalanceSheet.h : main header file for the BALANCESHEET application
//

#if
!defined(AFX_BALANCESHEET_H__2252E038_E590_4A59_857D_BB3CAF24BADE__INCLUDED_
)
#define AFX_BALANCESHEET_H__2252E038_E590_4A59_857D_BB3CAF24BADE__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

#ifndef __AFXWIN_H__
        #error include 'stdafx.h' before including this file for PCH
#endif

#include "resource.h"       // main symbols

/////////////////////////////////////////////////////////////////////////////
// CBalanceSheetApp:
// See BalanceSheet.cpp for the implementation of this class
//

class CBalanceSheetApp : public CWinApp
{
```

```
public:
	CBalanceSheetApp();

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CBalanceSheetApp)
	public:
	virtual BOOL InitInstance();
	//}}AFX_VIRTUAL

// Implementation
	//{{AFX_MSG(CBalanceSheetApp)
	afx_msg void OnAppAbout();
		// NOTE - the ClassWizard will add and remove member functions here.
		//    DO NOT EDIT what you see in these blocks of generated code !
	//}}AFX_MSG
	DECLARE_MESSAGE_MAP()
};


/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_BALANCESHEET_H__2252E038_E590_4A59_857D_BB3CAF24BADE__INCLUDED_
)

// BalanceSheetDoc.h : interface of the CBalanceSheetDoc class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_BALANCESHEETDOC_H__CDA9B65E_31A9_44C4_B388_510F25EFE89D__INCLUD
ED_)
#define
AFX_BALANCESHEETDOC_H__CDA9B65E_31A9_44C4_B388_510F25EFE89D__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
#include "BalanceSheetSet.h"


class CBalanceSheetDoc : public CDocument
{
protected: // create from serialization only
	CBalanceSheetDoc();
	DECLARE_DYNCREATE(CBalanceSheetDoc)

// Attributes
public:
	CBalanceSheetSet m_balanceSheetSet;
```

```
// Operations
public:

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CBalanceSheetDoc)
        public:
        virtual BOOL OnNewDocument();
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CBalanceSheetDoc();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CBalanceSheetDoc)
                // NOTE - the ClassWizard will add and remove member functions here.
                //    DO NOT EDIT what you see in these blocks of generated code !
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_BALANCESHEETDOC_H__CDA9B65E_31A9_44C4_B388_510F25EFE89D__INCLUD
ED_)

// BalanceSheetSet.h : interface of the CBalanceSheetSet class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_BALANCESHEETSET_H__675B20D0_A977_4328_ADB8_DE5B0875AB03__INCLUD
ED_)
#define
AFX_BALANCESHEETSET_H__675B20D0_A977_4328_ADB8_DE5B0875AB03__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CBalanceSheetSet : public CRecordset
{
public:
```

```
        CBalanceSheetSet(CDatabase* pDatabase = NULL);
        DECLARE_DYNAMIC(CBalanceSheetSet)

// Field/Param Data
        //{{AFX_FIELD(CBalanceSheetSet, CRecordset)
        CString  m_BALANCE_SHEET_PAN;
        CTime   m_ASSES_YEAR_1;
        CTime   m_ASSES_YEAR_2;
        CString  m_BILLS_PAYABLE;
        CString  m_SUNDRY_CREDITORS;
        CString  m_LOANS;
        CString  m_OUTSTANDING_EXPENSES;
        CString  m_CAPITAL;
        CString  m_NET_PROFIT;
        CString  m_INTEREST_ON_CAPITAL;
        CString  m_DRAWINGS;
        CString  m_NET_LOSS;
        CString  m_INCOME_TAX;
        CString  m_CASH_IN_HAND;
        CString  m_CASH_AT_BANK;
        CString  m_INVESTMENTS;
        CString  m_BILLS_RECEIVABLE;
        CString  m_SUNDRY_DEBTORS;
        CString  m_CLOSING_STOCK;
        CString  m_STORES;
        CString  m_PLANT_AND_MACHINERY;
        CString  m_FREEHOLD_PREMISES;
        CString  m_UNEXPIRED_EXPENSES;
        CString  m_GOODWILL;
        CString  m_CLIENT_RECORD_PAN;
        CString  m_CLIENT_NAME;
        CString  m_FATHERS_NAME;
        CTime   m_DOB;
        CString  m_ADDRESS;
        CString  m_TELEPHONE;
        CString  m_SEX;
        CString  m_INDV_HUF_FIRM_AOP_LA;
        CString  m_RESIDENT_NR_NOR;
        CString  m_WARD_CIRCLE_SPECIAL_RANGE;
        CString  m_PINCODE;
        //}}AFX_FIELD

// Overrides
        // ClassWizard generated virtual function overrides
        //{{AFX_VIRTUAL(CBalanceSheetSet)
        public:
        virtual CString GetDefaultConnect();            // Default connection string
        virtual CString GetDefaultSQL();    // default SQL for Recordset
        virtual void DoFieldExchange(CFieldExchange* pFX);          // RFX support
        //}}AFX_VIRTUAL

// Implementation
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
```

```
#endif

};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_BALANCESHEETSET_H__675B20D0_A977_4328_ADB8_DE5B0875AB03__INCLUD
ED_)


// BalanceSheetView.h : interface of the CBalanceSheetView class
//
/////////////////////////////////////////////////////////////////////////////

#if
!defined(AFX_BALANCESHEETVIEW_H__774DD445_FAD5_4645_97DD_D6294D2B8197__INCLU
DED_)
#define
AFX_BALANCESHEETVIEW_H__774DD445_FAD5_4645_97DD_D6294D2B8197__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CBalanceSheetSet;

class CBalanceSheetView : public CRecordView
{
protected: // create from serialization only
	CBalanceSheetView();
	DECLARE_DYNCREATE(CBalanceSheetView)

public:
	//{{AFX_DATA(CBalanceSheetView)
	enum { IDD = IDD_BALANCESHEET_FORM };
	CBalanceSheetSet* m_pSet;
	//}}AFX_DATA

// Attributes
public:
	CBalanceSheetDoc* GetDocument();

// Operations
public:

// Overrides
	// ClassWizard generated virtual function overrides
	//{{AFX_VIRTUAL(CBalanceSheetView)
	public:
	virtual CRecordset* OnGetRecordset();
	virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
	protected:
	virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
```

```
        virtual void OnInitialUpdate(); // called first time after construct
        virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
        virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
        virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
        //}}AFX_VIRTUAL

// Implementation
public:
        virtual ~CBalanceSheetView();
#ifdef _DEBUG
        virtual void AssertValid() const;
        virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
        //{{AFX_MSG(CBalanceSheetView)
        afx_msg void OnRecordAddrecord();
        afx_msg void OnRecordDeleterecord();
        afx_msg void OnSave();
        afx_msg void OnClose();
        //}}AFX_MSG
        DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in BalanceSheetView.cpp
inline CBalanceSheetDoc* CBalanceSheetView::GetDocument()
   { return (CBalanceSheetDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately before the previous line.

#endif //
!defined(AFX_BALANCESHEETVIEW_H__774DD445_FAD5_4645_97DD_D6294D2B8197__INCLU
DED_)
```

9. Code Efficiency:

In **this** project we used code in efficient way that any error and exceptional may handle. Application gives a suitable error message for any error in meaningful manner. On the basis of this error message user correct the error. Errors are trapped by the use of code

```
CRecordset* CSectionView::OnGetRecordset()
{
   if ( m_pSet != NULL )
      return m_pSet;        // Recordset already allocated

   m_pSet = new CSectionSet( NULL );
   try
   {
      m_pSet->Open( );
   }
   catch( CDBException* e )
   {
      AfxMessageBox( e->m_strError,
                  MB_ICONEXCLAMATION );
      // Delete the incomplete recordset object
      delete m_pSet;
      m_pSet = NULL;
      e->Delete();
   }
   return m_pSet;
}
```

10. Optimization of Code:

Optimization of Code is required for a language to be Object Oriented Language. Visual C++ is an Object Oriented Language; hence Object Oriented Features are available there. Coding of all the forms is much optimal as when ever a certain procedure is required many times it is written as Function for code optimization under the Form Module or Standard Module as required.

For connection with database we will use following code: -

```
CString CBalanceSheetSet::GetDefaultConnect()
{
	return _T("ODBC;DSN=DATA_SOURCE_FOR_PROJECT");
}

CString CBalanceSheetSet::GetDefaultSQL()
{
	return
_T("[DIGAMBER].[BALANCE_SHEET],[DIGAMBER].[CLIENT_RECORD],[DIGAMBER].[INCOME
_TAX_RECORD]");
}
```

Following code will insert values into database:

```
void CBalanceSheetSet::DoFieldExchange(CFieldExchange* pFX)
{
	//{{AFX_FIELD_MAP(CBalanceSheetSet)
	pFX->SetFieldType(CFieldExchange::outputColumn);
	RFX_Text(pFX, _T("[DIGAMBER].[BALANCE_SHEET].[PAN]"),
m_BALANCE_SHEET_PAN);
	RFX_Date(pFX, _T("[ASSES_YEAR_1]"), m_ASSES_YEAR_1);
	RFX_Date(pFX, _T("[ASSES_YEAR_2]"), m_ASSES_YEAR_2);
	RFX_Text(pFX, _T("[BILLS_PAYABLE]"), m_BILLS_PAYABLE);
	RFX_Text(pFX, _T("[SUNDRY_CREDITORS]"), m_SUNDRY_CREDITORS);
	RFX_Text(pFX, _T("[LOANS]"), m_LOANS);
	RFX_Text(pFX, _T("[OUTSTANDING_EXPENSES]"), m_OUTSTANDING_EXPENSES);
	RFX_Text(pFX, _T("[CAPITAL]"), m_CAPITAL);
	RFX_Text(pFX, _T("[NET_PROFIT]"), m_NET_PROFIT);
	RFX_Text(pFX, _T("[INTEREST_ON_CAPITAL]"), m_INTEREST_ON_CAPITAL);
	RFX_Text(pFX, _T("[DRAWINGS]"), m_DRAWINGS);
	RFX_Text(pFX, _T("[NET_LOSS]"), m_NET_LOSS);
	RFX_Text(pFX, _T("[INCOME_TAX]"), m_INCOME_TAX);
	RFX_Text(pFX, _T("[CASH_IN_HAND]"), m_CASH_IN_HAND);
	RFX_Text(pFX, _T("[CASH_AT_BANK]"), m_CASH_AT_BANK);
	RFX_Text(pFX, _T("[INVESTMENTS]"), m_INVESTMENTS);
	RFX_Text(pFX, _T("[BILLS_RECEIVABLE]"), m_BILLS_RECEIVABLE);
	RFX_Text(pFX, _T("[SUNDRY_DEBTORS]"), m_SUNDRY_DEBTORS);
	RFX_Text(pFX, _T("[CLOSING_STOCK]"), m_CLOSING_STOCK);
	RFX_Text(pFX, _T("[STORES]"), m_STORES);
```

```
	RFX_Text(pFX, _T("[PLANT_AND_MACHINERY]"), m_PLANT_AND_MACHINERY);
	RFX_Text(pFX, _T("[FREEHOLD_PREMISES]"), m_FREEHOLD_PREMISES);
	RFX_Text(pFX, _T("[UNEXPIRED_EXPENSES]"), m_UNEXPIRED_EXPENSES);
	RFX_Text(pFX, _T("[GOODWILL]"), m_GOODWILL);
	RFX_Text(pFX, _T("[DIGAMBER].[CLIENT_RECORD].[PAN]"),
m_CLIENT_RECORD_PAN);
	RFX_Text(pFX, _T("[CLIENT_NAME]"), m_CLIENT_NAME);
	RFX_Text(pFX, _T("[FATHERS_NAME]"), m_FATHERS_NAME);
	RFX_Date(pFX, _T("[DOB]"), m_DOB);
	RFX_Text(pFX, _T("[ADDRESS]"), m_ADDRESS);
	RFX_Text(pFX, _T("[TELEPHONE]"), m_TELEPHONE);
	RFX_Text(pFX, _T("[SEX]"), m_SEX);
	RFX_Text(pFX, _T("[INDV_HUF_FIRM_AOP_LA]"), m_INDV_HUF_FIRM_AOP_LA);
	RFX_Text(pFX, _T("[RESIDENT_NR_NOR]"), m_RESIDENT_NR_NOR);
	RFX_Text(pFX, _T("[WARD_CIRCLE_SPECIAL_RANGE]"),
m_WARD_CIRCLE_SPECIAL_RANGE);
	RFX_Text(pFX, _T("[PINCODE]"), m_PINCODE);
	//}}AFX_FIELD_MAP
}
```

Following code will update the database:

```
CBalanceSheetSet::CBalanceSheetSet(CDatabase* pdb)
	: CRecordset(pdb)
{
	//{{AFX_FIELD_INIT(CBalanceSheetSet)
	m_BALANCE_SHEET_PAN = _T("");
	m_ASSES_YEAR_1 = 0;
	m_ASSES_YEAR_2 = 0;
	m_BILLS_PAYABLE = _T("");
	m_SUNDRY_CREDITORS = _T("");
	m_LOANS = _T("");
	m_OUTSTANDING_EXPENSES = _T("");
	m_CAPITAL = _T("");
	m_NET_PROFIT = _T("");
	m_INTEREST_ON_CAPITAL = _T("");
	m_DRAWINGS = _T("");
	m_NET_LOSS = _T("");
	m_INCOME_TAX = _T("");
	m_CASH_IN_HAND = _T("");
	m_CASH_AT_BANK = _T("");
	m_INVESTMENTS = _T("");
	m_BILLS_RECEIVABLE = _T("");
	m_SUNDRY_DEBTORS = _T("");
	m_CLOSING_STOCK = _T("");
	m_STORES = _T("");
	m_PLANT_AND_MACHINERY = _T("");
	m_FREEHOLD_PREMISES = _T("");
	m_UNEXPIRED_EXPENSES = _T("");
	m_GOODWILL = _T("");
	m_CLIENT_RECORD_PAN = _T("");
	m_CLIENT_NAME = _T("");
	m_FATHERS_NAME = _T("");
	m_DOB = 0;
	m_ADDRESS = _T("");
	m_TELEPHONE = _T("");
```

```
        m_SEX = _T("");
        m_INDV_HUF_FIRM_AOP_LA = _T("");
        m_RESIDENT_NR_NOR = _T("");
        m_WARD_CIRCLE_SPECIAL_RANGE = _T("");
        m_PINCODE = _T("");
        m_nFields = 35;
        //}}AFX_FIELD_INIT
        m_nDefaultType = snapshot;
}
```

11. Validation checks:

Visual C++ provides a way to easily handle the validation test for Text box, Combo box, Dtpicker and any other controls .So that only valid data may be inserted by user.

Those handle these validations. Following code will force the validation of maximume characters, as well as type of data to be entered: -

```
void CIncomeTaxRecordView::DoDataExchange(CDataExchange* pDX)
{
	CRecordView::DoDataExchange(pDX);
	//{{AFX_DATA_MAP(CIncomeTaxRecordView)
	DDX_FieldText(pDX, IDC_AGR_INCOME, m_pSet->m_AGRICULTURAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_AGRICULTURAL_INCOME, 15);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR1, m_pSet->m_ASSES_YEAR_1);
	DDX_DateTimeCtrl(pDX, IDC_ASSESS_YEAR2, m_pSet->m_ASSES_YEAR_2);
	DDX_FieldText(pDX, IDC_PAN, m_pSet->m_INCOME_TAX_RECORD_PAN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_RECORD_PAN, 10);
	DDX_FieldText(pDX, IDC_ADD_SURCHARGE, m_pSet->m_ADDITIONAL_SURCHARGE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_ADDITIONAL_SURCHARGE, 15);
	DDX_FieldText(pDX, IDC_BALANCE_TAX, m_pSet->m_BALANCE_TAX, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_BALANCE_TAX, 15);
	DDX_FieldText(pDX, IDC_CAP_GAIN_TOTAL, m_pSet->m_TOTAL_CAPITAL_GAINS,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_CAPITAL_GAINS, 15);
	DDX_FieldText(pDX, IDC_GROSS_TOT_INC, m_pSet->m_GROSS_TOTAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_GROSS_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_INC_EXMPT, m_pSet->m_INCOME_TO_BE_EXEMPTED,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TO_BE_EXEMPTED, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_BUS_PROF, m_pSet-
>m_INC_FROM_BUS_OR_PROF, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INC_FROM_BUS_OR_PROF, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_OTH_SRC, m_pSet-
>m_INCOME_FROM_OTHER_SOURCES, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_OTHER_SOURCES, 15);
	DDX_FieldText(pDX, IDC_INC_FRM_SAL, m_pSet->m_INCOME_FROM_SALARY,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_SALARY, 15);
	DDX_FieldText(pDX, IDC_INC_HOUSE_PROP, m_pSet-
>m_INCOME_FROM_HOUSE_PROPERTY, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FROM_HOUSE_PROPERTY, 15);
	DDX_FieldText(pDX, IDC_INC_NOR_RATE, m_pSet->m_INCOME_AT_NORM_RATE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_AT_NORM_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_OTH_PERS, m_pSet-
>m_INCM_OTHER_PERS_TO_ADDED, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCM_OTHER_PERS_TO_ADDED, 15);
```

```
	DDX_FieldText(pDX, IDC_INC_SPCL_RATE, m_pSet->m_INCOME_AT_SPCL_RATE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_AT_SPCL_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_TAX_NOR_RATE, m_pSet-
>m_INCOME_TAX_NORM_RATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_NORM_RATE, 15);
	DDX_FieldText(pDX, IDC_INC_TAX_SPCL_RATE, m_pSet-
>m_INCOME_TAX_SPCL_RATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_TAX_SPCL_RATE, 15);
	DDX_FieldText(pDX, IDC_LESS_ADV_TAX_PAID, m_pSet->m_ADVANCE_TAX_PAID,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_ADVANCE_TAX_PAID, 15);
	DDX_FieldText(pDX, IDC_LESS_DEDUCT_VI_A, m_pSet-
>m_LESS_DEDUCTION_UNDER_VIA, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_DEDUCTION_UNDER_VIA, 15);
	DDX_FieldText(pDX, IDC_LESS_REBATE, m_pSet->m_LESS_REBATE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_REBATE, 15);
	DDX_FieldText(pDX, IDC_LESS_RELIEF, m_pSet->m_LESS_RELIEF, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_LESS_RELIEF, 15);
	DDX_FieldText(pDX, IDC_LESS_TAX_DED_SOURCE, m_pSet-
>m_TAX_DEDUC_AT_SOURCE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_DEDUC_AT_SOURCE, 15);
	DDX_FieldText(pDX, IDC_LESS_TOT_SELF_ASSESS_TAX_PAID, m_pSet-
>m_TOT_SELF_ASSESS_TAX_PAID, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOT_SELF_ASSESS_TAX_PAID, 15);
	DDX_FieldText(pDX, IDC_LONG_TERM_TOT, m_pSet->m_TOTAL_LONG_TERM_GAIN,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_LONG_TERM_GAIN, 15);
	DDX_FieldText(pDX, IDC_NET_TAX_PAYABLE, m_pSet->m_NET_TAX_PAYABLE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_NET_TAX_PAYABLE, 15);
	DDX_FieldText(pDX, IDC_PREV_YR_INC, m_pSet->m_INCOME_FOR_PREV_YEAR,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_INCOME_FOR_PREV_YEAR, 15);
	DDX_FieldText(pDX, IDC_SHORT_TERM_TOTAL, m_pSet-
>m_TOTAL_SHORT_TERM_GAIN, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_SHORT_TERM_GAIN, 15);
	DDX_FieldText(pDX, IDC_TAX_ON_TOT_INC, m_pSet->m_TAX_ON_TOTAL_INCOME,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_ON_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_TAX_PAYABLE, m_pSet->m_TAX_PAYABLE, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TAX_PAYABLE, 15);
	DDX_FieldText(pDX, IDC_TOT_INCOME, m_pSet->m_TOTAL_INCOME, m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_INCOME, 15);
	DDX_FieldText(pDX, IDC_TOT_TAX_PAYABLE, m_pSet->m_TOTAL_TAX_PAYABLE,
m_pSet);
	DDV_MaxChars(pDX, m_pSet->m_TOTAL_TAX_PAYABLE, 15);
	//}}AFX_DATA_MAP
}
```

12. Testing:

This is an important stage regarding to product and can overlap with programming stage. This phase consist of testing this project to make sure that the expected output is generated by a given output. It is important to test all kinds of input, not just valid input. Suppose that WE have a form that instruct user to enter the amount, to be given as income tax, is greater than amount specified in slab. Although We obviously want to test values such as only numeric value, alphabets & special characters should not be allowed there. We should also try inputs like -100000 and hello. The importance of software testing and its implementing with respect to software quality cannot be overemphasized. Software testing is a critical element of software quality assurance and represent the ultimate review of specification, design and code generation.

Testing Objectives:

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has a high probability of finding an as-yet undiscovered error.
- A successful test is one that uncovers an as-yet-undiscovered error.

System testing is done when the entire system has been fully integrated. The purpose of the system testing is to test how the different modules interact with each other and whether the entire system provides the functionality that was expected.

Testing Consist Of The Following Steps:

1) Program Testing
2) String Testing
3) System Testing
4) System Documentation
5) User Acceptance Testing

Various Levels Of Testing: Before implementation the system is tested at two levels, Level I and Level II.

(A) LEVEL I TESTING (ALPHA TESTING):

At this level a test data is prepared for testing. Project leaders test the system on this test data keeping the following points into consideration:

- ✓ Proper Error handling
- ✓ Exit Points in code
- ✓ Exception handling
- ✓ Input / Output format
- ✓ Glass Box testing
- ✓ Black Box testing

If the system is through with testing phase at LEVEL I then it is passed on to LEVEL II.

(B) LEVEL II TESTING (BETA TESTING):

Here the testing is done on the live database. If errors are detected then it is sent back to LEVEL I for modification otherwise it is passed on to LEVEL III

This is the level at which the system actually becomes live and implemented for the use of END USERS.

(C) LEVEL III:

Here the error – free and properly tested system is implemented.

QUALITY ASSURANCE:

Proper documentation is must for maintenance of any software. Apart from In-line documentation while coding, help files corresponding to each program were prepared so as to tackle the person- dependency of the existing system.

The philosophy behind testing is to find errors. Test cases are devised with the sole purpose in mind. A "Test case" is a set of data that a system will process as normal input. However, the data was created with excess intent of determining whether the system will process them correctly. Thus testing is a process of executing programs with the explicit intention of finding errors, and careful testing makes the system error free. Following were the steps that were/are implemented for testing.

a) Unit testing*:*

In unit testing the program are tested which make up the system. The software units in a system are modules and routines that are assembled and integrated to perform a specific function. Unit testing keep focus on modules and routines, independent of one another to locate errors.

b) System testing*:*

System testing does not test the software per routine or module but rather the integration of each module, in the system. It also tests the differences between the system & its original objective, current specifications and system documentation. The primary job thus is to check compatibility of individual modules.

During testing of Income Tax Evaluation & Maintenance System, the above-mentioned steps were performed and detected problems were corrected deliberately. Wrong data was entered to check for different validations kept in the program and the user was made aware of messages generated at such error performed. Actual procedure in the system is incorporated to suit the user of system. The user was given training for one week. If the result of the new system perfectly matches with the current manual system then manual system of maintaining the register should be totally stopped. Since the system is menu driven and on line it gives result at any time. The user has now to enter the data properly.

13. Implementation:

The following points are necessary for the implementation of the Income Tax Evaluation & Maintenance System:

- ✓ Procurement of necessary hardware and software for installation.
- ✓ Preparation of the user manual for the fulfillment of the user's requirements.
- ✓ To train the end user to how to operate the system.
- ✓ To run the system parallel with the existing system on trial basis for evaluation

14. Maintenance:

For the project, 'Income Tax Evaluation & Maintenance System' we have used the Visual C++ 6.0 as front end, and we know that the Executable file created by Visual C++ is a true EXE i.e. one can run it in an environment where the Visual C++ does not exist.

Extra features that is provided by Visual C++, is the facility to make packages and deployment of software very easily and we make the software in a particular version. While doing the maintenance work we can make another version with the more features including the existing features. After completion of maintains work, one can run a batch program that is available in package. It will make a new exe file which replace the older one. This makes us easy to make changes to the software while having the database consistent.

15. Security Measures Taken:

In the 'Income Tax Evaluation & Maintenance System' we have taken mainly the three security preservers through software coding, to protect the data and system by unauthorized users:

1) User of the system will have to go through the login screen and must enter the User Id and Password to enter to the Main Menu. If any one who doesn't no the password and gives wrong user id and password then system will inform about wrong information entered. If he/she cancels this login prompt then application will unload itself.
2) Most important is the use of Oracle Database Management system. It has its own password. Only valid user can enter in oracle database that knows user id and password. We also create trigger in oracle for security. It get fired when any updation, modification, deletion will be done on any table. It keeps record of date, time, user id, name of table, old value, and altered value on which above operation is done, and thus we can keep record of any discrepancy done.
3) The use of Windows-NT will not provide access the system to unauthorized users.

The above security will stop the unauthorized user to use the system. We Make two provisions in case of hardware failure. These are:

(1) For hardware failure especially in the case of hard disk failure, oracle has the Option, of MIRROR hard disk, which back up the record in another hard disk.

(2) In the case of server failure Windows NT has the facility to run another CPU, which are, connected parallel to server and back up all the data of server and invoked in the case of server failure automatically.

16. Cost Estimation of project:

The cost estimation of project has been derived using function-point metrics. Function-oriented metrics were first proposed by Albretcht. Who suggested a measure called the

function point. Function points are derived using an empirical relationship based on

Information Domain value	Optimistic	Likely	Pessimistic	Estimated Cost	Weight	Function-Point Count
No of Input	5	9	14	9	5	45
No. Of Output	20	25	28	25	4	100
No. Of Inquiries	5	7	10	7	4	28
No. Of Files	14	14	17	14	9	126
No. Of External Interface	10	14	16	14	7	98
Cost Count						**397**

countable (direct) measures of software's information domain and assessment of software Complexity.

Function points are computed by completing the table shown below. Five information domain characteristics are determined and counts are provided in the appropriate table location. For the purposes of this estimate, the complexity-weighting factor is assumed to be average.

Function Point Domain:

Complexity Adjustment Factor:

Factor	Value (0 TO 5) Not applicable to absolutely essential
Backup and Recovery	4
Data Communications	4
Distributed processing	0
Performance Critical	4
Existing Operating Environment	2

On-line Data Entry	0
Input Transaction Over Multiple Screen	0
Master File Updated On-line	0
Information Domain Values Complex	5
Internal Processing Complex	5
Code Designed For Reuse	4
Conversion / Installation in Design	4
Multiple Installations	0
Application Designed For Change	5
Total $\sum(F_i)$	**37**

FPestimated = Count Total * [0.65 + 0.01 * $\sum(F_i)$]

= 397 *1.02

= 405

Finally, The estimated FP is derived and it is 405.

Taking an average of 51 FP / Month

Estimated Effort Time = 405/ 51 = 8 Person-Months Approx.

Project Cost Estimation Sheet:

Cost Head	Unit Rate (Rs./Month)	Estimated Usage (Person- Months)	Cost (Rs.)
Man Power Cost	7000	8	56000
Hardware Cost	2000	8	1600
Software Cost:			

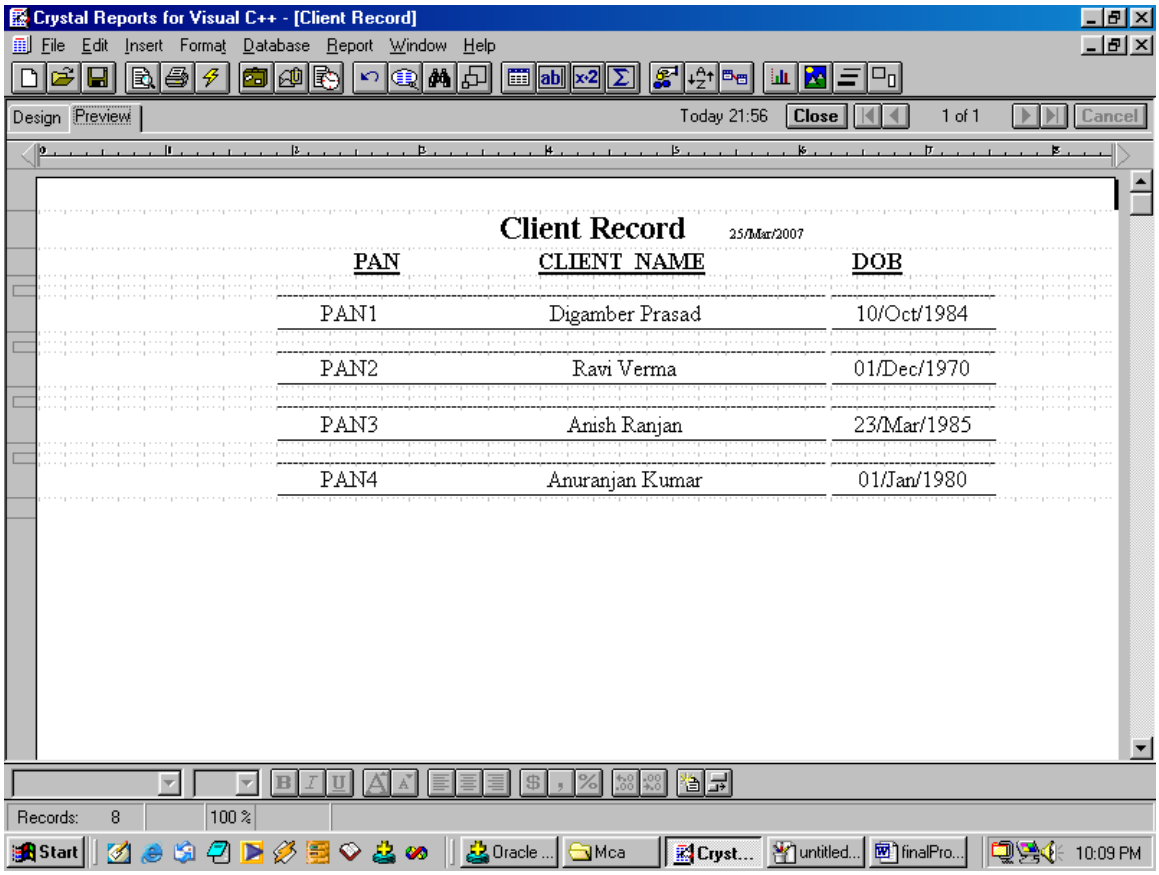
Window NT	1000 Per User	1 User for 8 Months	8000
Visual C++ 6.0	1000 Per User	1 User for 8 Months	8000
Seagate Crystal Report	800 Per User	1 User for 1 Months	800
Travel Cost to Client Site	100 Per Trip	3 Trip /Month for 8 Months	2400
Training Cost	5000	1	5000
Administration	200 Per Seat	2 Seat Per Month for 8 Months	3200
Total		**Rs.**	**99400**

17. Reports:

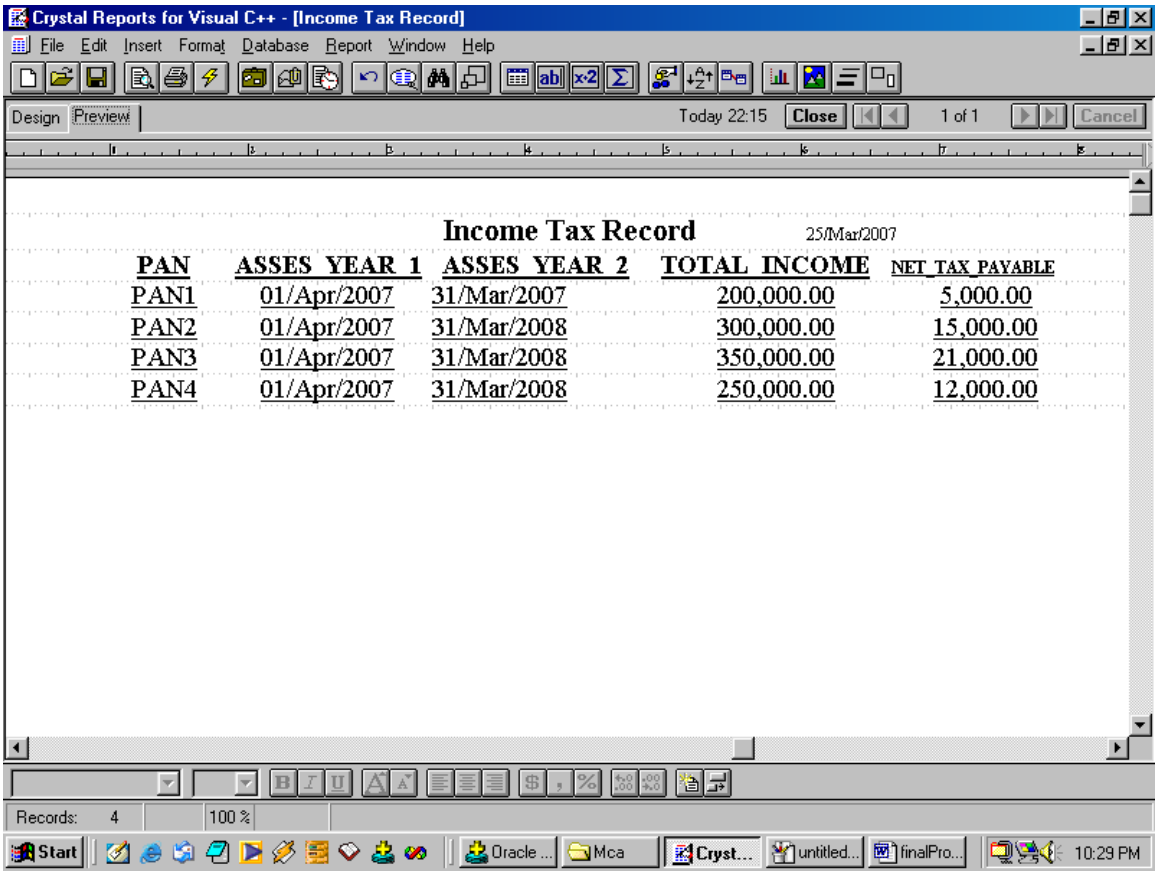
Here we include some Output of Report which are saved through Keyboard

<Print Screen> Command.

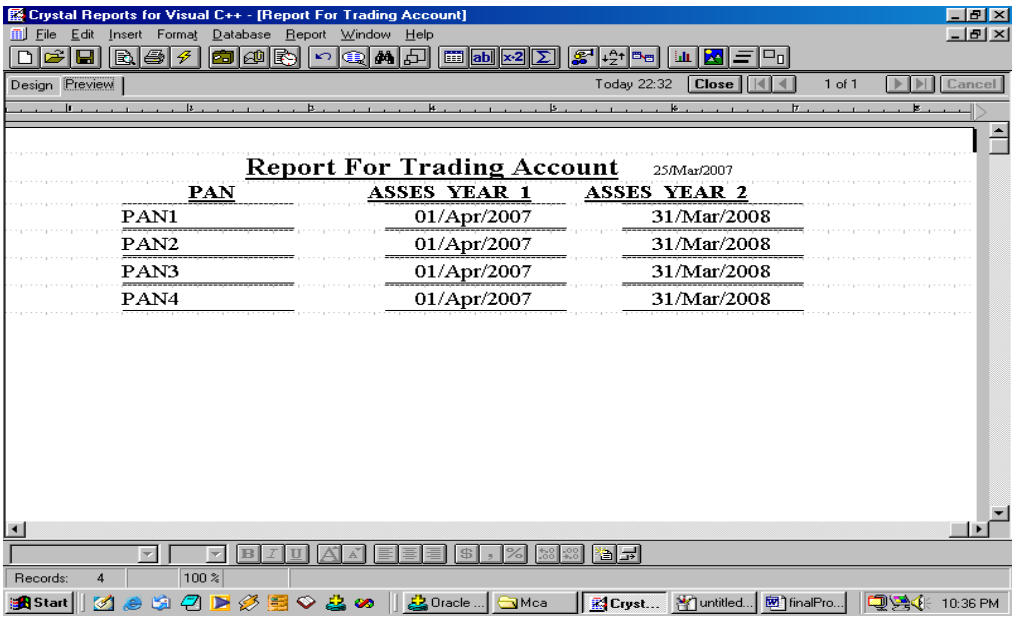
Report for Client record is as follows: -



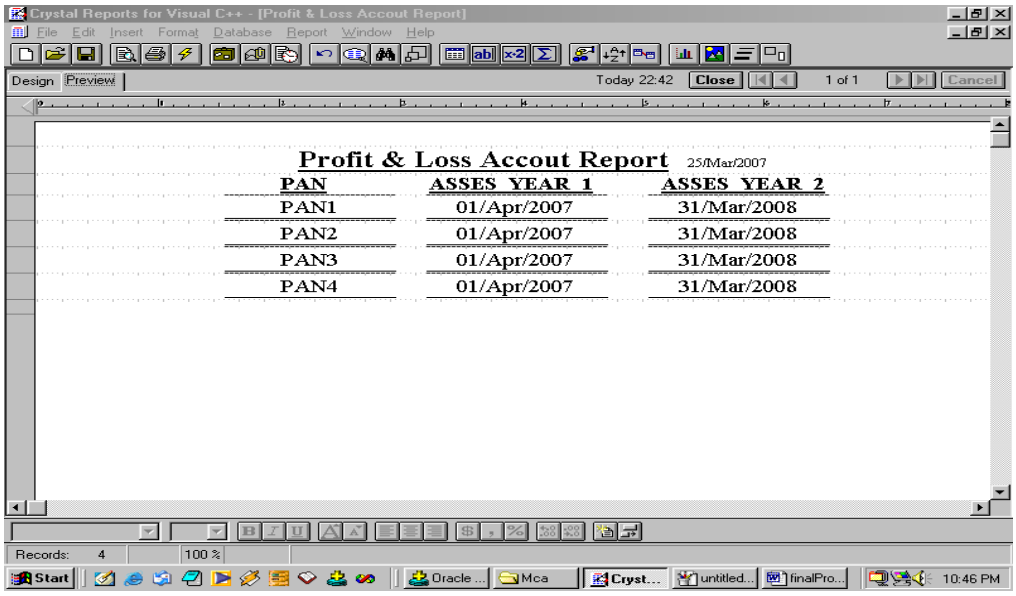
Report for Income Tax Record is as follows: -



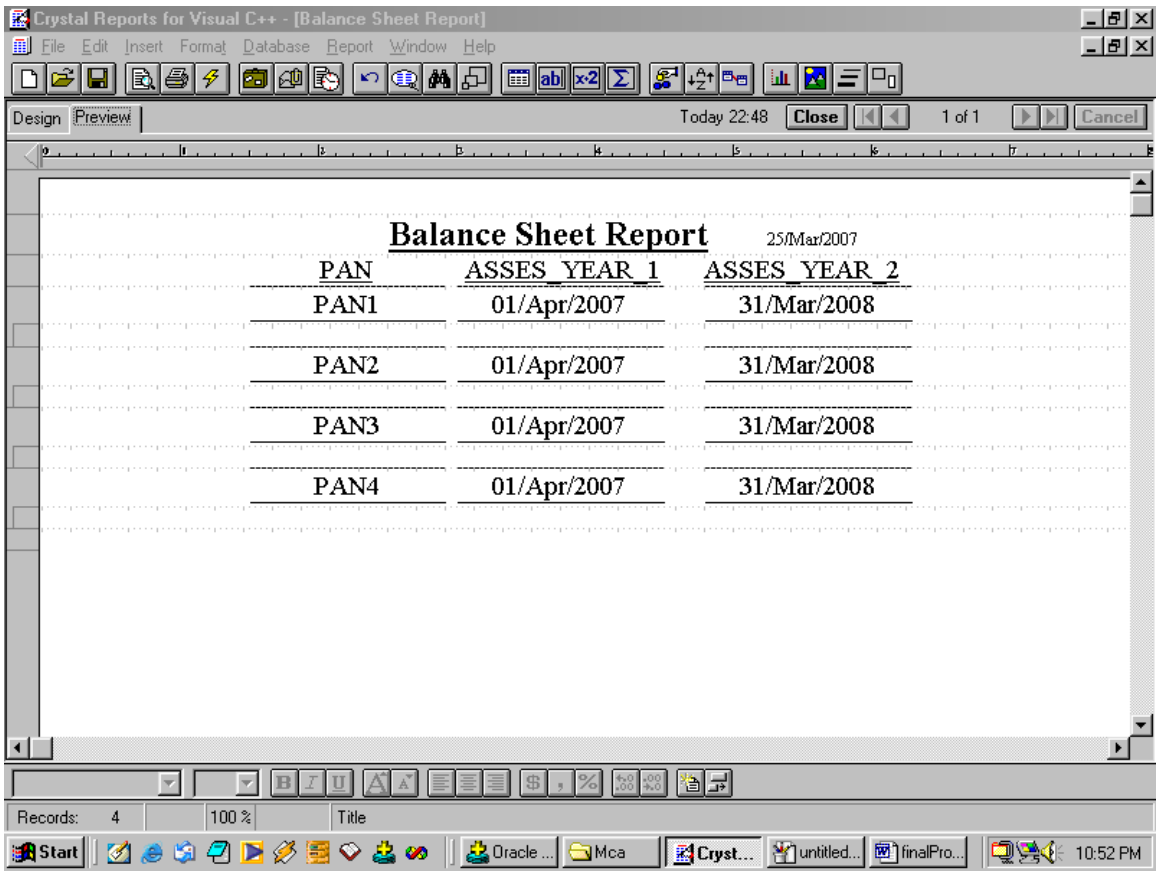
Report for Trading Account is as followed: -



Report for Profit & Loss Account is as follows: -



Report for Balance Sheet is as follows: -



18. PERT Chart:

PERT stands for program evaluation review technique. Unlike bar charts, PERT can be both a cost and a time management system. PERT is organized by events and activities or tasks. PERT has several advantages over bar charts and is likely to be used with more complex projects. One advantage of pert is that it is scheduling device that also shows graphically which task must be completed before others are begun. Also by displaying the various task paths, PERT enables the calculation of critical path. Each path consists of combinations of task, which must be completed. The time and cost associated with each task along a path are calculated and the path, which requires greatest amount of time, is critical path. Calculation of the critical path enables project managers to monitor this series of tasks more likely than others and to shift resources to it begins to fall behind schedule.

Pert controls time and costs during the project and also facilitates finding the right balance between completing it within the budget. Pert recognizes that projects are complex, that some task must be completed before others can be started. The appropriate way to manage a project is to define and control each task. Because project often falls behind schedule, pert is designed to facilitate getting a project back on schedule. Pert is based in part on the premise that subjective estimates of total completion time for a project usually greatly inferior to the sum of subjective estimates for each task. As with Gantt charts, to build a pert chart for a project, one must determine the dependence of the activities required for completion of the project and estimate how long each will take. Then one must determine the dependence of the activities on each other. In facts, the pert chart gives a graphical representation of this information. Clearly this technique does not help in deciding which activities are necessary or how long each will take, but it does force the manager to take the necessary planning steps to answer these questions.

Pert chart are available in next page:

Some of the advantages of pert chart:-

- ✓ It forces the manager to plan.
- ✓ It shows the interrelationships among the task in the project.
- ✓ It clearly identifies critical path of the project.
- ✓ It exposes all possible parallelism in the activities and thus helps in allocating resources.
- ✓ It allows scheduling and simulation of alternative schedules.
- ✓ It enables the manager to monitor and control the project.

19. Gantt Chart:

Gantt chart is also known as Bar chart. ***Hennery L. Gantt*** has developed Gantt chart. Gantt charts are a project control technique that can be used for several purposes, including scheduling, budgeting and resources planning. A Gantt chart is a bar chart, with each bar representing an activity. The bars are drawn against a time line. The length of each bar is proportional to the length of time planned for the activity. A Gantt charts help in scheduling the activities of a project, but it does not help in identifying them. One can begin with activities identified in work breakdown structure as we did below for quality management project during the scheduling activity, and also during implementation of the project quality management system, new activities may be identified that were not envisioned during the initial planning. The project manager must then go back and revise the breakdown structure and the schedules to deal with news activities. The Gantt chart in the figure is actually an enhanced version of standard Gantt charts. In below figure of Gantt chart, horizontal bars have been shown the duration of actions or tasks. The white bar shows the

Slack time that is, latest time by which a task must be finished. Gantt chart, in which, each bar would be represented on of the engineers. Gantt charts are useful for resource, planning and scheduling. When a bar chart is used as a project control method, milestones or checkpoints usually are placed at the completion of each task. They indicate the completion of a particular task and are the basis for determining whether the task and the project are on schedule. Reviewer can ask whether resources allocated have been properly utilized and whether the task has been satisfactorily completed. However, because the bar chart incorporates only the scheduling dimension of a project, it gives little indication of which tasks must be completed before others are begun, and projects cost must be accumulated and evaluated using other method.

For convenient the Gantt charts are available in next page.

20. Future Scope of the Project:

As in this project I am going to concentrate mostly on the return record regarding firms along with their all concerned enclosures. So, obviously future scope as well as future enhancement of this project is luminous, in its improved version all other categories of return filer can be included. At the same time, introduction of network in this project will make it universal in the sense to make it available to all the users, residing at distant.

21. Bibliography

(1) Oracle8 PL/SQL Programming - SCOTT URMAN (TATA McGRAW-HILL EDITION)

(2) An Introduction To Database System -BIPIN C. DESAI (Galgotia Publications Pvt Ltd.)

(5) Visual C++ 6– Pappas Murray (TATA MCGRAW HILL)

(6) CS-05 Element Of System Analysis And Design (I.G.N.O.U)

(7) CS-06 Introduction To Database Management Systems (I.G.N.O.U)

(8) Software Engineering –Roger S. Pressman (McGraw-HILL EDITION)

(9) An Integrated Approach to Software Engineering –Pankaj Jalote (Narosa Publishing House)

22. Appendix A:

SQL For Creating User:

In ORACLE, for creating new user, one must be login through SYSTEM User ID.

The command, through which we create a new user “digamber”, is written below.

The password for this User ID is “digamber”. The whole command from Appendix A to D

is original and spooled by ORACLE SPOOL Command.

```
SQL> create user digamber identified by digamber;

User created.

SQL> grant create session, resource to digamber;

Grant succeeded.

SQL> connect digamber/digamber;
```
Connected.

23. Appendix B:

SQL For Creating Database:

The whole database is created under User ID “digamber”. The command for creating Database is written below.

Command for CLIENT_RECORD:

```
CREATE TABLE CLIENT_RECORD
  (
      PAN VARCHAR2(10) PRIMARY KEY, CLIENT_NAME VARCHAR2(30),
         FATHERS_NAME VARCHAR2(30), DOB DATE, PINCODE VARCHAR2(6),
      ADDRESS VARCHAR2(60), TELEPHONE VARCHAR2(15), SEX NUMBER(1),
      INDV_HUF_FIRM_AOP_LA NUMBER(1), RESIDENT_NR_NOR NUMBER(1),
      WARD_CIRCLE_SPECIAL_RANGE VARCHAR2(20)
   )
```

Command for INCOME_TAX_RECORD:

```
CREATE TABLE INCOME_TAX_RECORD
  (
      PAN VARCHAR2(10) CONSTRAINT FK_INCOME_TAX_RECORD REFERENCES
CLIENT_RECORD (PAN),
      ASSES_YEAR_1 DATE, ASSES_YEAR_2 DATE, INCOME_FOR_PREV_YEAR
NUMBER(15, 2),
      RETURN_ORIGINAL_REVISED NUMBER(1), INCOME_FROM_SALARY NUMBER(15,
2),
      INCOME_FROM_HOUSE_PROPERTY NUMBER(15, 2), INC_FROM_BUS_OR_PROF
NUMBER(15, 2),
      TOTAL_SHORT_TERM_GAIN NUMBER(15, 2), TOTAL_LONG_TERM_GAIN
NUMBER(15, 2),
      TOTAL_CAPITAL_GAINS NUMBER(15, 2), INCOME_FROM_OTHER_SOURCES
NUMBER(15, 2),
      INCM_OTHER_PERS_TO_ADDED NUMBER(15, 2), GROSS_TOTAL_INCOME
NUMBER(15, 2),
      LESS_DEDUCTION_UNDER_VIA NUMBER(15, 2), TOTAL_INCOME NUMBER(15,
2),
      AGRICULTURAL_INCOME NUMBER(15, 2), INCOME_TO_BE_EXEMPTED
NUMBER(15, 2),
```

```
      INCOME_AT_NORM_RATE NUMBER(15, 2), INCOME_TAX_NORM_RATE
NUMBER(15, 2),
      INCOME_AT_SPCL_RATE NUMBER(15, 2), INCOME_TAX_SPCL_RATE
NUMBER(15, 2),
      TAX_ON_TOTAL_INCOME NUMBER(15, 2), LESS_REBATE NUMBER(15, 2),
      TAX_PAYABLE NUMBER(15, 2), ADDITIONAL_SURCHARGE NUMBER(15, 2),
      TOTAL_TAX_PAYABLE NUMBER(15, 2), LESS_RELIEF NUMBER(15, 2),
NET_TAX_PAYABLE NUMBER(15, 2),
      TAX_DEDUC_AT_SOURCE NUMBER(15, 2), ADVANCE_TAX_PAID NUMBER(15,
2),
      INTEREST_PAYABLE NUMBER(15, 2), SELF_ASSESS_BEFORE_31_MAY
NUMBER(15, 2),
      SELF_ASSESS_AFT_31_MAY NUMBER(15, 2), TOT_SELF_ASSESS_TAX_PAID
NUMBER(15, 2),
      BSR_CODE_OF_BANK_BRAN VARCHAR2(15), DATE_OF_DEPOSIT DATE,
      SERIAL_NO_OF_CHALLAN VARCHAR2(10), AMOUNT NUMBER(15, 2),
      NAME_OF_BANK_FOR_CHALLAN VARCHAR2(20),
      BAL_TAX_PAYABLE_REFUND NUMBER(15, 2), BALANCE_TAX NUMBER(15, 2),
      ENCLOSURE1 VARCHAR2(20), ENCLOSURE2 VARCHAR2(20), ENCLOSURE3
VARCHAR2(20),
      ENCLOSURE4 VARCHAR2(20), ENCLOSURE5 VARCHAR2(20), ENCLOSURE6
VARCHAR2(20)
 )
```

Command for enq_TRADING_ACCOUNT:

```
CREATE TABLE TRADING_ACCOUNT
     (
      PAN Varchar2(10), ASSES_YEAR_1 Date, ASSES_YEAR_2       Date,
            TO_OPENING_STOCK Number(15,2), TO_STOCK Number(15,2),
TO_PURCHASES Number(15,2),
            TO_CARRIAGE Number(15,2), TO_OCTROI Number(15,2),
            TO_IMPORT_DUTY_CUSTOMS Number(15,2), TO_WAGES Number(15,2),
            TO_COAL_WATER_GAS Number(15,2), TO_HEATING_LIGHTING_POWER
Number(15,2),
            TO_MANU_ASSEM_EXPEN Number(15,2), TO_CONSUMABLE_STORES
Number(15,2),
            TO_DRCT_FACT_PROD_EXP Number(15,2), TO_ROYALTY
Number(15,2),
            TO_GROSS_PROFIT Number(15,2), BY_SALES Number(15,2),
            BY_CLOSING_STOCK Number(15,2), BY_STOCK Number(15,2),
            BY_GROSS_LOSS   Number  (15,2),
            CONSTRAINT FK_TRADING_ACCOUNT FOREIGN KEY (PAN) REFERENCES
CLIENT_RECORD(PAN),
            CONSTRAINT PK_TRADING_ACCOUNT PRIMARY KEY(PAN,
ASSES_YEAR_1, ASSES_YEAR_2)
 )
```

Command for PL_ACCOUNT:

```
CREATE TABLE PL_ACCOUNT
  (
      PAN Varchar2(10), ASSES_YEAR_1 Date, ASSES_YEAR_2 Date,
      TO_GROSS_LOSS Number(15,2), TO_SALARIES_WAGES Number(15,2),
      TO_OFFICE_GODOWN_RENT Number(15,2),
      TO_OFFICE_EXPENSES Number(15,2),
      TO_MISC_SUNDRY_EXPENSE Number(15,2), TO_INSURANCE Number(15,2),
```

```
      TO_STATION_PRINT Number(15,2), TO_STAFF_WELF_EXPENSE
Number(15,2),
      TO_LIGHT_WATER_ELECT Number(15,2), TO_ESTAB_EXPENSE Number(15,2),
      TO_POST_TLGRM_FAX_COUR_PH Number(15,2),
      TO_LAW_CHARGES Number(15,2), TO_REPAIRS Number(15,2),
      TO_DISTRIBUTION_EXPENSES Number(15,2), TO_TRAVEL_EXPENSE
Number(15,2),
      TO_GENERAL_EXPENSES Number(15,2), TO_STABLE_EXPENSES
Number(15,2),
      TO_SELLING_EXPENSES Number(15,2), TO_CARRIAGE_OUTWARD
Number(15,2),
      TO_CARRIAGE_ON_SALES Number(15,2), TO_INDIRECT_WAGES
Number(15,2),
      TO_AUDIT_FEES Number(15,2), TO_ENTERTAIN_EXPENSES Number(15,2),
      TO_INTEREST_PAID Number(15,2), TO_DISCOUNT_ALLOWED Number(15,2),
      TO_BAD_DEBTS Number(15,2), TO_RESERVE_FOR_BAD_DEBTS Number(15,2),
      TO_DEPRECIATION Number(15,2), TO_INTEREST_ON_CAPITAL
Number(15,2),
      TO_DISCOUNTING_CHARGES Number(15,2), TO_BANK_CHARGES
Number(15,2),
      TO_EXPORT_CHARGES Number(15,2), TO_TRADE_EXPENSES Number(15,2),
      TO_ADMIN_EXPENSES Number(15,2), TO_FINANCIAL_EXPENSES
Number(15,2),
      TO_COMMISSION_PAID Number(15,2),    TO_ADVERTISEMENT
Number(15,2),
      TO_CHARITY Number(15,2), TO_SAMPLE_EXPENSES Number(15,2),
       TO_LICENCE_FEE Number(15,2), TO_DELIVERY_CHARGES Number(15,2),
      TO_BROKERAGE Number(15,2), TO_SALES_TAX Number(15,2),
      TO_LOSS_ON_SALE_OF_ASSET Number(15,2), TO_LOSS_BY_THEFT_ACCIDENT
Number(15,2),
      TO_NET_PROFIT Number(15,2), BY_GROSS_PROFIT Number    (15,2),
BY_INTEREST_RECEIVED Number(15,2),
      BY_RENT_RECEIVED Number(15,2), BY_DISCOUNT_RECEIVED Number(15,2),
      BY_DIVIDENDS_RECEIVED Number(15,2), BY_PROF_FROM_SALE_OF_ASSET
Number(15,2),
      BY_REFUND_OF_TAX Number(15,2), BY_COMPENSAT_RECEIVED
Number(15,2),
      BY_APPRENTICESHIP_PREMIUM Number(15,2), BY_DIFFER_IN_EXCHANGE
Number(15,2),
      BY_INTEREST_ON_DRAWINGS Number(15,2),
      BY_DISCOUNT_ON_CREDITORS Number(15,2), BY_BAD_DEBTS_RECOVERED
Number(15,2),
      BY_MISC_RECEIPTS Number(15,2), BY_INC_IN_VALUE_OF_ASSET
Number(15,2),
      BY_INCOME_FROM_INVESTMENT Number(15,2), BY_RES_FOR_BAD_DOUBTS
Number(15,2),
      BY_NET_LOSS Number(15,2),
      CONSTRAINT FK_PL_ACCOUNT FOREIGN KEY (PAN) REFERENCES
CLIENT_RECORD(PAN),
      CONSTRAINT PK_PL_ACCOUNT PRIMARY KEY(PAN, ASSES_YEAR_1,
ASSES_YEAR_2)
 )

--------------------------------------------------------------------------------
```

Command for BALANCE_SHEET:

```
CREATE TABLE BALANCE_SHEET
    (
      PAN Varchar2(10), ASSES_YEAR_1 Date, ASSES_YEAR_2 Date,
BILLS_PAYABLE Number(15,2),
      SUNDRY_CREDITORS Number(15,2), LOANS Number(15,2),
      OUTSTANDING_EXPENSES Number(15,2), CAPITAL Number(15,2),
NET_PROFIT Number(15,2),
      INTEREST_ON_CAPITAL Number(15,2), DRAWINGS Number(15,2), NET_LOSS
Number(15,2),
      INCOME_TAX Number(15,2), CASH_IN_HAND Number(15,2), CASH_AT_BANK
Number(15,2),
      INVESTMENTS Number(15,2), BILLS_RECEIVABLE Number(15,2),
SUNDRY_DEBTORS Number(15,2),
      CLOSING_STOCK Number(15,2), STORES Number(15,2),
PLANT_AND_MACHINERY Number(15,2),
      FREEHOLD_PREMISES Number(15,2), UNEXPIRED_EXPENSES Number(15,2),
GOODWILL Number(15,2),
      CONSTRAINT FK_BALANCE_SHEET FOREIGN KEY (PAN) REFERENCES
CLIENT_RECORD(PAN),
      CONSTRAINT PK_BALANCE_SHEET PRIMARY KEY(PAN, ASSES_YEAR_1,
ASSES_YEAR_2)
 )
```

24. Appendix C:

List of ASCII Code used in program to validate the “Text box”, “Combo box”, and

“Dtpicker” control. It is used under “Keypress” event of corresponding control.

KEY	ASCII CODE
Back space	8
Return Key	13
Space	32
Comma (,)	44
Minus (-)	45
Dot (.)	46
Forward slace (/)	47
0 - 9	48 - 57
@	64
A - Z	65 – 90
a - z	97 - 122
Underscore (_)	95